

ROS

Robot Operating System

- Part 2 -

Cheolmin Jeong

강사 소개



■ 정철민 (Cheolmin Jeong)

■ 現 한양대학교 전기공학과 박사과정

■ 現 Unicore Robotics CTO

■ 한양대학교 지능형시스템제어 연구실 소속

(지도교수: 강창목 교수, <https://uniconlab.wixsite.com/main>)

■ 관심분야: 최적제어, 선형제어, 예측제어, 고장진단, 자율주행, SLAM

■ 적용플랫폼: 자율주행차량, 모바일로봇, 드론, 4족보행로봇 등

■ Projects:

- 자율주행 전동휠체어 핵심 기술 개발
- 순찰로봇의 도심지 적용을 위한 실외 자율주행 기술 개발
- 인천공항 폐기물 이송 로봇 개발
- UAV 기체 상태 이상 진단 검증 및 연계 모듈 개발
- 다목적 개인이동형 서비스 로봇 플랫폼 기술개발 및 실증
- 자율주행 전동휠체어 의료기기인증 및 표준안을 위한 기능 구현
- 온디바이스 AI 컴퓨팅 기반 AI 융합형 모빌리티 디바이스 기술 개발 및 실증
- ...



Contents

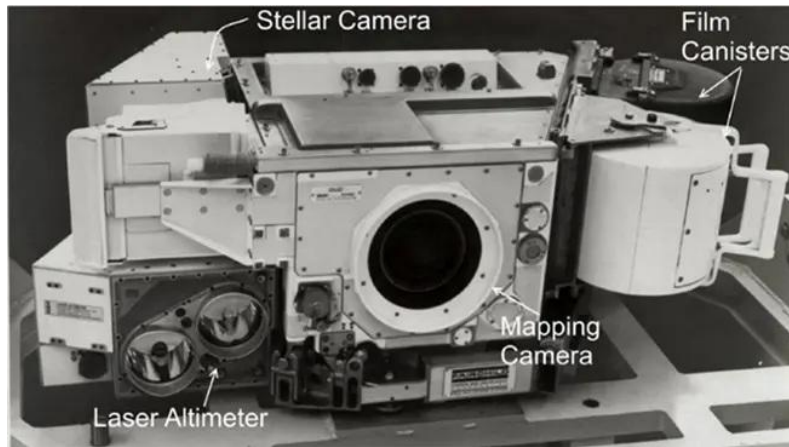
1. LiDAR 기술 이해
2. SLAM
3. Navigation
4. Advanced ROS Techniques

Appendix

LiDAR 기술 이해

1. LiDAR 기술 이해

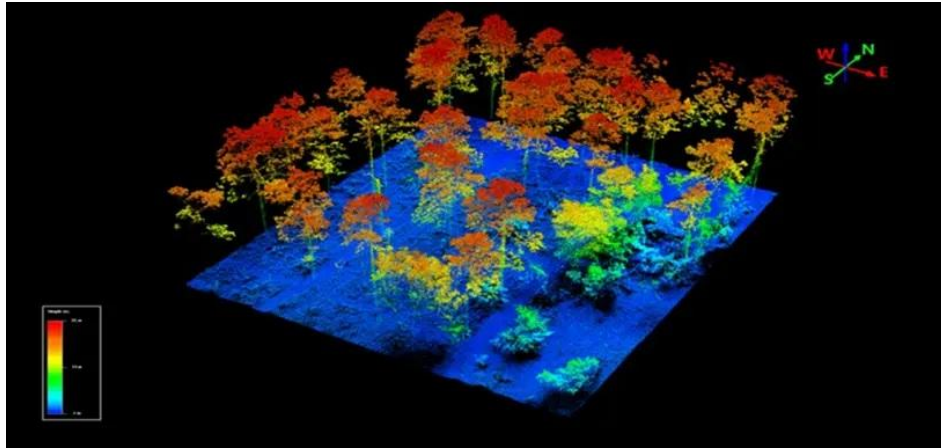
1) What is LiDAR?



- 1961년 레이저의 등장, 미국의 Hughes Aircraft Company는 레이저와 이미징 광학을 결합하여 레이저 신호가 대상에 도달하고 대상에서 돌아오는 데 걸리는 시간을 계산하여 대상의 거리를 측정하는 LIDAR와 유사한 장치를 개발한 최초의 회사였음.
- 원래 COLIDAR(Coherent Light Detecting and Ranging)라는 이름이 붙은 이 기술은 주로 위성 추적에 사용 되었음. LIDAR의 초기 응용 분야인 기상학으로 확장되어 구름과 대기 오염 감지에 도움.
- LIDAR가 널리 알려지게 된 것은 1971년 아폴로 15호 임무에 참여한 우주인들이 레이저 고도계를 사용하여 달 표면 위의 우주선 고도를 측정 했을 때.

1. LiDAR 기술 이해

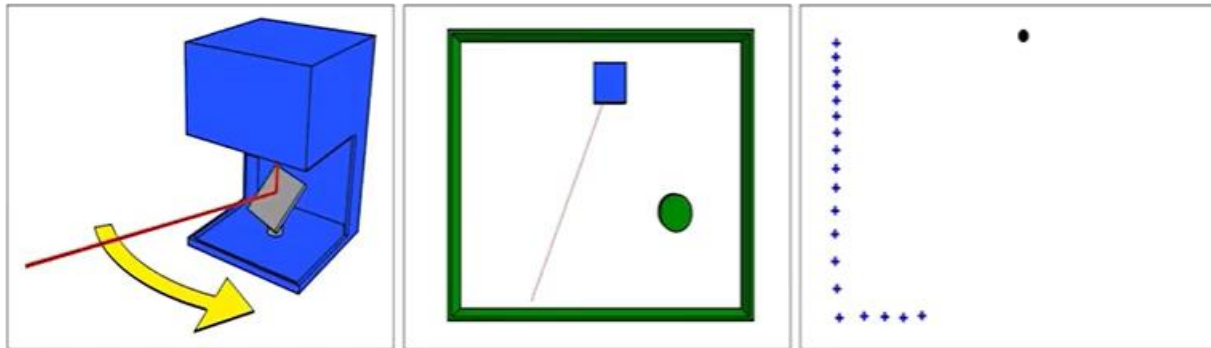
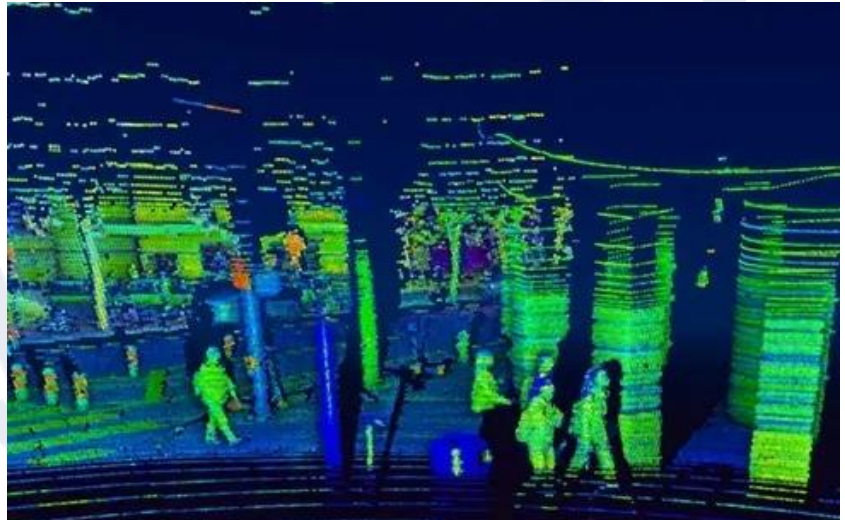
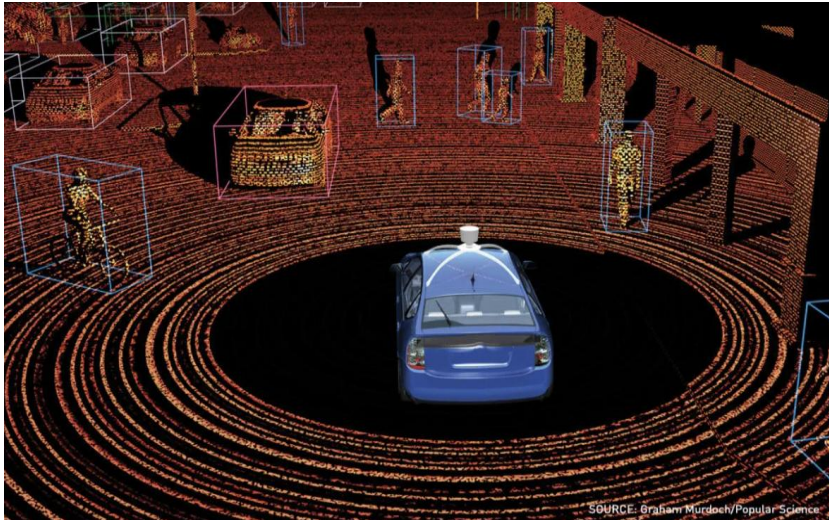
1) What is LiDAR?



- LiDAR는 물체를 향해 레이저 펄스를 방출한 다음 펄스의 일부를 센서로 다시 반사, 펄스가 돌아오는 데 걸리는 시간을 측정하여 물체의 위치를 계산.
- 1~100cm의 전파를 감지에 사용하는 RADAR 와 달리 LiDAR는 자외선(UV)에서 적외선(IR) 파장 스펙트럼(250nm~10μm)의 전자기파(EM)를 사용.
- LiDAR는 근거리에서 매우 정확한 측정이 가능.
- LiDAR는 일반적으로 매핑 및 측량에 사용되어 주변 환경의 자세한 3D 맵을 생성. GPS 데이터 및 관성 측정과 결합하면 측정된 스캔데이터는 정확한 영역을 표현하는 포인트 클라우드를 형성함.

1. LiDAR 기술 이해

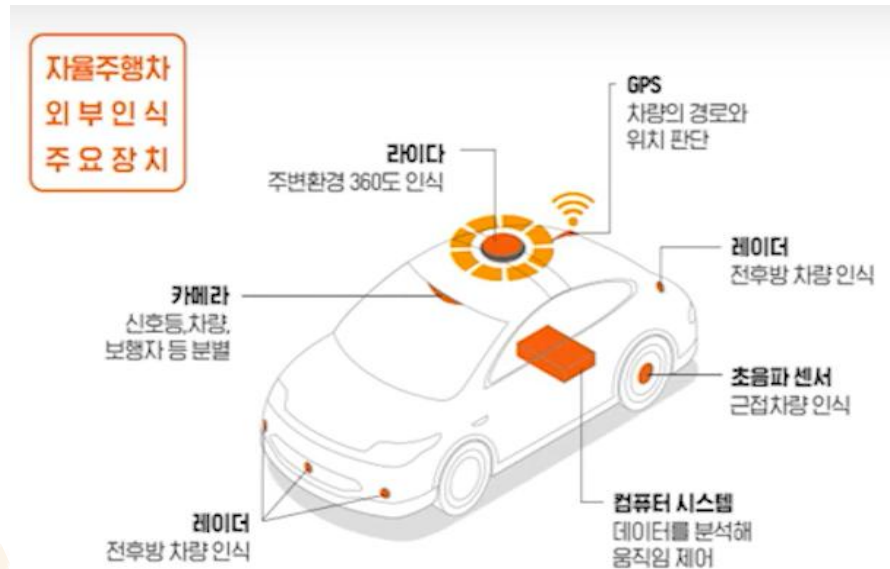
1) What is LiDAR?



- LiDAR(Light Detection And Ranging) 센서는 빛의 이동 시간 (Time of Flight)을 측정하는 원리를 통해 높은 정밀도의 3차원 공간 정보를 고속으로 획득할 수 있는 센서

1. LiDAR 기술 이해

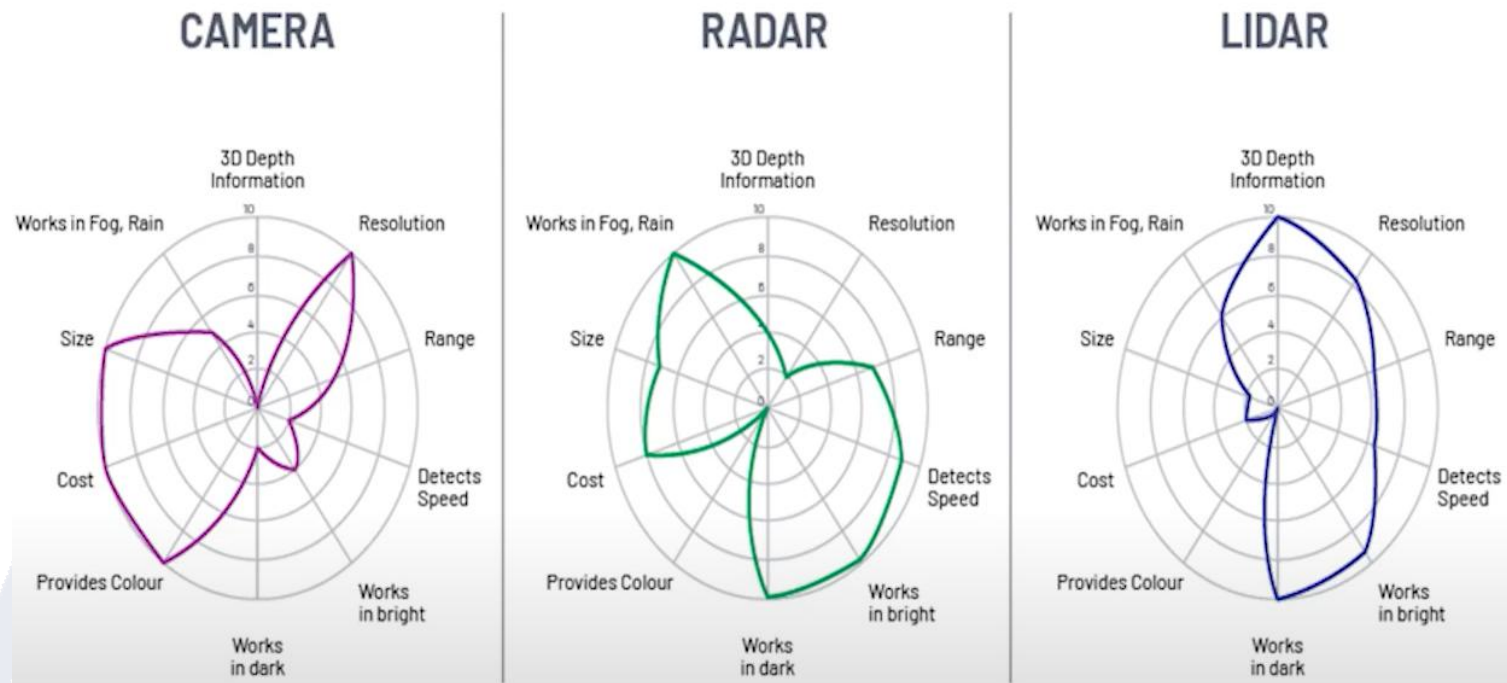
1) What is LiDAR?



- 자율주행 자동차가 안전하게 주행하기 위해서는 주변 환경을 인지해 주행에 필요한 정보를 제공하는 센서 시스템이 필수적
- 자율주행 자동차의 센서 시스템은 개별 센서의 기능, 비용, 디자인을 비롯한 여러 요인을 고려하여 최적으로 설계
- 각 센서의 장점과 단점을 상호 보완할 수 있도록 여러 센서 데이터를 조합하는 **센서 퓨전 (sensor fusion)** 기술은 자율주행 시스템의 신뢰성과 안정성을 높이는데 매우 중요한 기술

1. LiDAR 기술 이해

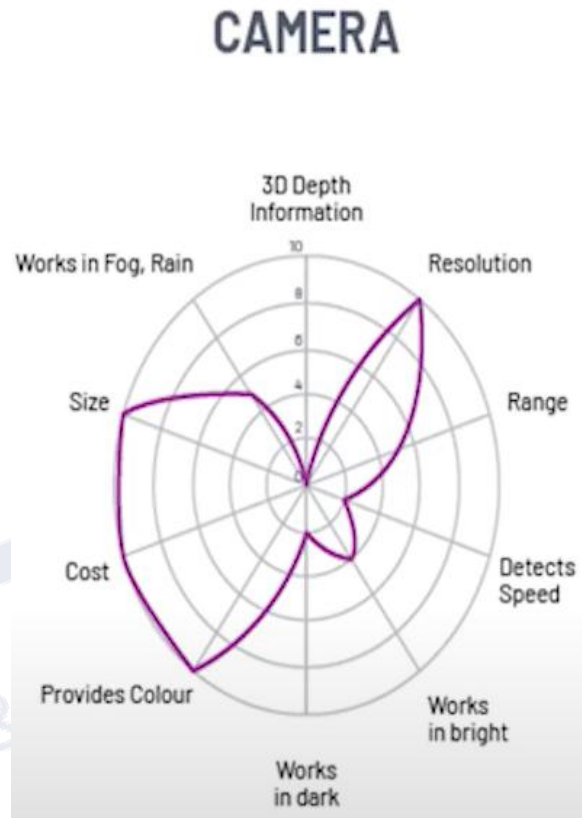
1) What is LiDAR?



- 자율주행 자동차의 안전한 주행을 위해 주변 환경을 인지하여 주행에 필요한 정보를 제공하는 센서 시스템이 필수적
- 주변 환경 인지를 위해 사용되는 대표적인 센서로는 Camera, Radar, LiDAR 등이 있으며, 각 센서 별 뚜렷한 장/단점이 존재

1. LiDAR 기술 이해

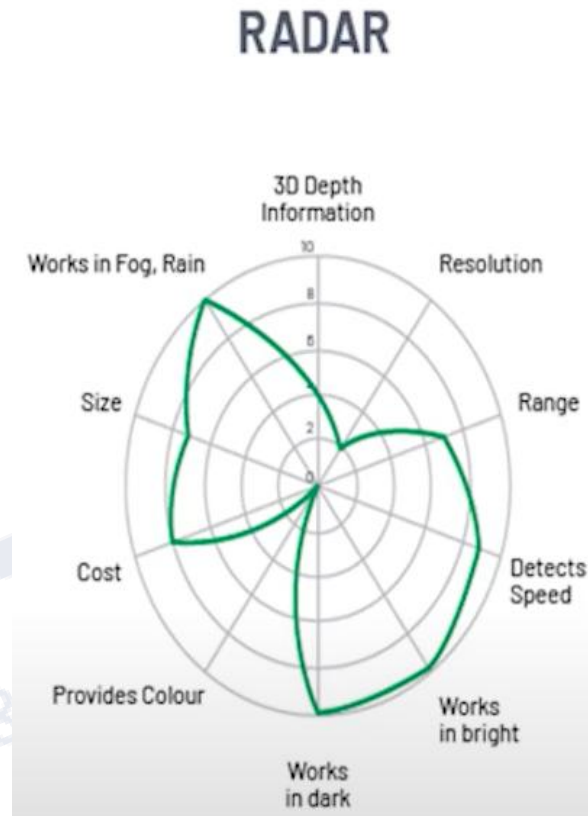
1) What is LiDAR?



- Camera 센서는 100만 화소급 이상의 고해상도 감지를 통해 감지 대상의 종류를 정확히 감지할 수 있지만, 감지 거리가 짧고 감지 대상의 속도 측정이 어려우며 악천후 환경 및 야간 감지 시 성능이 크게 떨어짐

1. LiDAR 기술 이해

1) What is LiDAR?



- Radar 센서는 감지 대상의 거리, 속도를 정확하게 측정할 수 있으며, 전자기파를 이용하기 때문에 야간, 악천후 상황에서 열화가 적지만, 낮은 해상도로 인해 객체 구분 및 인식은 제한적

1. LiDAR 기술 이해

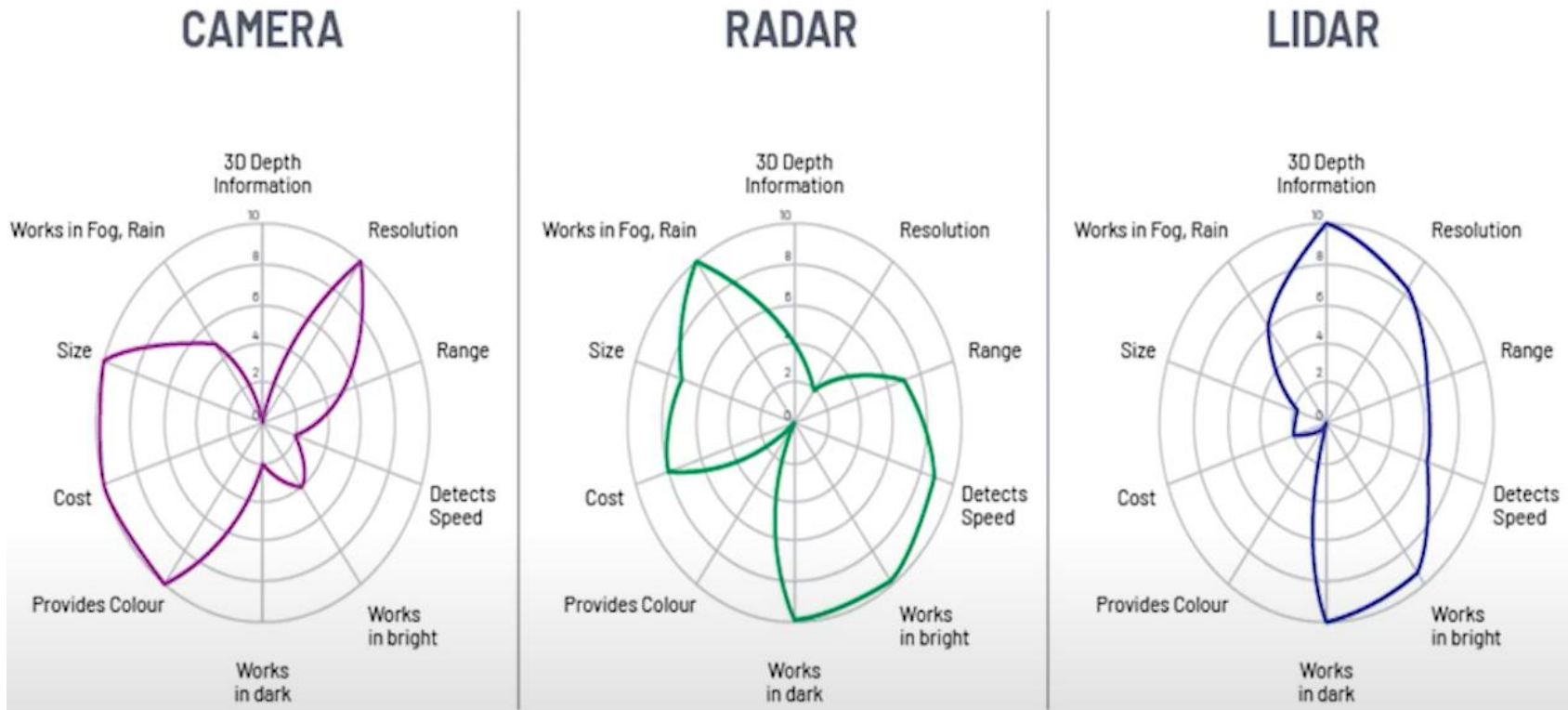
1) What is LiDAR?



- LiDAR 센서는 카메라 센서 대비 감지 거리가 뛰어나고 야간에도 감지 성능 열화가 전혀 없으며, 레이더 센서 대비 고해상도 감지 및 객체 인식이 가능함으로써 두 센서의 단점을 보완할 수 있음

1. LiDAR 기술 이해

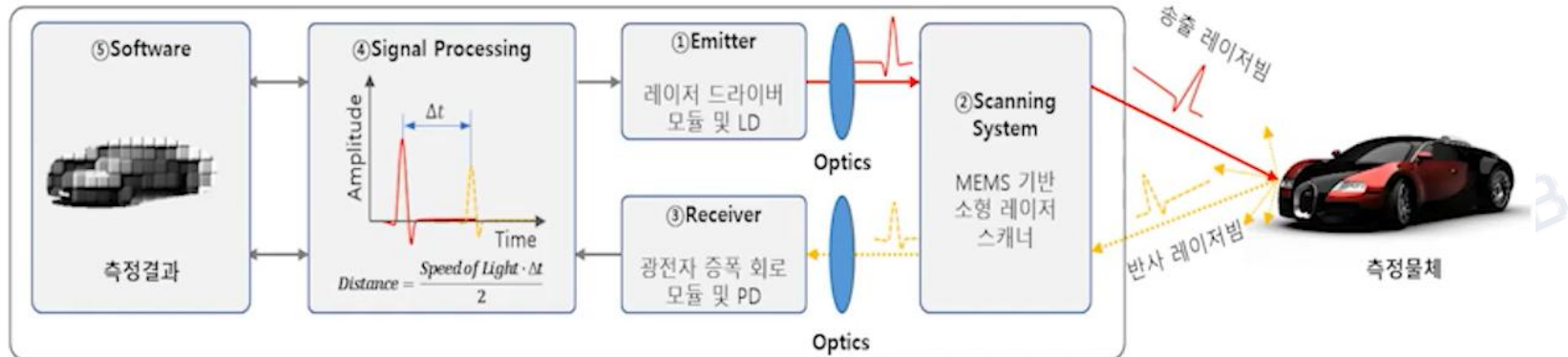
1) What is LiDAR?



- 따라서, 3단계 이상의 자율주행 기능 구현 시 전방 및 측/후방 감지 신뢰성 확보를 위해 기존의 Camera, Radar 센서와 함께 LiDAR 센서를 추가로 적용이 필요

1. LiDAR 기술 이해

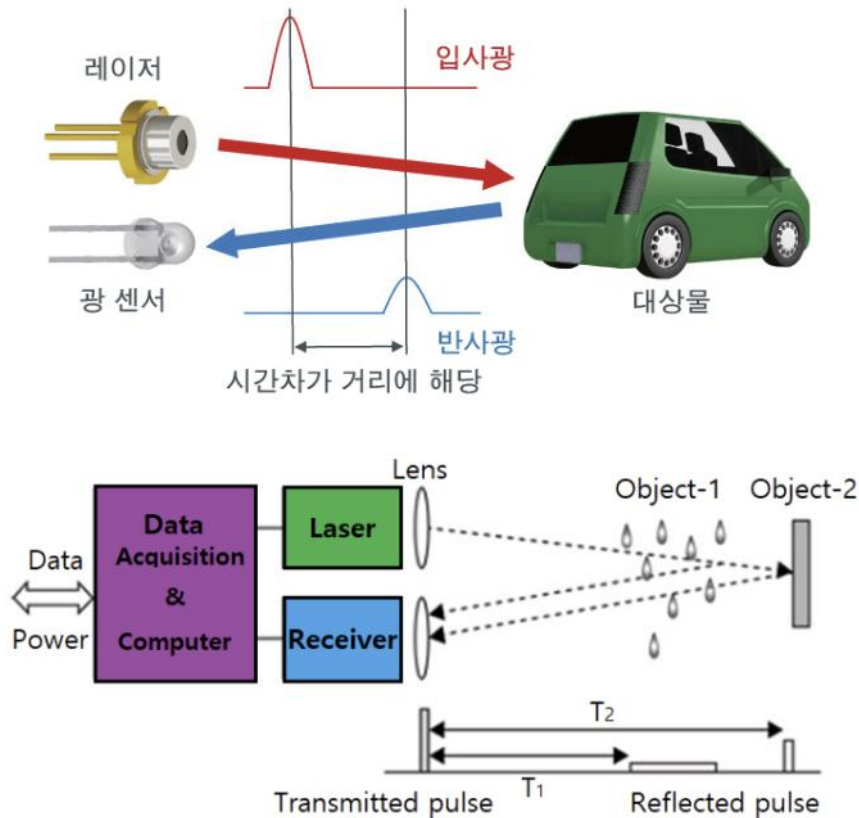
1) What is LiDAR?



- LiDAR 센서는 자율주행 자동차, 모바일 로봇, 드론 및 산업용 기기 등의 응용 분야에서 핵심 센서로 활용됨
- LiDAR 센서 시스템은 크게 측정 방식(measurement process), 발광 소자(emitter laser), 수광 소자(receiver photodetector), 빔 스티어링(beam steering)의 4가지 핵심 기술로 구성됨
- LiDAR 센서의 측정 방식으로는 비행 시간을 측정해 거리를 계산하는 ToF(Time-of-Flight) 방식과 주파수의 위상 변위를 이용해 거리를 측정하는 FMCW(Frequency Modulated Continuous Wave) 방식이 대표적으로 존재

1. LiDAR 기술 이해

2) LiDAR 센서의 원리



자료 출처: https://freecon.co.kr/?act=board&bbs_code=solu&bbs_mode=view&bbs_seq=22

$$d = \frac{c \times t}{2}$$

- d : 목표물까지의 거리
- c : 빛의 속도
(공기 중에서 대략 299,792,458 m/s)
- t : 레이저 빔이 목표물에 도달 후,
다시 돌아오는데 걸린 전체 시간

- LiDAR 센서는 레이저 빛을 조사한 뒤, 그 빛이 물체에 반사되어 돌아오는 반사광의 비행 시간을 측정하여 대상체까지의 거리를 측정하는 원리를 가짐

1. LiDAR 기술 이해

LiDAR 센서의 거리 측정 방식

- **ToF 측정 방식**은 송수신 소자 및 시스템 구성이 단순하여, 시스템 구현을 위한 기술 난이도가 낮고 저가로 구성이 가능하지만, 신호 처리를 통한 감지 성능(SNR) 향상에 한계가 있고, 짧은 시간에 고출력 펄스 형태로 광원을 수신해야 한다는 특징이 있음
- **FMCW 측정 방식**은 주파수 변조를 통해 일관된(Coherent) 방식의 신호 처리가 가능하며 감지 성능(SNR)을 극대화 할 수 있고, 도플러 효과를 통해 속도 검출이 가능하다는 특징이 있음. 그러나, 주파수 변조를 위해 시스템 구성이 복잡하고, 전장용 신뢰성 확보가 어려우며, 소형화 및 저가화에 한계가 있다는 특징이 있음.

1. LiDAR 기술 이해

LiDAR 센서의 거리 측정 방식

* ToF(time-of-flight): 비행 시간을 측정해 거리를 계산 방법

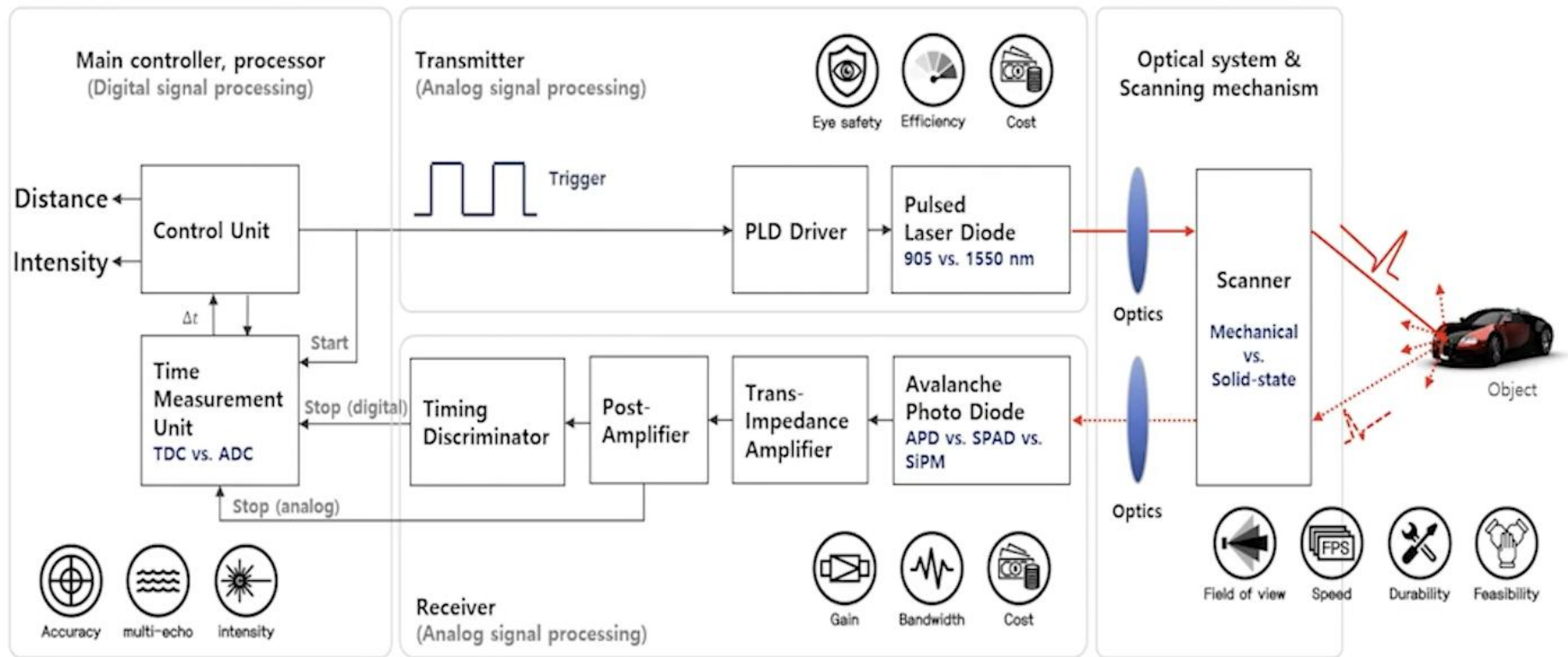
* FMCW(Frequency Modulated Continuous Wave): 주파수의 위상 변위를 이용해 거리를 측정하는 방법

거리 계산	원리	신호처리 (SNR 개선)	시스템 복잡도 및 가격	차량용 신뢰성	속도 계산
TOF*	단순 시간 계산	열세	우세	우세	열세
FMCW*	주파수(파장) 변조	우세	열세	열세	우세

Copyright © 2021 SOSLAB, Professional LiDAR Sensor Provider All rights reserved.

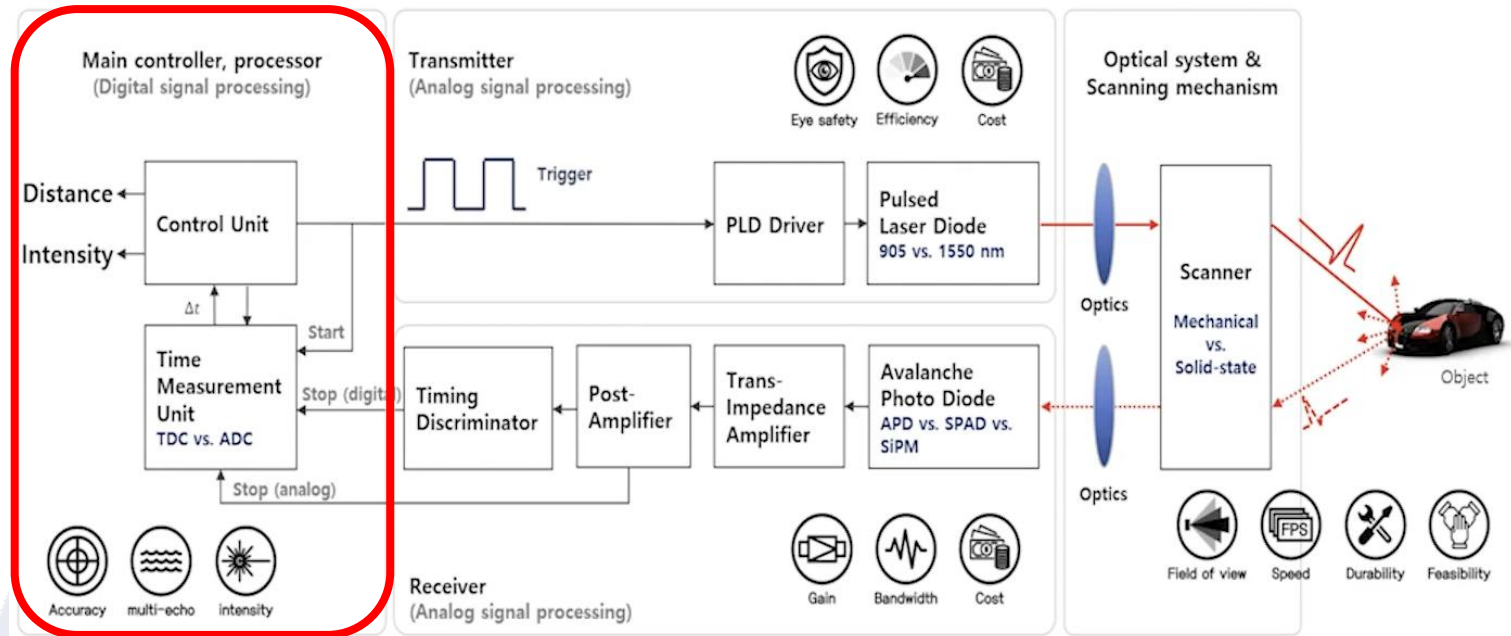
1. LiDAR 기술 이해

3) LiDAR 센서의 구조



1. LiDAR 기술 이해

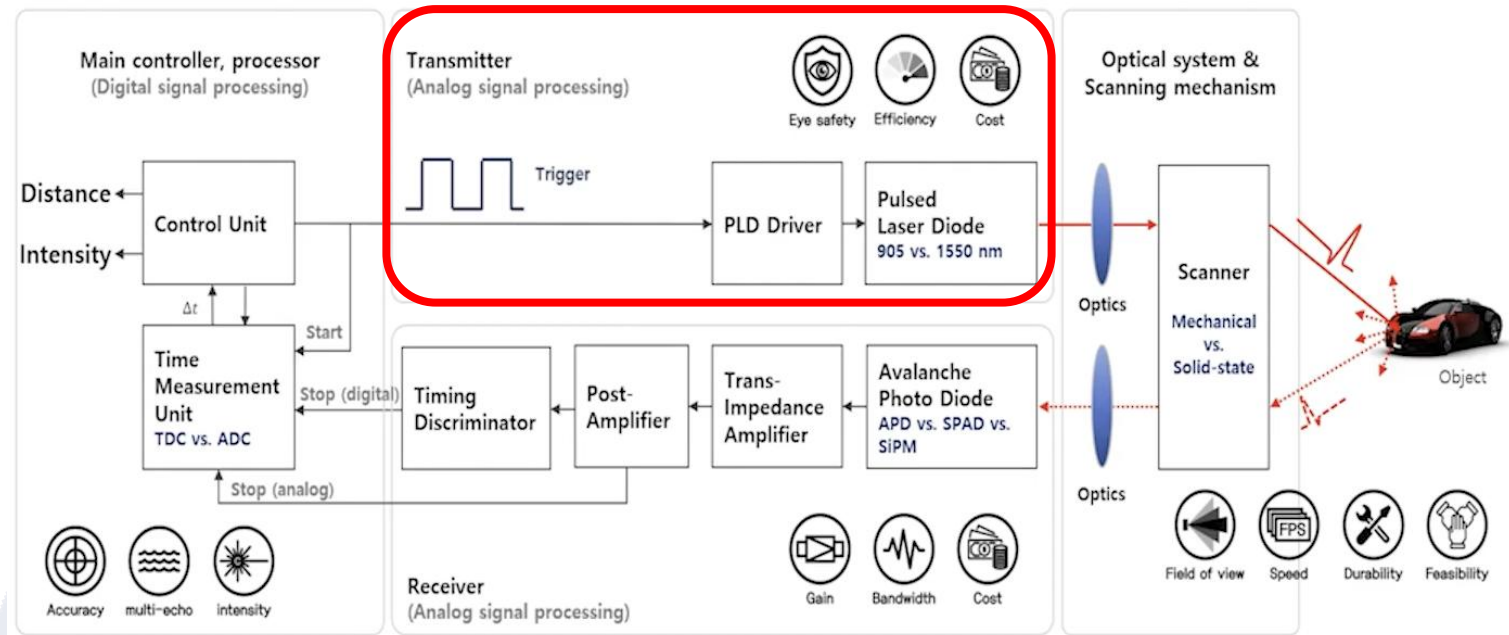
3) LiDAR 센서의 구조 – Main controller



- 역할: Main controller는 LiDAR 센서의 모든 작동을 제어하고, 데이터 처리를 관리하며, 시스템 간의 통신을 조정
- 기능: 시스템의 타이밍을 제어하고, 수신된 데이터를 분석하여 사용자가 원하는 형태로 데이터를 처리하는 역할을 수행

1. LiDAR 기술 이해

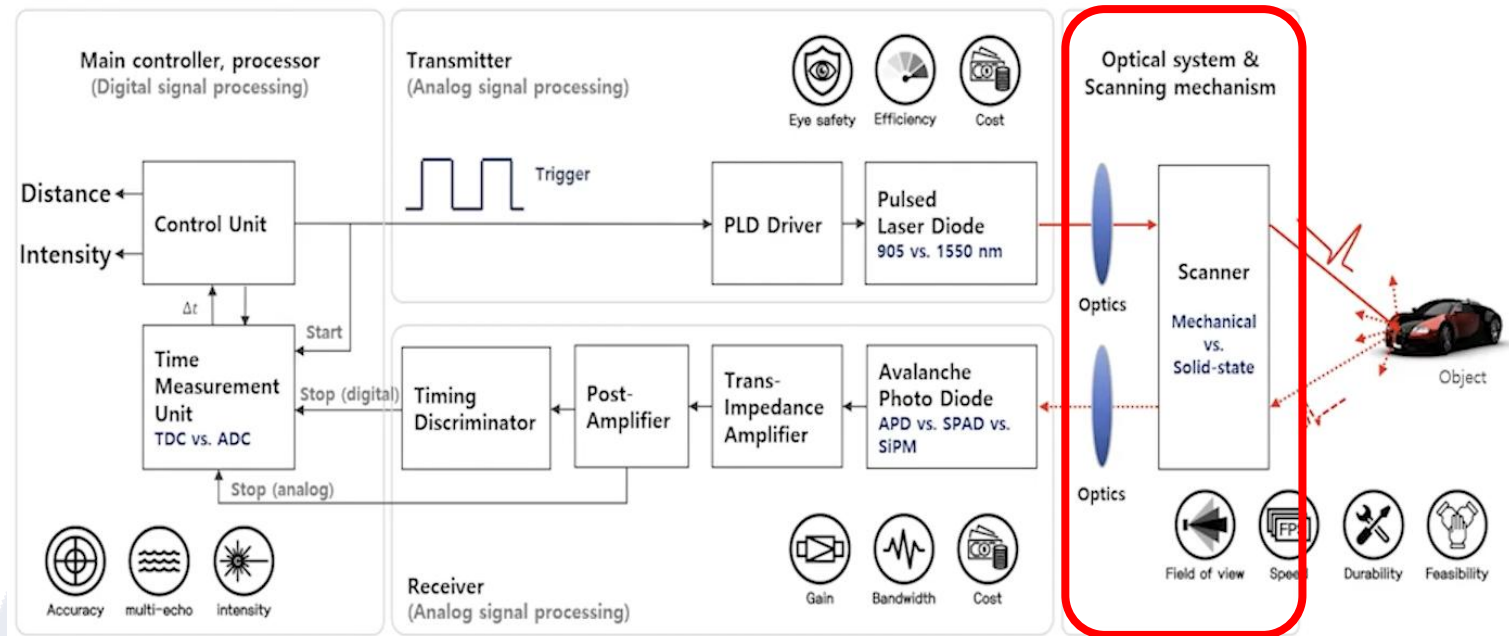
3) LiDAR 센서의 구조 – Transmitter(발신기)



- 역할: Transmitter는 발광 소자(emitter)를 통해 레이저 빔을 발사하여 물체에 도달하도록 하며, 물체에 반사되어 Receiver로 돌아오는 방식으로 거리를 측정
- 특성: 일반적으로 레이저 다이오드가 사용되며, 파장은 주로 적외선 범위에서 선택됨

1. LiDAR 기술 이해

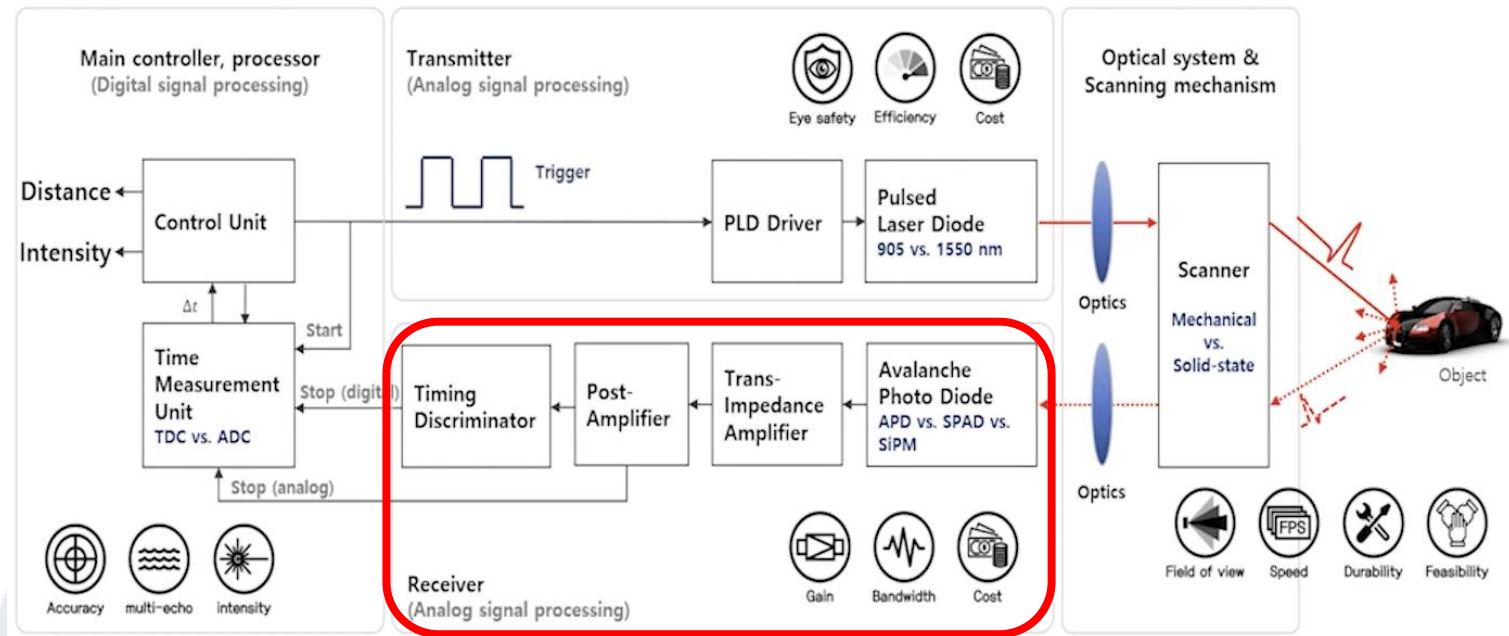
3) LiDAR 센서의 구조 – Scanner(스캐너)



- 역할: Scanner는 발사된 레이저 빔의 방향 조절을 수행하며, 이를 통해 LiDAR는 주변 환경을 360도 또는 필요한 각도만큼 스캔 가능. 이러한 기술을 ‘빔 스티어링 (Beam Steering)’이라고 함
- 기능: Scanner는 Spinning과 같은 기계적 회전 방식이나 MEMS Mirror와 같은 전자적 방식으로 구현될 수 있으며, 스캔 범위와 정밀도는 사용되는 Scanner의 종류에 따라 달라질 수 있음

1. LiDAR 기술 이해

3) LiDAR 센서의 구조 – Receiver(수신기)



- 역할: Receiver는 수광 소자(photodetector)를 통해 반사된 레이저 빔을 감지하고 수신하며, 해당 정보를 바탕으로 물체의 위치, 형태. 그리고 거리에 대한 계산이 가능
- 기능: Receiver는 고감도의 광학 센서를 사용하여 아주 약한 빛의 신호도 포착 가능

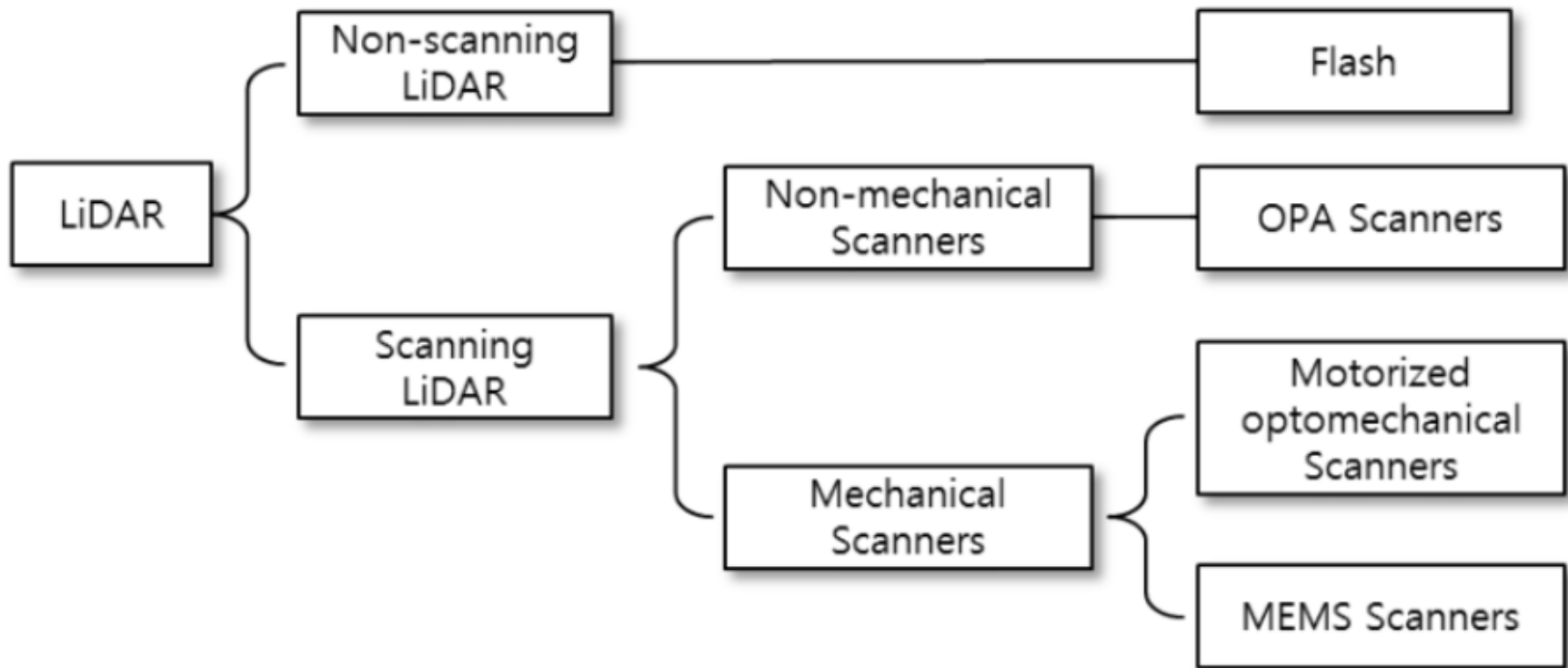
1. LiDAR 기술 이해

LiDAR 센서의 기술 구성 요소

- 발광 소자(emitter)는 목표 물체로 레이저 빛을 발사하는 역할을 수행
- 수광 소자(photodetector)는 반사되어 돌아오는 빛을 검출하는 역할을 수행
- 빔 스티어링(beam steering)은 발광 소자로부터 발사된 LiDAR 센서의 레이저 빛을 스캐너(Scanner)를 통해 특정 방향으로 정밀하게 조절하는 역할을 수행
- LiDAR 시스템에는 빛을 원하는 방향으로 조사할 수 있는 기계식(mechanical), 고정형(solid-state) 스캐너 장치들을 이용해 특정 공간 또는 목표 지점에 대해 주기적으로 측정을 반복하여 주변 환경에 대한 3차원 공간 정보를 획득

1. LiDAR 기술 이해

4) 스캐닝 방식에 따른 LiDAR 센서의 종류



자료 출처: <https://veganwithbacon.tistory.com/324>

1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – 기계식(mechanical) 스캐닝

- LiDAR 센서의 스캐닝 방식은 기계식 회전(mechanical) 방식과 고정형(solid-state) 방식으로 구분됨
- 기계식 회전 방식은 송수신 모듈을 모터에 장착하여 직접 회전하는 방식과, 송수신 모듈은 고정하고 반사 거울을 모터에 장착하여 스캐닝하는 방식이 대표적
- 두 방식 모두 시스템 구성에 필요한 기술 난이도가 상대적으로 낮기 때문에 현재 제품화 되어 판매 중인 LiDAR 센서는 모두 기계식 회전 방식
- 그러나 기계식 회전 방식의 LiDAR 센서는 스캐닝을 위한 모터 및 구조물이 추가되어 사이즈가 크고, 고가의 재료비가 필요하다는 한계점을 가짐

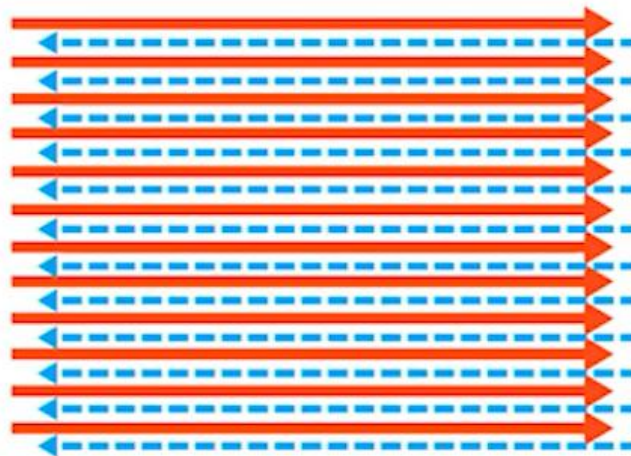
1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – 고정식(solid-state) 스캐닝

- 고정형 스캐닝 방식은 전자기적 공진 특성을 이용한 MEMS Mirror 스캐닝 방식과, 스캐너 없이 직접 송수신을 하는 Flash 방식으로 구분
- MEMS Mirror 방식은 스캐너 사이즈가 작고 상대적으로 저렴한 단가로 구성할 수 있지만, 기계적 회전 방식 대비 감지 성능이 떨어지며, 차량용 진동/내구 신뢰성에 취약함
- Flash 방식은 스캐너가 없기 때문에 사이즈, 단가, 신뢰성 측면에서 우수하지만, 감지 성능은 가장 떨어짐
- Optical Phase Array (OPA) 방식은 기술 성숙도가 낮아 자율주행 차량용으로는 고려하지 않음

1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – Spinning LiDAR 예시



광원/수광 모듈 세트 $\times$ N 개: N 채널 라이다



1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – Spinning LiDAR 예시



1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – Spinning LiDAR의 특징

장점

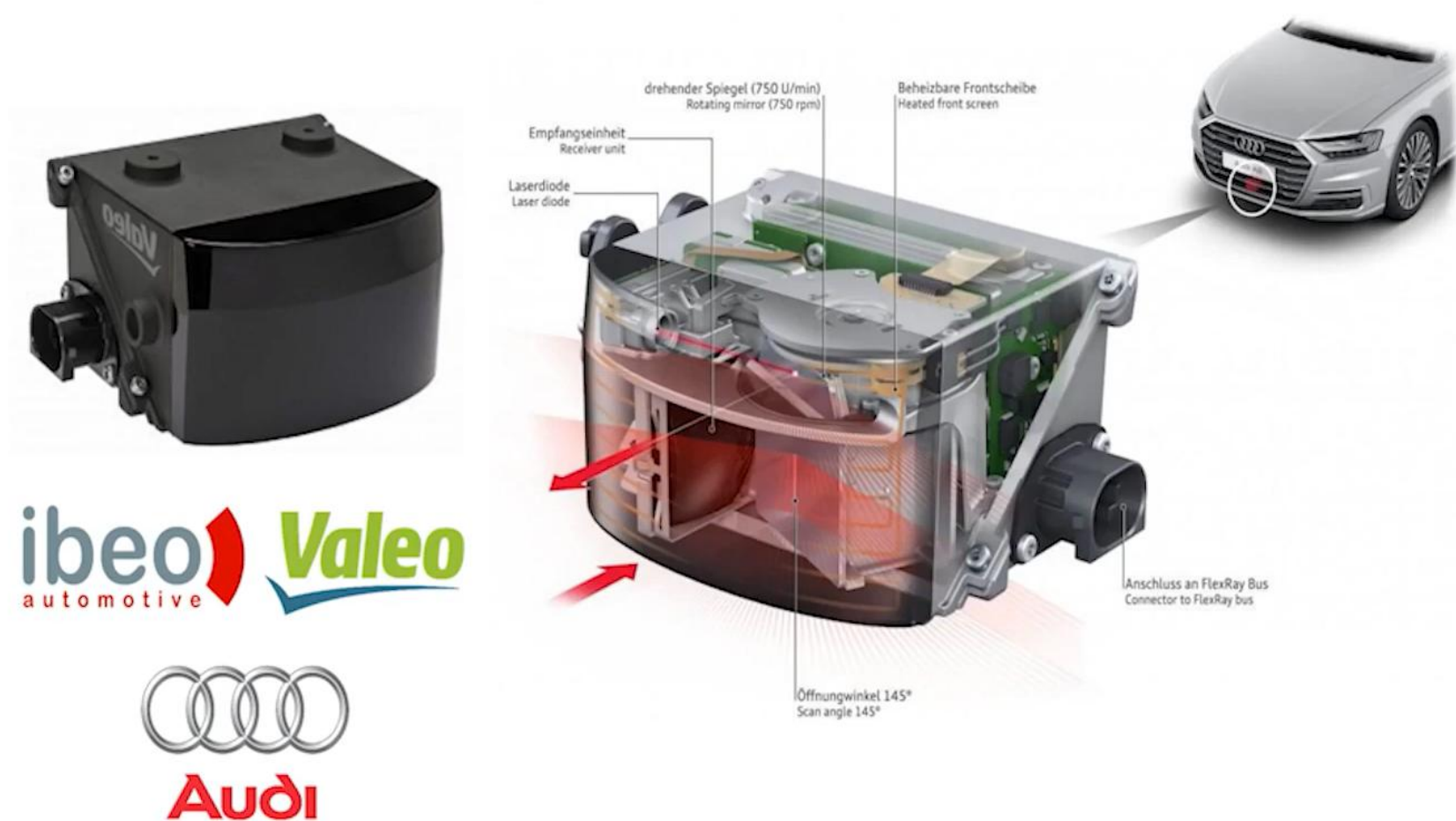
- 한 줄로 정렬된 레이저 소자들이 모터를 통해 360도 회전함으로써 **전방향에 대한 거리를 측정할 수 있어 고성능을 가짐**
- 한 줄에 레이저 소자를 16개 집적하는 경우 16채널, 32개를 배치하는 경우 32채널의 해상도로 분류됨

단점

- Spinning LiDAR는 많은 소자를 필요로 하므로 **시스템 구성 비용이 높아짐**
- 많은 소자를 필요로 하는 시스템의 특성으로 인해 **소형화가 어려움**
- 모터가 직접 회전하는 방식의 한계점으로 인해 **내구도가 쉽게 저하될 수 있음**
- 360도를 회전하는 LiDAR의 구조상, **자동차 범퍼 내 탑재 불가**

1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – Polygon Mirror Scanning LiDAR 예시



1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – Polygon Mirror Scanning LiDAR 특징



장점

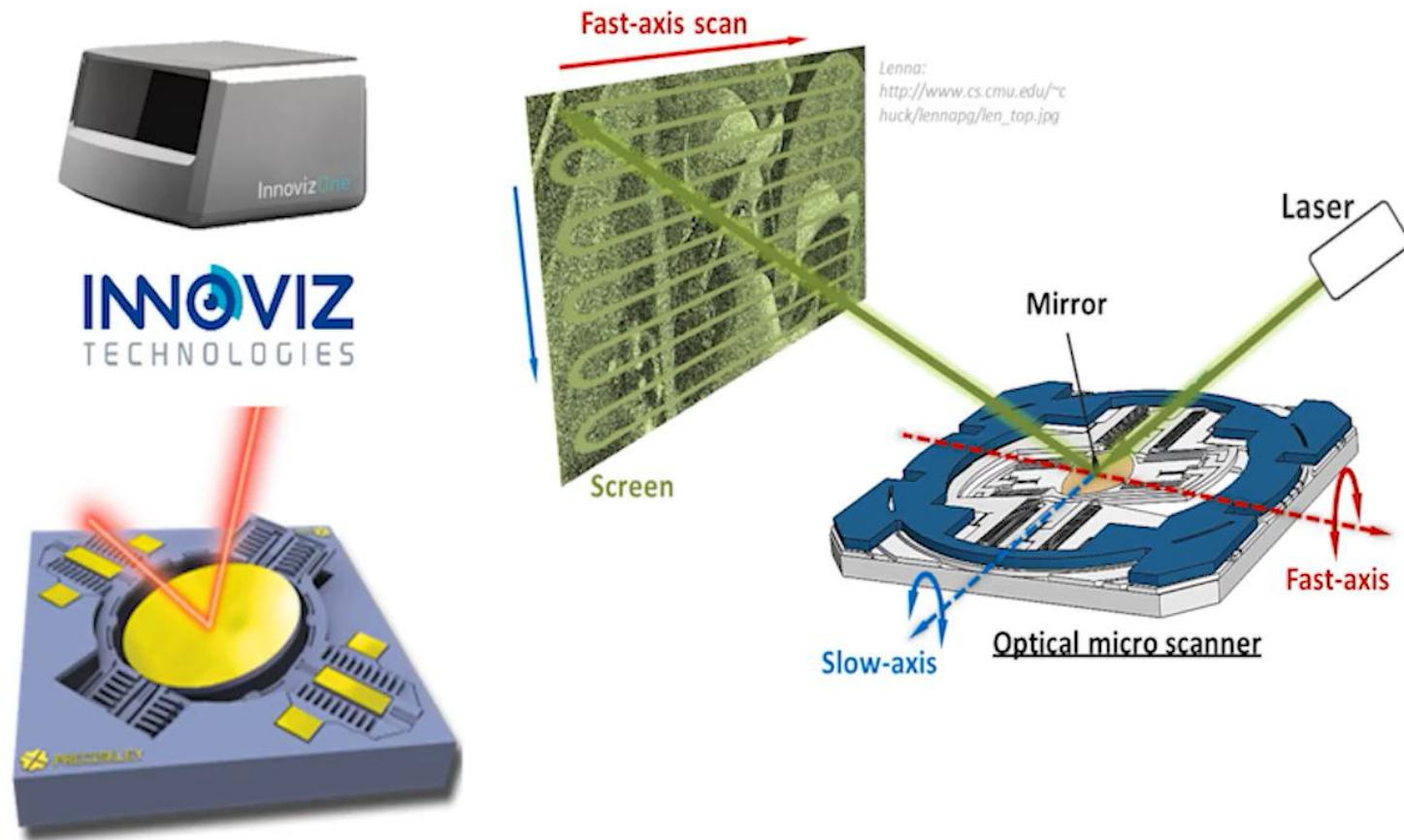
- 레이저 소자들이 직접 회전하는 것이 아닌 **거울과 같은 반사체가 회전**하여 거리 데이터를 취득함으로써, **최소한의 레이저 소자**를 바탕으로 주변 환경에 대한 스캐닝이 가능
- **최소한의 레이저 소자**를 기반으로 구성되므로 **소형화가 가능**해져, **차량에도 적용 가능**

단점

- 여전히 모터가 직접 회전하는 방식의 한계점으로 인해 **내구도가 쉽게 저하**될 수 있음
- Spinning LiDAR 대비 **해상도가 낮음**

1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – MEMS Mirror Scanning LiDAR 예시



- MEMS actuator라는 아주 작은 actuator를 이용하여 레이저 신호를 2축으로 조사함으로써 면 광원을 생성하는 기술

1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – MEMS Mirror Scanning LiDAR 예시



- 이스라엘의 ‘이노비즈’사에서 제작한 MEMS Mirror Scanning 방식의 LiDAR 예시

1. LiDAR 기술 이해

LiDAR 센서의 스캐닝 방식 – MEMS Mirror Scanning LiDAR 특징

장점

- 적은 레이저를 바탕으로 고해상도의 3차원 데이터 취득 가능 (Polygon Mirror LiDAR 대비)
- 소형화, 생산성 향상, 가격 절감 등을 통해 양산성 확보가 가능 (Spinning LiDAR 대비)

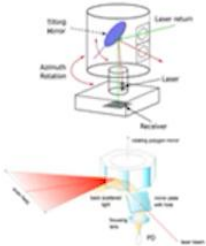
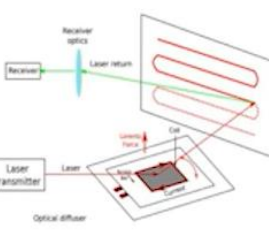
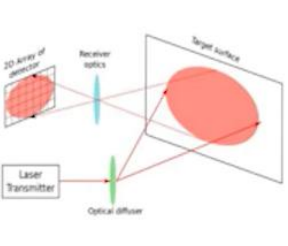
단점

- 화각이 작아 멀티 광원 사용 및 광정렬 문제 발생
- 저속 축에 대한 내구성 이슈가 존재

1. LiDAR 기술 이해

스캐닝 방식에 따른 LiDAR 센서의 특징 정리

<https://www.youtube.com/watch?v=3EehCU3csJQ>

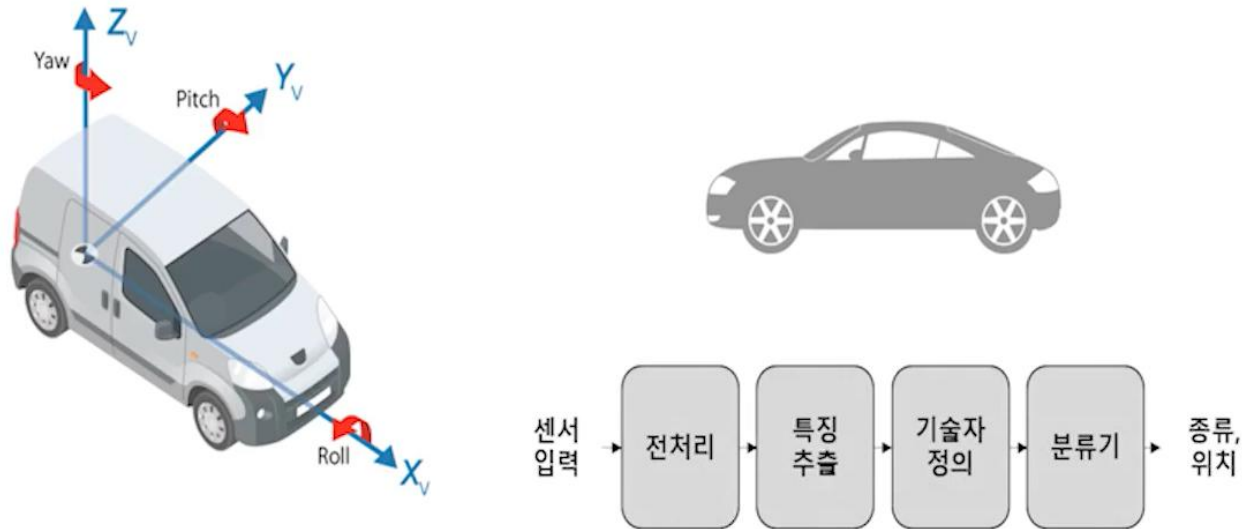
스캐닝방식	기계식 회전	MEMS Mirror	Flash
시스템 구성			
크기 및 가격	열세	보통	우세
차량용 신뢰성	보통	열세	우세
해상도	보통	보통	우세
감지 거리	우세	보통	열세

Copyright © 2022 SOSLAB, Professional LiDAR Sensor Provider All rights reserved.

- 기계식 회전 방식은 긴 감지 거리와 높은 해상도를 가지지만, 크기 및 가격 측면에서 제품화가 어렵다는 한계점을 가지고 있음.
- MEMS Mirror 또는 Flash 방식을 통해 크기 및 가격을 낮춤으로써 실제 차량에 들어갈 수 있는 형태의 LiDAR 센서에 대한 개발이 활발히 진행되고 있음

1. LiDAR 기술 이해

5) 자율주행 기술 실현을 위한 LiDAR 센서의 역할



- 차량 주변의 동적인 환경을 측정하고 주변 환경을 묘사
- 보행자, 차량, 자전거, 도로 표면, 차선 경계, 교통 신호 등을 인지
- 인지 정보를 바탕으로 주행에 대한 제어 방식을 계획하고 결정
- 다양한 센서(Camera, Radar, LiDAR)와 딥러닝 기술을 활용

1. LiDAR 기술 이해

Camera 기반 객체 인식 기술의 한계

- 객체 인식 및 추적 알고리즘은 대부분 2D Camera 센서를 기반으로 개발되어 왔으며, 상용화된 자율주행 2단계 차량에 적용되어 Lane Departure Warning (LDW), Lane Keeping Assist(LKA), Traffic Sign Recognition (TSR) 등의 ADAS 기능을 수행
- Camera 센서 기반 객체 인식 알고리즘은 100M 픽셀 이상급의 고해상도 데이터를 바탕으로 높은 인식률을 얻을 수 있으며, 기존에 촬영된 수 많은 이미지를 바탕으로 보행자, 차량, 이륜차, 차선 등의 다양한 객체를 딥러닝(deep learning) 기반의 학습을 통해 성능을 지속해서 향상할 수 있다는 장점이 있음
- 하지만 **조도 환경이 매우 높거나 (예: 태양광이 Camera 센서로 직접 조사되는 경우 등) 낮은 경우 (예: 야간, 터널 환경 등) 인식률이 현저히 떨어지며**, 인식에 실패했을 경우 정확도 부족으로 인해 자율주행 기능이 정지되거나 사고가 발생하는 상황을 초래할 수 있음

1. LiDAR 기술 이해

LiDAR 기반 객체 인식 기술

- LiDAR 센서 출력인 3차원 점 군(3D Point-Cloud)을 기반으로 하는 객체 인식 알고리즘은 Camera 센서에 비해 낮은 해상도를 사용하므로 인식률은 다소 떨어지나, **인식률 성능이 조도 환경에 크게 영향을 받지 않으며 객체 인식에 실패하더라도 정확한 거리 데이터를 이용하여 사고를 방지할 수 있다는 장점이 있음**
- 최근에는 딥러닝 기반의 방법들로 대상 객체에 대한 3차원 공간 상의 정확한 위치, 거리, 이동 방향 등의 추정이 가능해지고 있어 Camera 센서 기반 객체 인식 기술의 한계점을 보완하는 형태로 활용되고 있음
- 또한 딥러닝 기반 객체 인식 기술의 성능을 높이기 위해 더 많은 LiDAR 데이터셋의 확보와 딥러닝 모델의 개선을 위한 연구도 활발하게 진행되고 있음
- 이러한 관점에서 라이다 점 군 데이터를 이용한 객체 인식 기술은 3단계 이상의 자율주행 기능 구현에 필수적임

1. LiDAR 기술 이해

LiDAR 기반 객체 인식 기술 예시



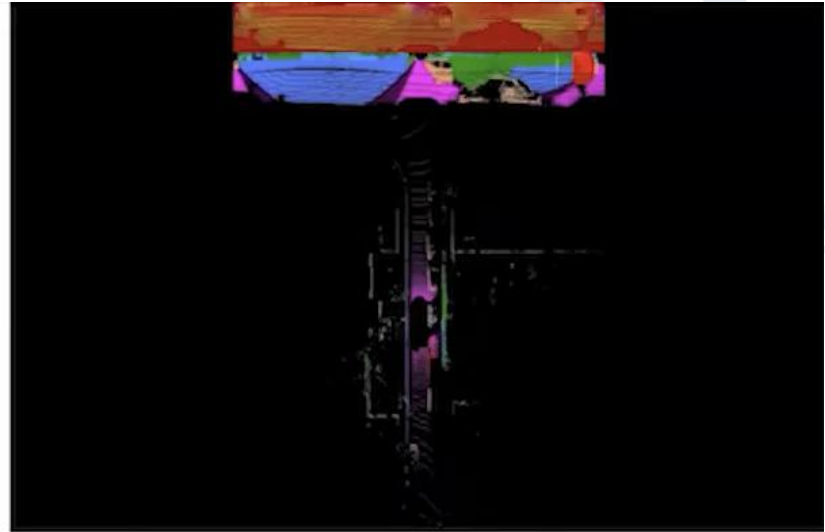
- 국내 자율주행 스타트업 서울 로보틱스의 LiDAR 기반 객체 인식 기술 데모 영상
- 360도 회전이 가능한 Spinning LiDAR를 기반으로 객체 인식 수행

1. LiDAR 기술 이해

LiDAR 기반 객체 인식 기술 최신 동향 – 인공지능 기반 객체 인식



라이다 3차원 객체 인식 기술 ©NVIDIA



라이다 3차원 의미론적 분할 기술 ©NVIDIA

- 최근에는 Camera 센서 기반의 객체 인식 분야 뿐만 아니라, LiDAR 센서를 기반으로 하는 객체 인식 분야에서도 딥러닝을 통한 특징 추출 기술들을 활용하여 높은 수준의 인지 성능을 보이는 알고리즘들이 개발되고 있음
- 딥러닝 모델의 성능은 데이터셋의 양과 품질에 좌우되기 때문에 Waymo, Uber ATG 등 자율주행 기업들을 중심으로 더 많은 LiDAR 데이터셋의 확보를 위한 연구가 활발하게 진행되고 있음

1. LiDAR 기술 이해

LiDAR 기반 객체 인식 기술 최신 동향 – 자율주행 데이터셋

구분	DATASET	Link
국 외	KITTI [Geiger2013]	http://www.cvlibs.net/datasets/kitti/
	NuScenes (Aptiv, Scale and nuTonomy Inc.)	https://scale.ai/open-datasets/nuscenes
	Apollo Open Transform (Baidu Inc.)	http://data.apollo.auto/
	Waymo Opendataset (Waymo)	https://waymo.com/open
국 내	KAIST Urban (KAIST)	http://irap.kaist.ac.kr/dataset/
	KODAS (LX 공간정보연구원)	http://kodas.or.kr/
	HD Map Dataset (NAVERLABS)	https://hdmap.naverlabs.com/



© The KITTI Vision Benchmark



© Lyft Level5



© ApolloScape

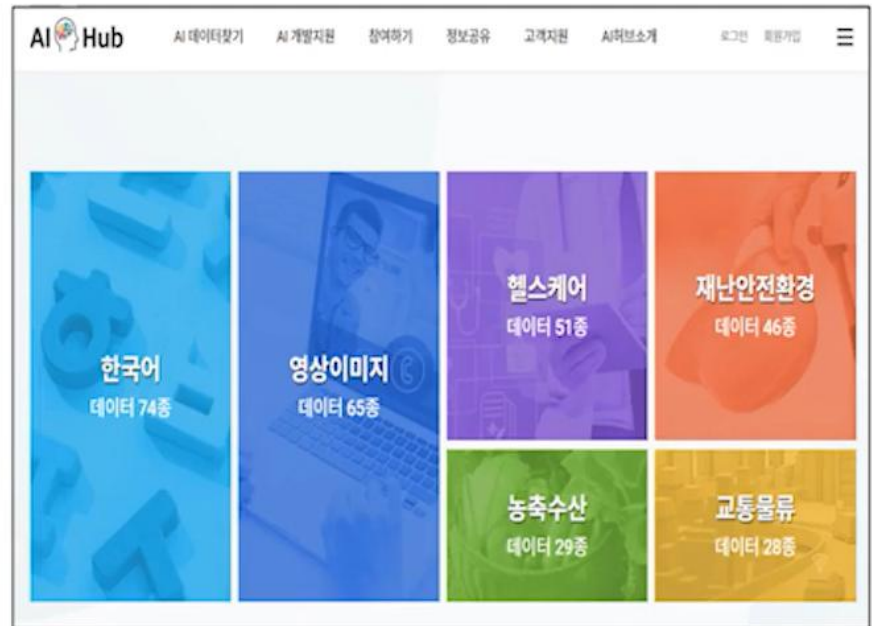
- 대표적인 데이터셋인 KITTI 데이터셋 뿐만 아니라, Google Waymo의 오픈 데이터셋과 같은 다양한 데이터셋을 기반으로 인공지능을 통한 객체 인식 기술들이 빠르게 발전하고 있음

1. LiDAR 기술 이해

LiDAR 기반 객체 인식 기술 최신 동향 – 자율주행 데이터셋



데이터 공유센터 (<http://www.kotsa.or.kr/avds>)



자율주행 공개 데이터셋 제공 (<https://aihub.or.kr/>)

- 국내 데이터 공유센터를 통해 자율주행 데이터셋을 수집 및 분석할 수 있도록 지원
- AiHub을 통해 국가 사업을 통해 수집한 다양한 데이터셋을 공개함으로써 인공지능 모델 개발의 속도를 높이하고자 함

SLAM

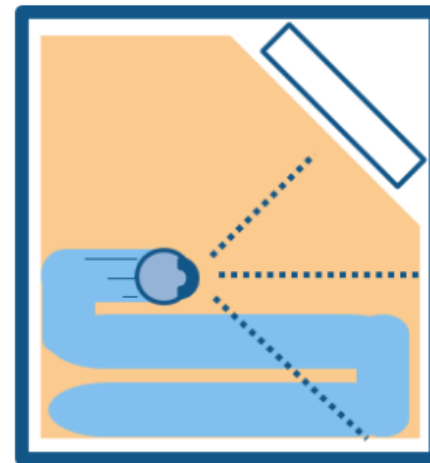
2. SLAM

1) What is SLAM?

- 최근 공항, 관공서, 가정집 등 주변에서 로봇을 쉽게 만나볼 수 있는데, 시간이 흐를수록 차량, 로봇, 드론 등 자율주행 플랫폼에 대한 수요가 증가하고 있음
- 이러한 자율주행 서비스에서 **SLAM**과 **navigation** 기술은 중요한 역할을 하며 필수요소가 되었음. 그 중 SLAM 알고리즘은 창고에서 선반을 정렬하는 이동 로봇의 길 찾기, 자율주행 차량의 빈 주차 공간 주차하기 또는 미지의 환경에서의 드론 조종을 통한 배송 등 다른 여러 응용 사례에서도 유용하게 사용할 수 있음



Without SLAM:
Cleaning a room randomly

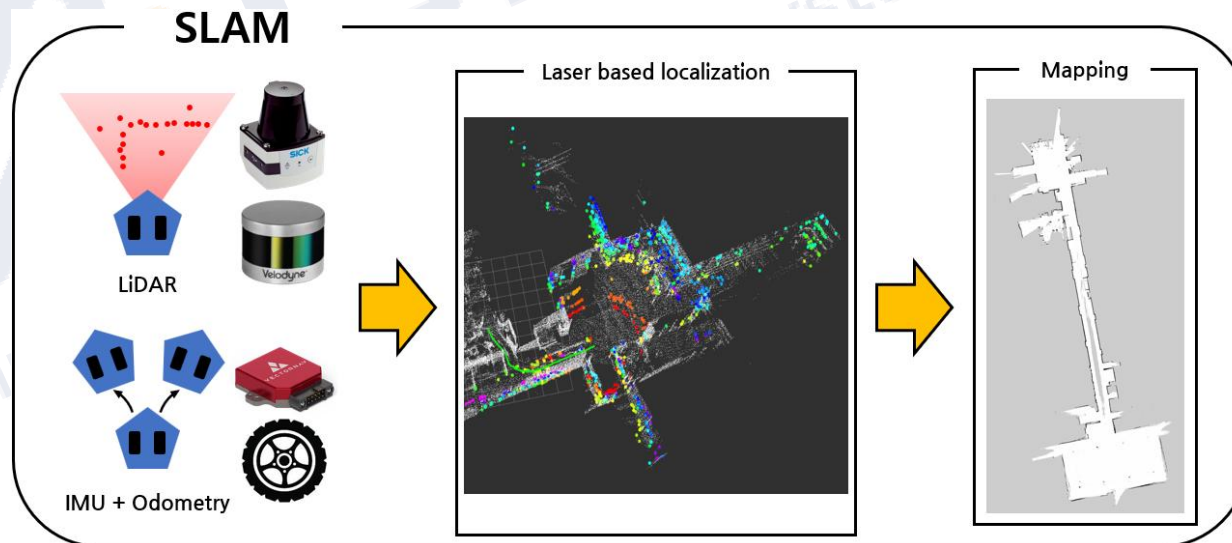


With SLAM:
Cleaning while understanding the room's layout

2. SLAM

1) What is SLAM?

- SLAM(Simultaneous Localization And Mapping)은 로봇 공학과 컴퓨터 비전 분야에서 로봇 또는 자율주행 차량이 알려지지 않은 환경을 탐색할 때, 해당 환경의 지도를 만들어 냄과 동시에 지속적으로 자신의 위치를 추정하는 기술
- SLAM은 로봇이 센서를 통해 주변 환경을 지속적으로 관찰하고 그 관찰 결과를 바탕으로 지도를 생성하며, 생성된 지도를 기반으로 자신의 위치를 업데이트
- SLAM의 주요 구성 요소는 'Localization'과 'Mapping'으로 구성

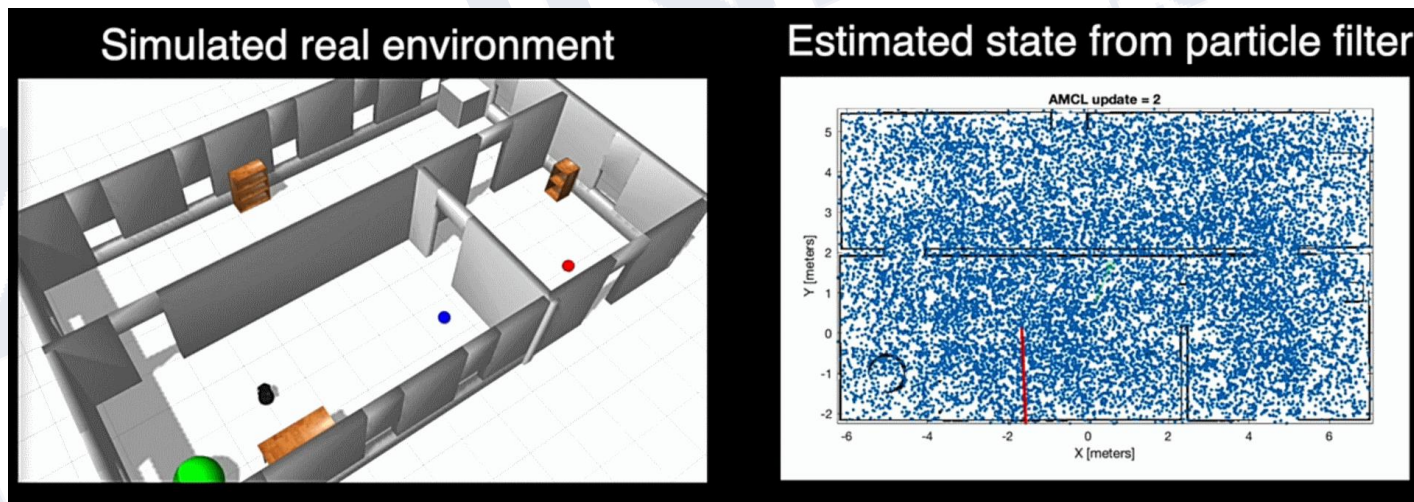


2. SLAM

1) What is SLAM?

- Localization

- Localization은 로봇이 현재 자신이 어디에 있는지를 파악하는 과정
- 로봇이 지도 상에서 자신의 위치를 정확하게 추정하고, 센서 데이터와 지도 데이터를 매칭하여 지속적으로 업데이트하는 과정을 포함

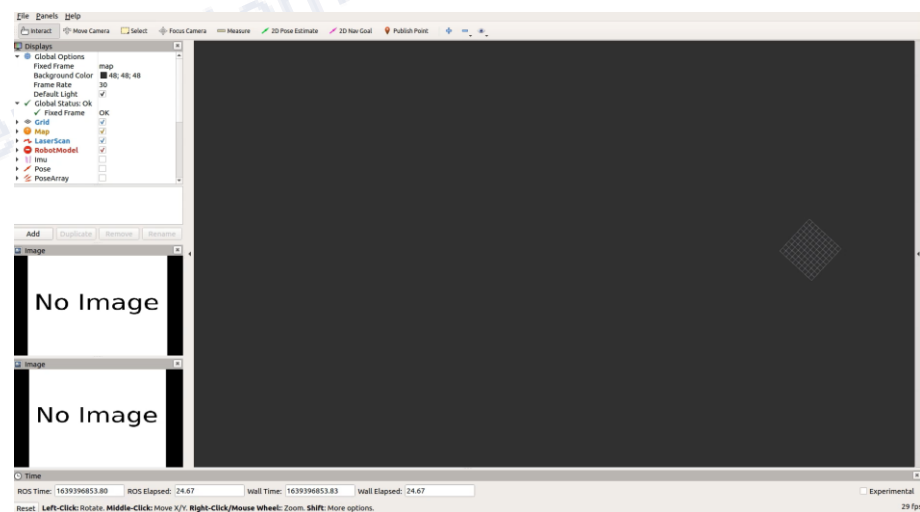
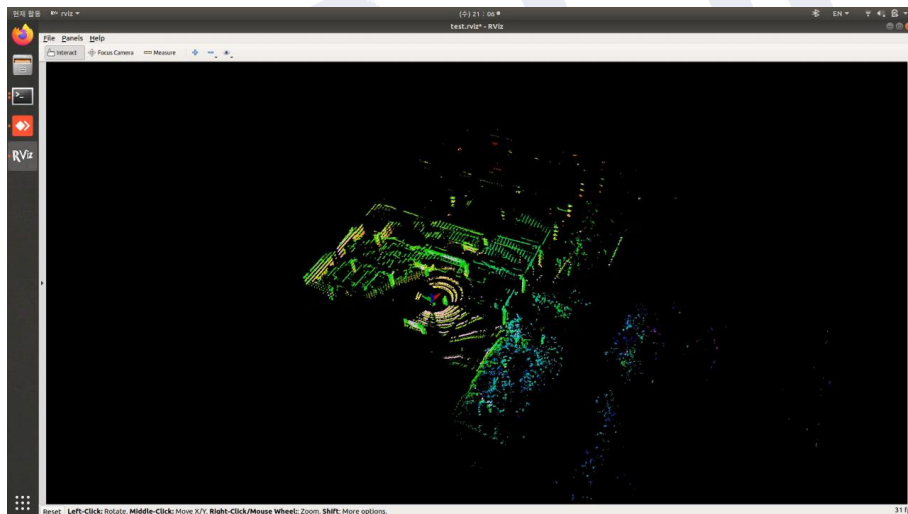


2. SLAM

1) What is SLAM?

● Mapping

- Mapping은 로봇이 환경을 탐색하면서 주변의 물리적인 특징들을 추출하여 지도를 생성하는 과정
- 생성된 지도는 로봇이 같은 환경을 다시 탐색할 때나 다른 로봇이 해당 환경을 파악하는데 사용될 수 있음



2. SLAM

2) SLAM의 종류

- SLAM은 크게 **Camera 기반**의 방식과 **LiDAR(Light Detection and Ranging) 기반**의 방식으로 구분
- **Camera 기반**의 SLAM은 일반적으로 **visual SLAM**이라 하며, Camera의 **image data**를 통해 환경을 매핑하고 위치를 추정함. Visual SLAM은, LiDAR 기반 SLAM 대비 **비용이 저렴**하며, **환경의 시각적 세부 정보를 취득 가능**
- **LiDAR 기반**의 SLAM은 레이저 센서(또는 거리 센서)의 **scan data**를 이용하는 방식임. 자율주행차량과 드론처럼 빠르게 이동하는 물체와 관련된 응용 사례에 주로 사용되며, 2D/3D 포인트 클라우드 데이터를 이용하면 **고정밀 거리 측정**이 가능하고, SLAM 알고리즘을 적용한 **정밀 지도 생성**에도 효과적임.
- 이러한 방식들 이외에 **다중센서를 이용한 SLAM** 알고리즘도 많이 활용되고 있음. 카메라, IMU(Inertial Measurement Unit, 관성 측정 장치), GPS, LiDAR, Radar 등을 포함한 다양한 센서를 활용하여 SLAM 알고리즘의 **정밀도와 견고성을 강화**함

2. SLAM

2) SLAM의 종류

- Visual SLAM

- 대표적인 visual SLAM 기법으로는 ORB(Oriented FAST and Rotated BRIEF) SLAM이 있음
- ORB SLAM은 single camera, stereo camera, RGB-D camera 에서 사용가능
- ORB SLAM 이외에도 LSD(Large-Scale Direct Monocular) SLAM 등 다양한 알고리즘이 존재하며, 현재해도 계속하여 개선된 visual SLAM 기법이 연구되고 있음

ORB-SLAM

Raúl Mur-Artal, J. M. M. Montiel and Juan D. Tardós

{raulmur, josemari, tardos} @unizar.es



Instituto Universitario de Investigación
en Ingeniería de Aragón
Universidad Zaragoza

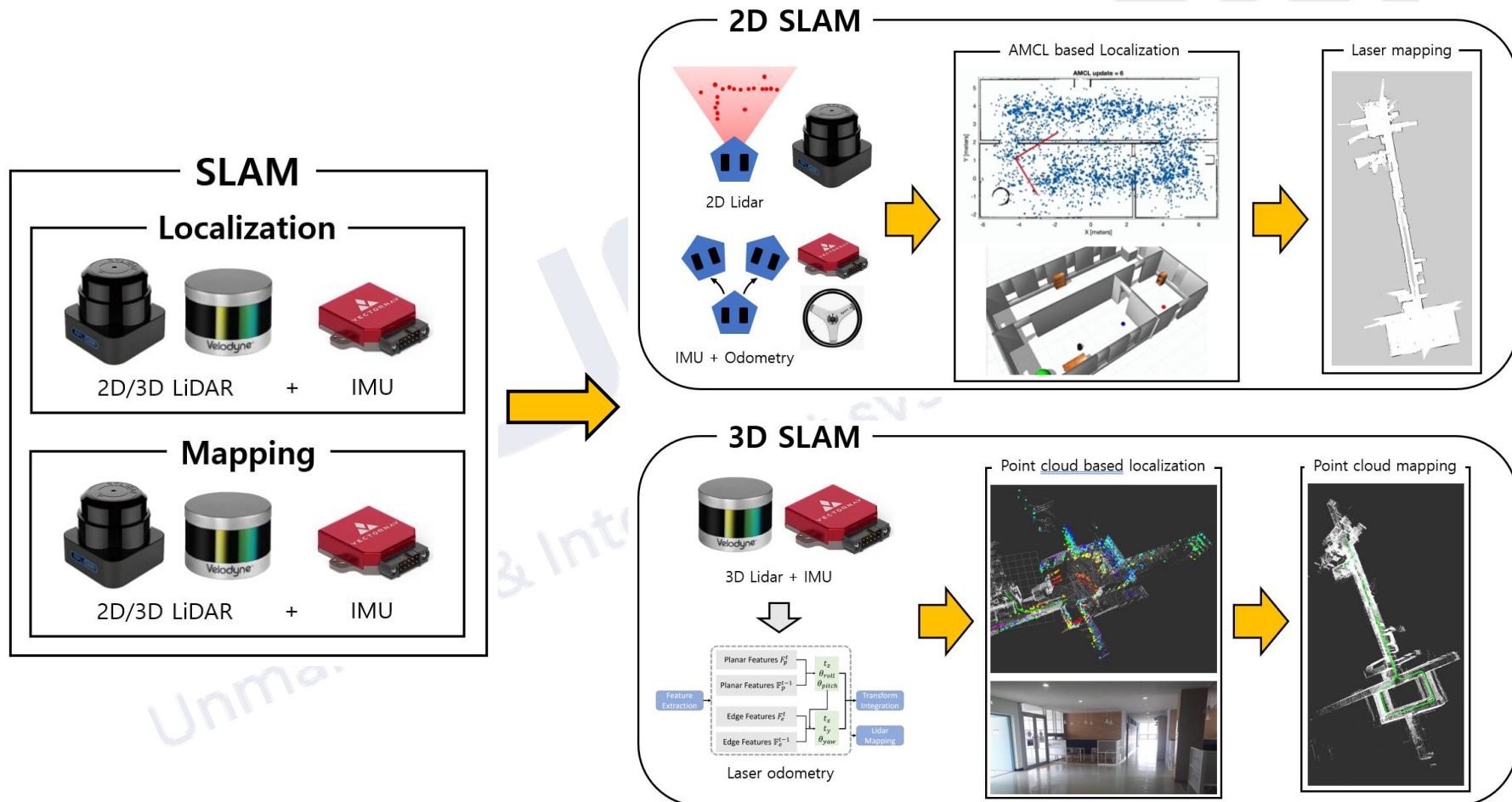


Universidad
Zaragoza

2. SLAM

2) SLAM의 종류

- LiDAR SLAM

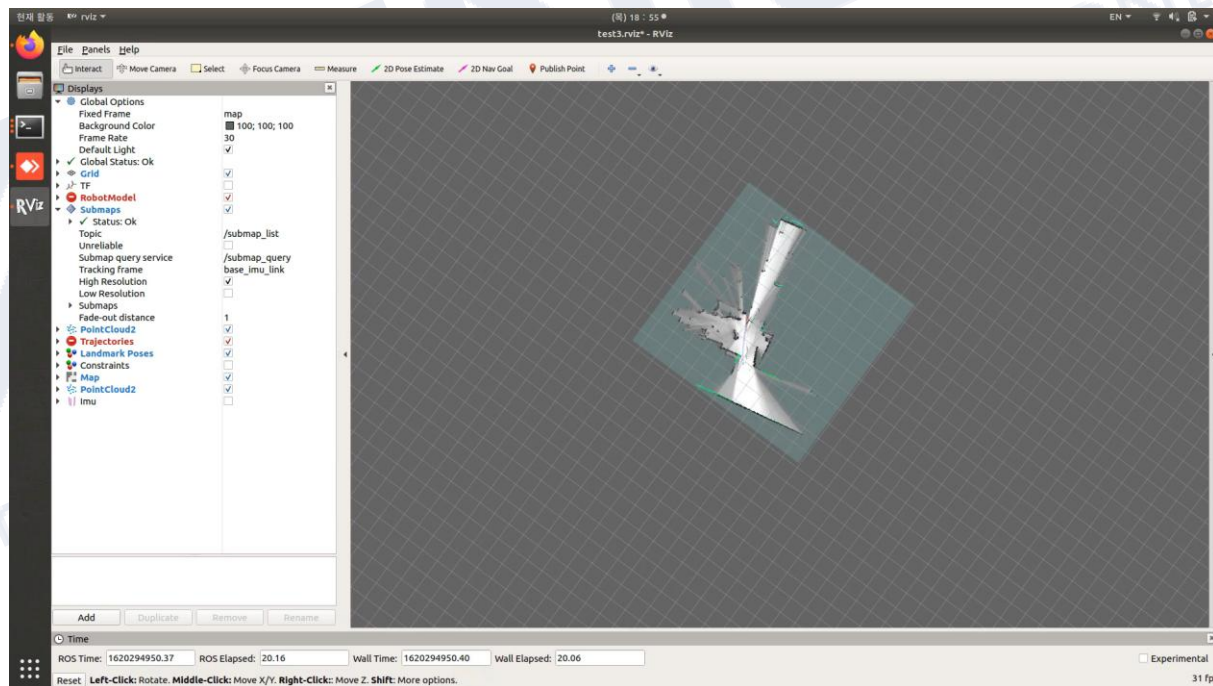


2. SLAM

2) SLAM의 종류

● LiDAR SLAM (2D SLAM)

- Google에서 개발한 **Cartographer** 알고리즘 기반 SLAM으로 loop closure 기능으로 위치 추정 및 맵 정합의 정확도가 높음
- 2D LiDAR SLAM에는 Gmapping, Hector, Lifelong 등의 알고리즘이 있음

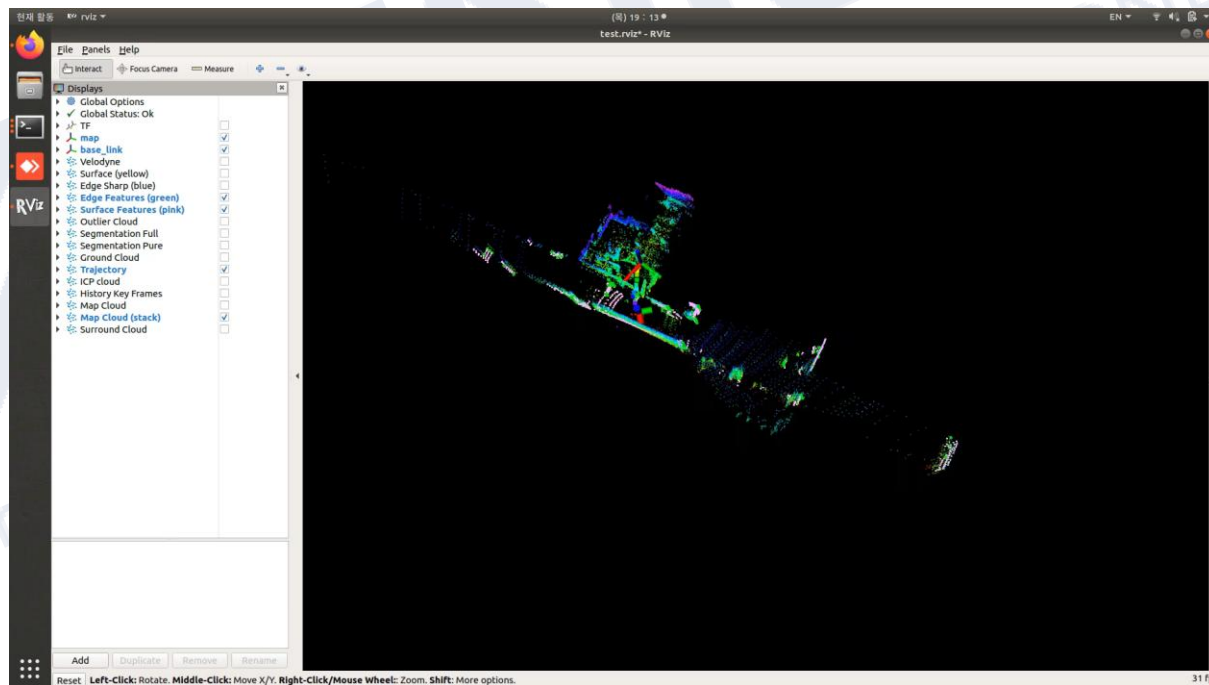


2. SLAM

2) SLAM의 종류

● LiDAR SLAM (3D SLAM)

- 데이터 연산량이 많은 3D LiDAR SLAM을 경량화하여 위치 추정 및 loop closing 연산 속도를 높인 것이 특징인 **LeGO-LOAM** 알고리즘
- 3D LiDAR SLAM에는 LOAM, 3D Cartographer, Octomap 등의 알고리즘이 있음

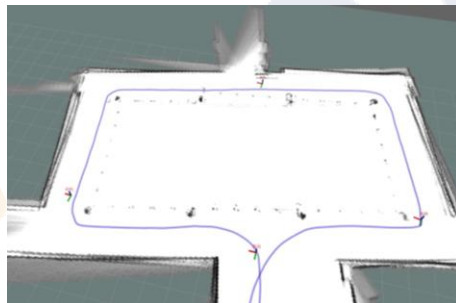


2. SLAM

2) SLAM의 종류

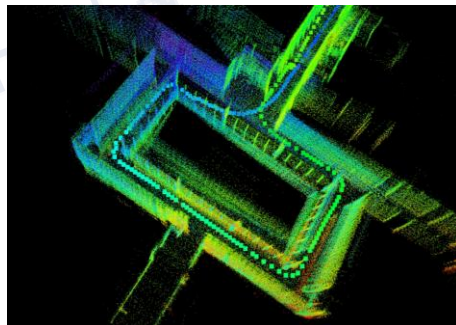
● LiDAR SLAM

- 2D LiDAR SLAM의 경우 한 채널만 사용하여 지도를 생성하기 때문에 LiDAR 감지 높이에 장애물이 감지되지 않으면 지도에 반영할 수 없음. 하지만 3D LiDAR SLAM의 경우 LiDAR의 감지 범위에 따라 넓은 범위의 장애물들을 모두 지도에 반영할 수 있어 정밀도가 높음. 그에 따라 센서의 비용이나 데이터 연산량이 크게 차이가 나므로 사용자의 상황에 따라 적절히 선택해야함



● 2D Mapping

- Hector mapping : 2D LIDAR
- Gmapping : 2D LIDAR, Odometry
- Lifelong : 2D LIDAR, Odometry
- **2D Cartographer** : 2D LIDAR, IMU, (Odometry)
- ...



● 3D Mapping

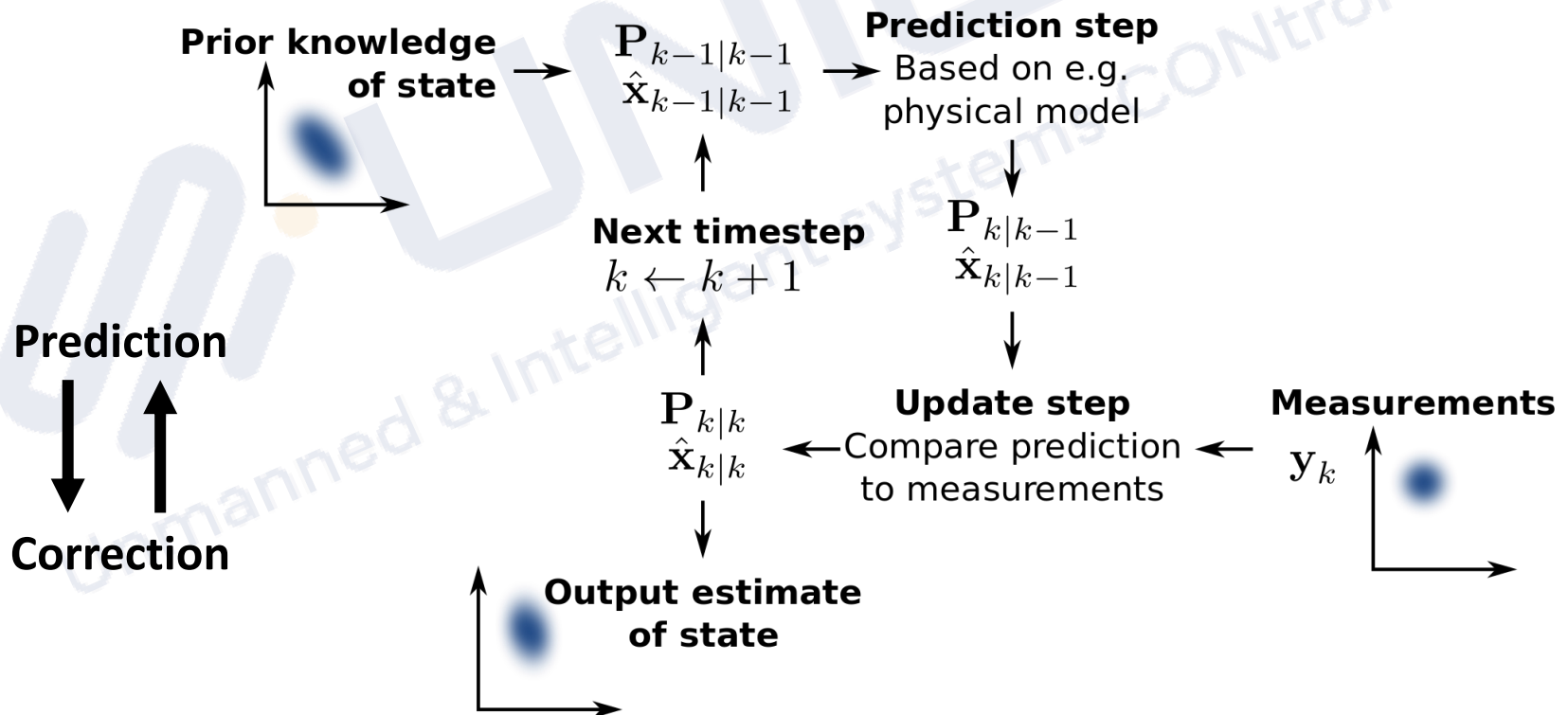
- LOAM : 3D LIDAR, IMU
- **LeGo-LOAM** : 3D LIDAR, IMU
- 3D Cartographer : 3D LIDAR, IMU, (Odometry)
- Octomap : 3D LIDAR, Depth camera
- ...

2. SLAM

3) LiDAR SLAM의 원리

- Kalman filter

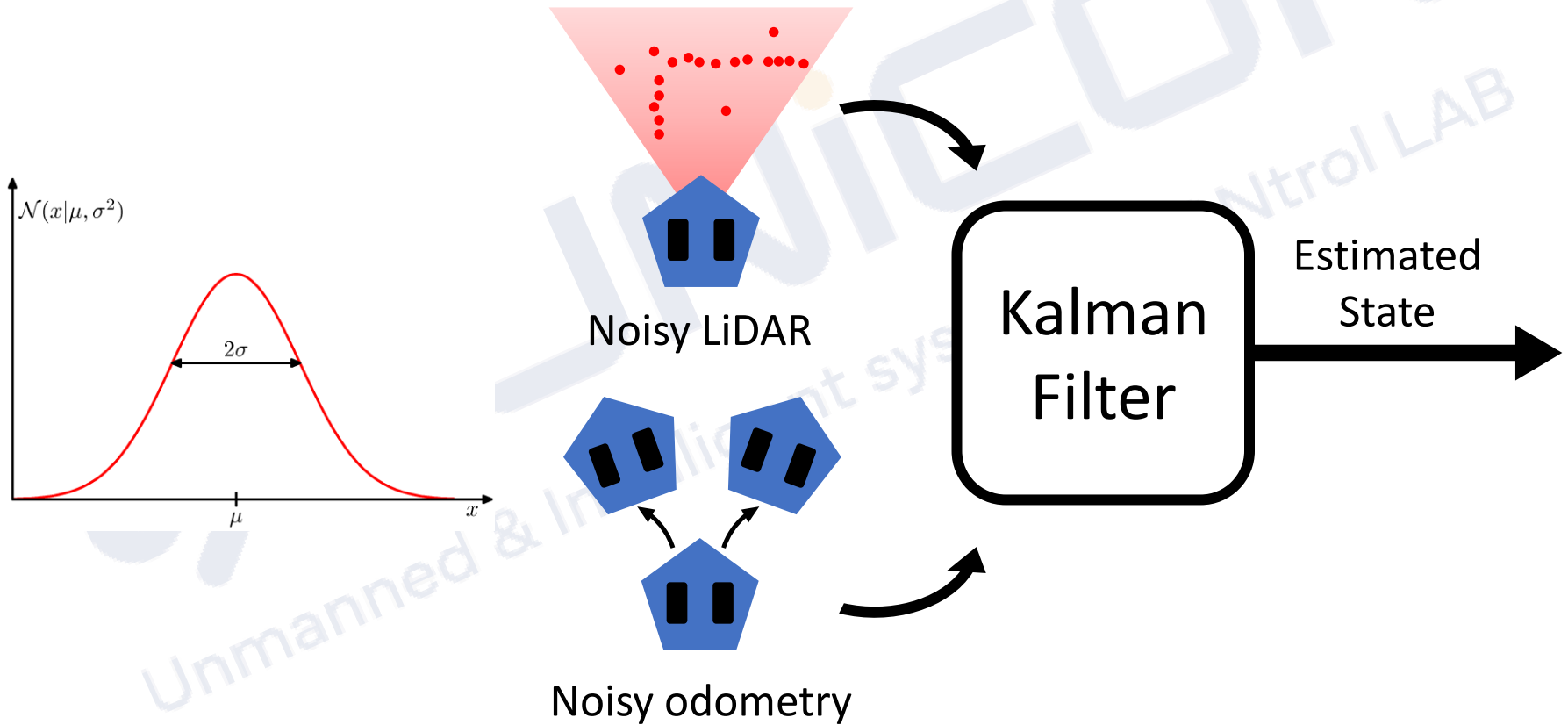
- 베이지스 확률을 기반으로 노이즈가 포함된 선형 시스템의 상태를 추정하는 재귀 필터
- 예측과 보정 단계를 반복하여 추정



2. SLAM

3) LiDAR SLAM의 원리

- Kalman filter

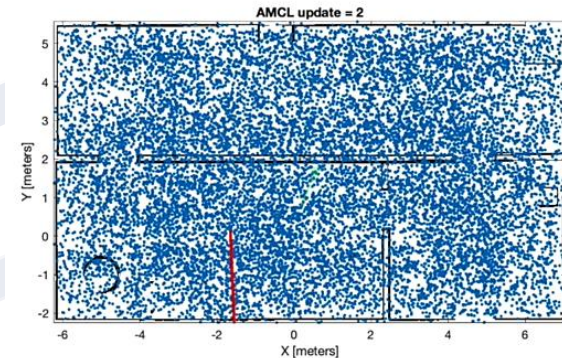


2. SLAM

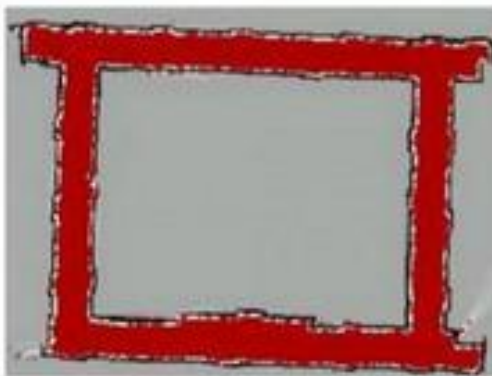
3) LiDAR SLAM의 원리

● AMCL

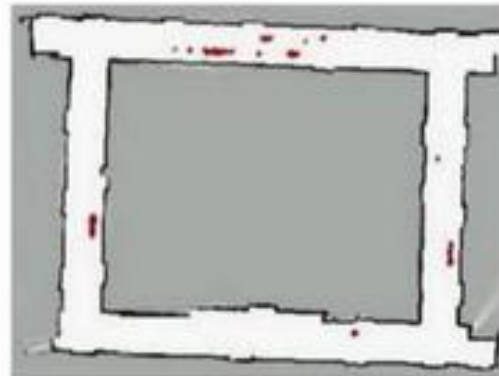
- 파티클(입자)로 표현된 무작위 샘플에서 센서 데이터에 기반하여 상태를 추정하는 Particle filter를 이용한 AMCL(Adaptive Monte Carlo Localization) 방식
- AMCL은 LiDAR의 레이저 스캔 데이터, 바퀴의 오도메트리 데이터, 관성 센서의 자세 데이터 등을 입력으로 받아 지도 상에서의 위치를 추정할 수 있음



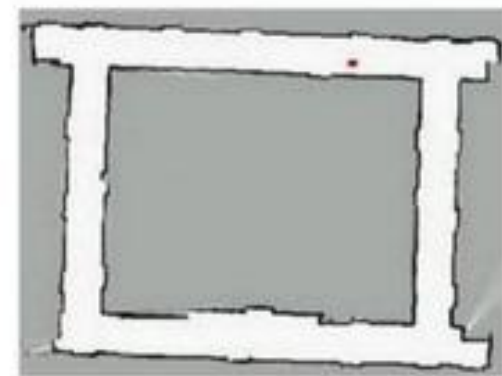
- ① Re-sampling
- ② Sampling
- ③ Importance sampling



t = 1 sec, approx 100,000 particles



t = 40 sec, approx 1,000 particles



t = 120 sec, approx 100 particles

2. SLAM

3) LiDAR SLAM의 원리

① Initialization (초기화)

→ 로봇은 초기 위치를 (0, 0) 또는 임의의 위치로 설정하고, 초기 지도 상태를 설정

② Sensor Data Acquisition (센서 데이터 획득)

→ 로봇은 LiDAR 및 IMU 센서 등을 바탕으로 주변 환경에 대한 정보를 수집

→ 센서 데이터를 통해 주변의 장애물, 랜드마크, 벽 등의 정보를 획득

③ Feature Extraction (특징점 추출)

→ 센서 데이터(ex. LiDAR 데이터)를 처리하여 장애물의 경계, 코너, 벽과 같은 유의미한 특징적인 점들을 추출

→ 해당 과정은 센서의 raw 데이터를 더 유용한 형태로 변환하는 과정

④ Scan Matching (스캔 매칭)

→ 현재 스캔 데이터와 이전 스캔 데이터를 비교하여 두 스캔 데이터 간의 최적의 정합(Alignment) 결과 도출

→ 스캔 매칭을 통해 로봇의 상대적인 이동(이동 거리 및 방향)을 계산하며, 로봇의 현재 위치를 추정

2. SLAM

3) LiDAR SLAM의 원리

⑤ Feature Matching (특징점 매칭)

- 스캔 매칭 결과를 바탕으로 현재 지도와 비교하여 특징점을 매칭
- 스캔 매칭 결과는 피처 매칭 과정의 중요한 입력으로 사용되며, 기존 지도에 존재하는 특징점과 새로 추출된 특징점을 비교하여 일치하는 특징점을 찾음

⑥ Pose Estimation (로봇 위치 추정)

- 스캔 매칭과 피처 매칭 결과를 종합하여 로봇의 위치를 더욱 정확하게 추정
- 여러 센서 데이터를 융합하여 위치 추정 정확도를 향상시킬 수 있음

⑦ Map Update (지도 업데이트)

- 로봇의 위치와 새로운 특징점 정보를 기반으로 지도 업데이트 수행
- 새로운 피처를 추가하거나 기존 피처의 위치를 조정하여 지도를 갱신

⑧ Loop Closure (루프 클로징)

- 로봇의 위치 추정 오차를 줄이고 지도의 일관성을 유지하기 위해, 로봇이 이미 탐색한 지역으로 돌아왔을 때 스캔 매칭 기반의 루프 클로징 과정을 수행하여 지도의 일관성 유지 및 정확도 향상

⑨ Iteration (반복)

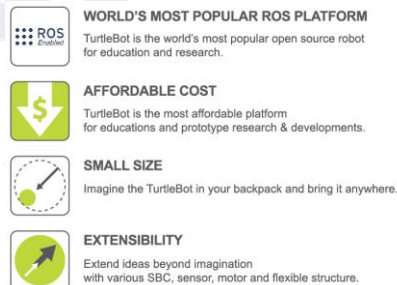
- 로봇이 주어진 환경을 완전히 탐색할 때까지 위 ① ~ ⑧ 단계를 반복함으로써 지도 작성 및 위치 추정 수행

2. SLAM

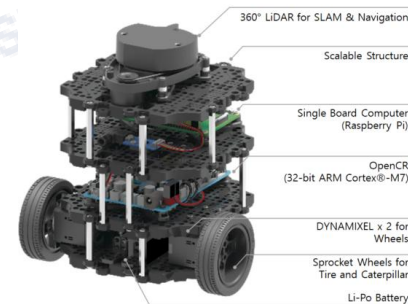
4) Turtlebot3 SLAM 실습

● Turtlebot

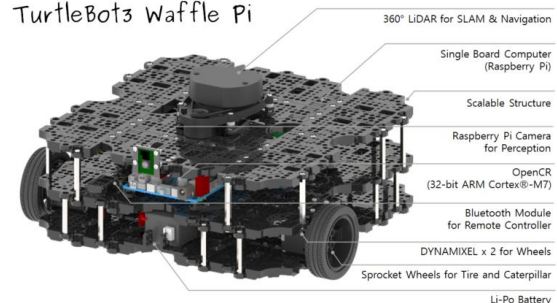
- TurtleBot은 ROS 표준 플랫폼 로봇으로 Turtle은 1967년 교육용 컴퓨터 프로그래밍 언어 Logo에 의해 구동된 Turtle 로봇에서 유래되었음. 또한 ROS 로고에 사용된 아홉 개의 점은 거북이 등껍질에서 유래했음. Logo의 Turtle에서 비롯된 TurtleBot은 ROS를 처음 접하는 사람들에게 TurtleBot을 통해 ROS를 쉽게 가르치기 위해 설계되었으며, 개발자와 학생들 사이에서 가장 인기 있는 플랫폼인 ROS의 표준 플랫폼이 되었음
- 2017년에 이전 모델들의 부족한 기능과 사용자 요구를 보완하기 위해 TurtleBot3가 개발되었으며, 구동을 위해 ROBOTIS의 스마트 액추에이터 DYNAMIXEL을 채택했음. TurtleBot3는 교육, 연구, 취미, 제품 프로토타이핑에 사용하기 위한 작고 저렴하며 프로그래밍 가능한 ROS 기반 모바일 로봇임



TurtleBot3 Burger



TurtleBot3 Waffle Pi

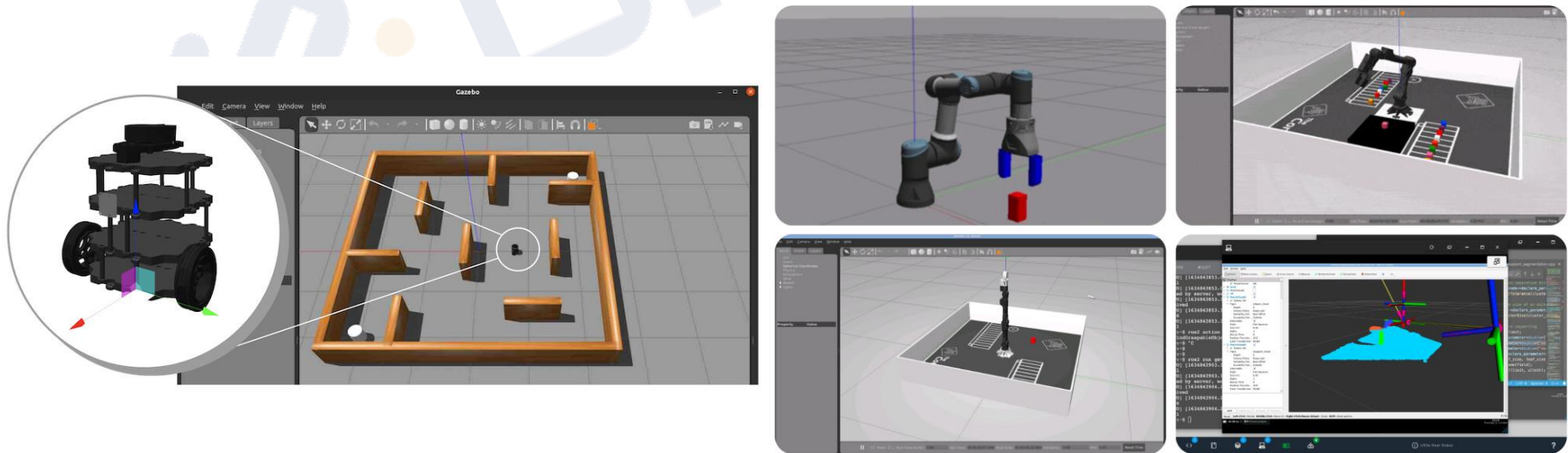


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo: Simulation tool

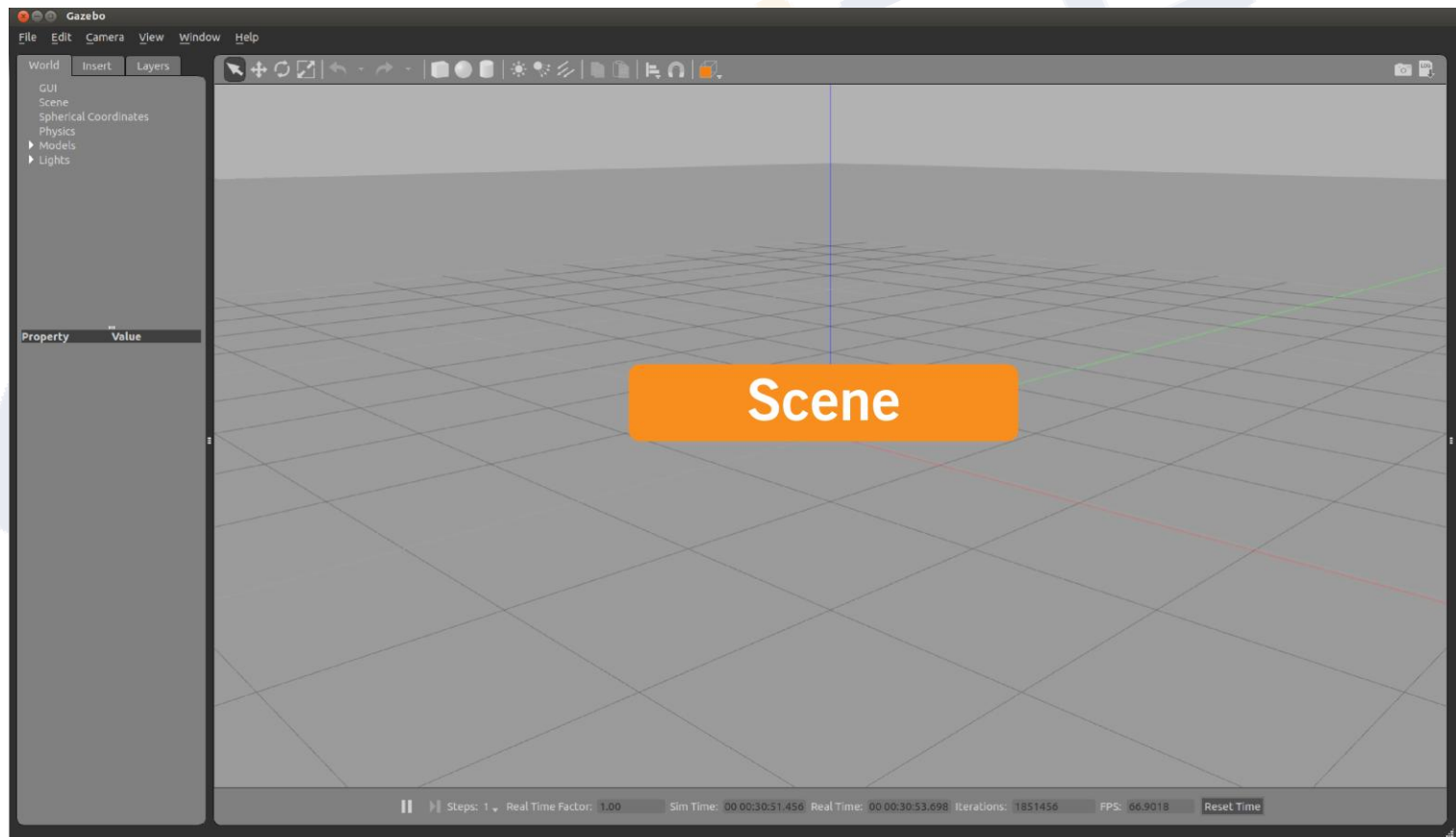
- 실제 환경과 유사한 3D 시뮬레이션 환경 제공
- 물리 엔진이 탑재되어 충돌, 관성, 마찰 등의 실제 물리 현상을 구현가능
- 다양한 센서(camera, LiDAR, IMU 등)를 구현할 수 있어 로봇의 환경 인식 및 자율주행 알고리즘 개발에 유용하게 활용할 수 있음
- 복잡한 로봇의 모델을 생성하고 제어할 수 있음
- ROS와 높은 호환성을 가지며, 다양한 플러그인을 통해 여러 기능을 확장하여 사용할 수 있음



2. SLAM

4) Turtlebot3 SLAM 실습

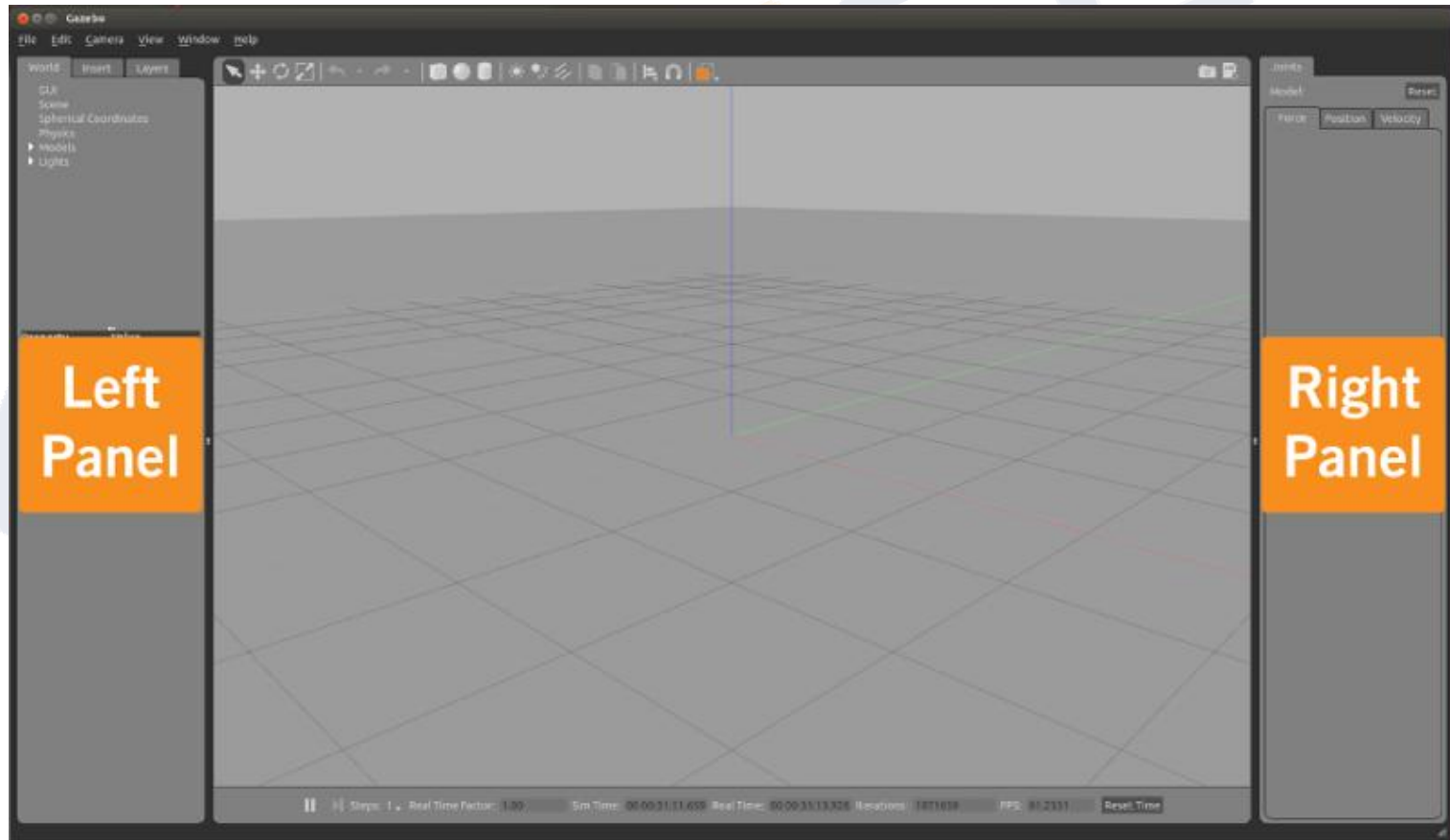
- Gazebo: Simulation tool
 - Scene: 시뮬레이션된 객체가 애니메이션화, 환경과 상호 작용



2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo: Simulation tool
 - **Left Panel:** World, Insert, Layers 탭으로 구성
 - **Right Panel:** 선택한 모델의 파츠(조인트)와 상호 작용



2. SLAM

4) Turtlebot3 SLAM 실습

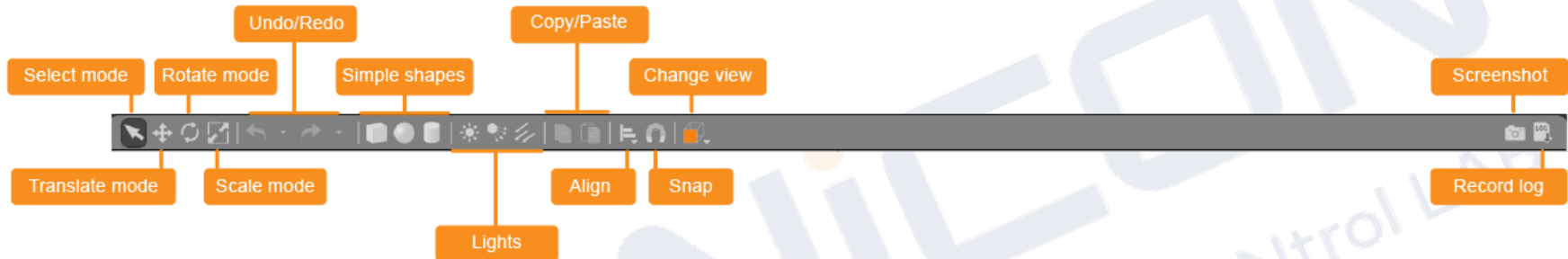
- Gazebo: Simulation tool

- **World:** 현재 장면에 있는 모델을 표시, 포즈와 같은 모델 매개변수를 보고 수정할 수 있고, 카메라 포즈를 조정하여 카메라 뷰 각도를 변경할 수 있음.
- **Insert:** 시뮬레이션에 새 객체(모델)를 추가하는 곳. 삽입하려는 모델을 클릭하고, Scene에서 다시 클릭하여 추가.
- **Layers:** 시뮬레이션에서 사용 가능한 다양한 시각화 그룹을 구성하고 표시. 레이어에는 하나 이상의 모델이 포함될 수 있고, 레이어를 켜거나 끄면 해당 레이어의 모델이 표시되거나 숨겨짐.

2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo: Simulation tool



상단 Toolbar

- Select mode: Scene 탐색
- Translate mode: 모델 이동
- Rotate mode: 모델 회전
- Scale mode: 모델 크기 조정
- Undo/Redo: 실행 취소/다시 실행
- Simple shapes: 간단한 도형 삽입
- Lights: 조명 추가
- Copy/Paste: 모델 복사/붙여넣기
- Align: 모델 정렬
- Snap: 한 모델을 다른 모델에 스냅
- Change view: Scene의 각도 변경

2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo: Simulation tool



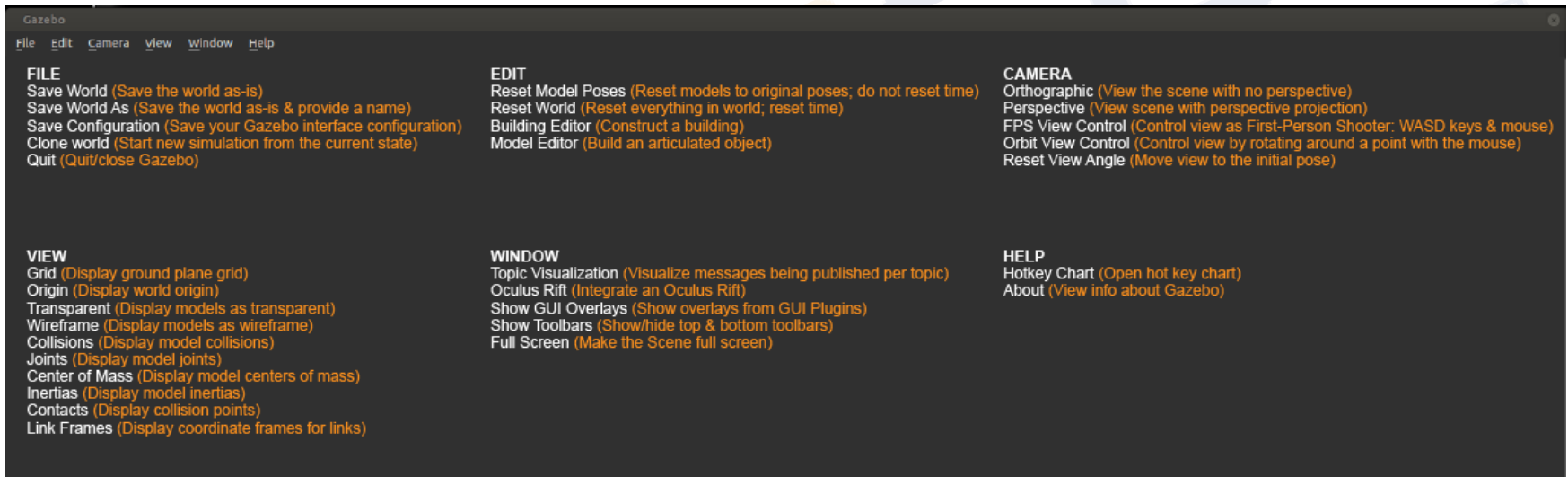
하단 Toolbar

- 시뮬레이션 시간 및 실제 시간과의 관계와 같은 시뮬레이션에 대한 데이터를 표시

2. SLAM

4) Turtlebot3 SLAM 실습

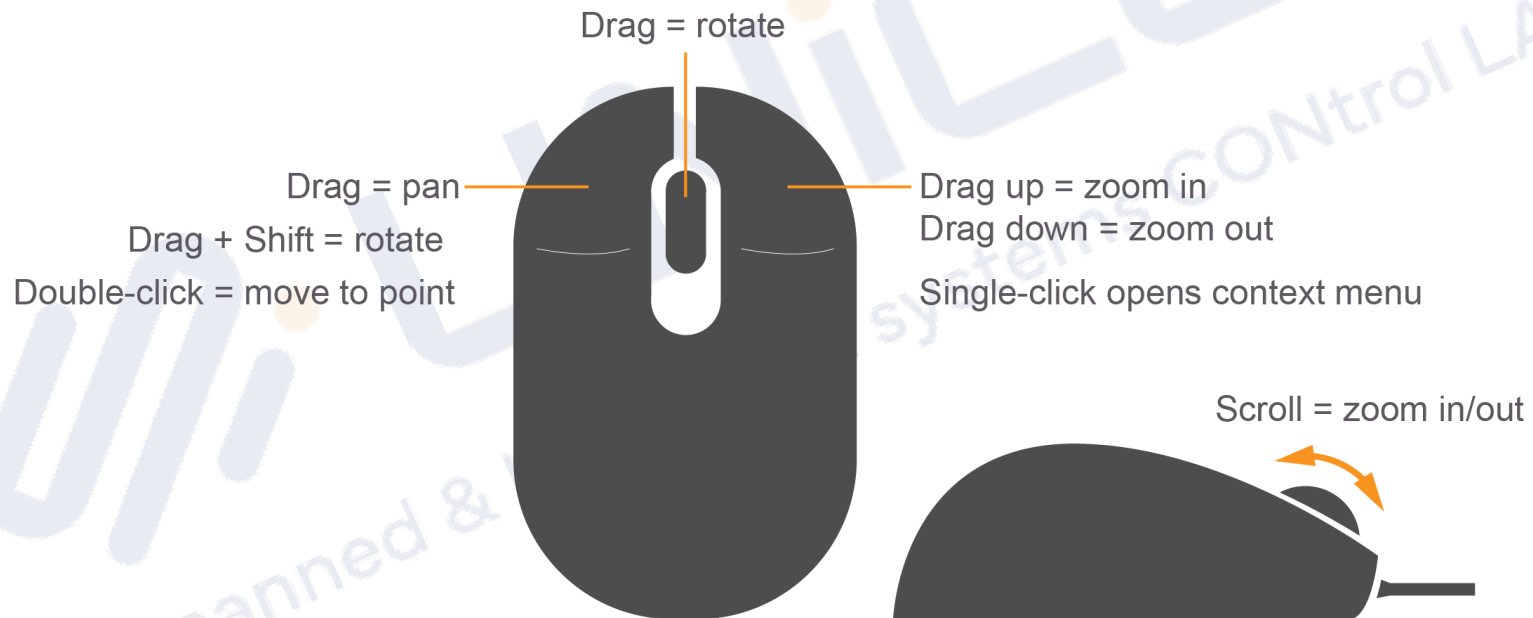
- Gazebo: Simulation tool
 - Menu



2. SLAM

4) Turtlebot3 SLAM 실습

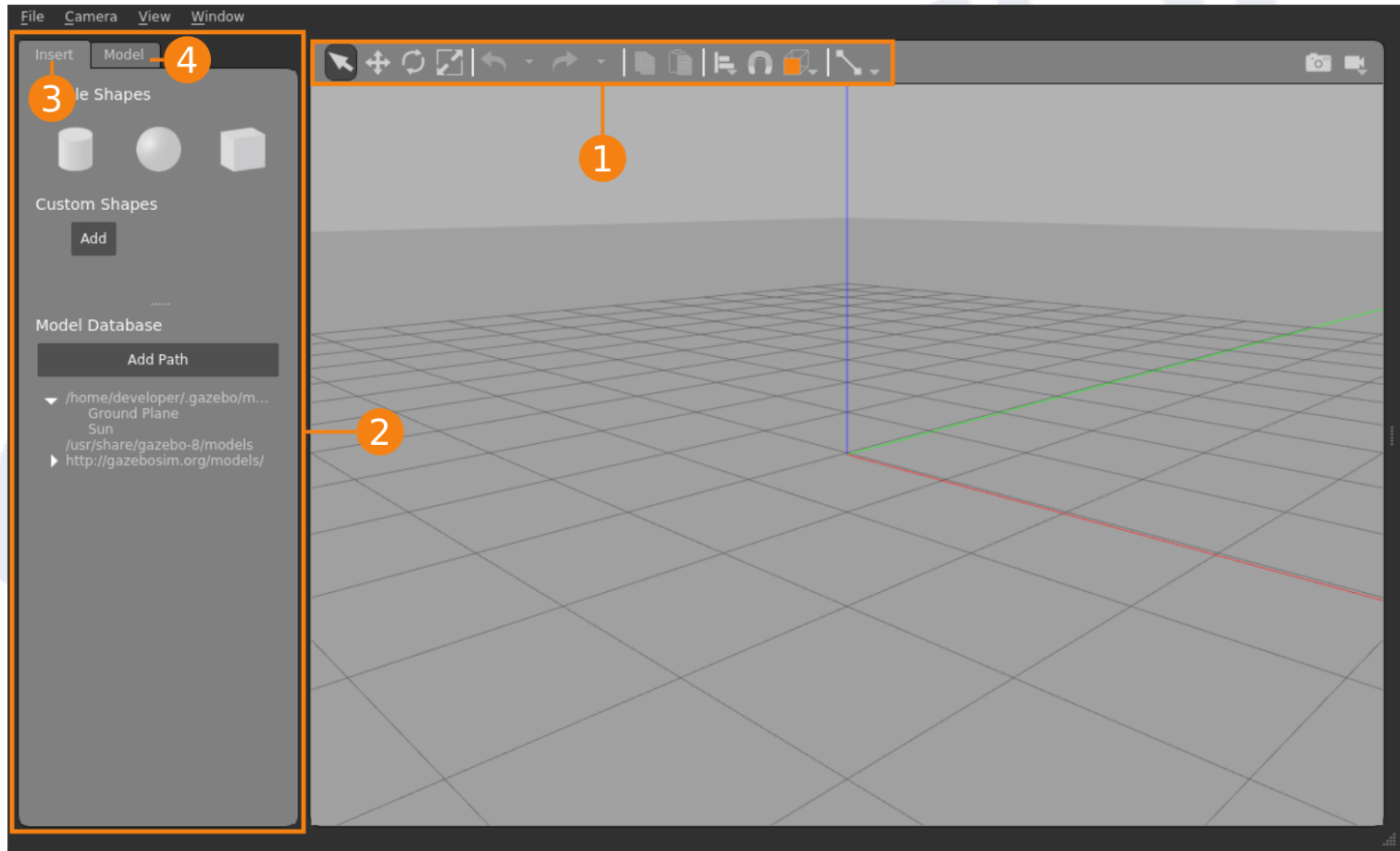
- Gazebo: Simulation tool
 - Mouse controls



2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성
 - Edit 메뉴에서 **Model Editor** 를 선택 (또는 Ctrl+M)



2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성

① **Toolbar**: 모델을 편집하기 위한 도구

② **Palette**: Left panel 이라고도 함. 모델을 편집하기 위한 두 개의 탭이 있음

③ **Insert tab**: Link 및 joint 모델 추가 도구

- **Simple Shapes**: 모델에 링크를 형성하기 위해 삽입할 수 있는 기본 기하 도형
- **Custom Shapes**: Custom mesh를 가져올 수 있음. 현재 COLLADA(.dae), 3D Systems(.stl), Wavefront(.obj) 및 W3C SVG(.svg) 파일을 지원함
- **Model Database**: 모델 목록에서 불러옴. 이는 간단한 모양과 같은 방식으로 모델 편집기에 삽입할 수 있고, 삽입되면 중첩 모델 이라고 함.

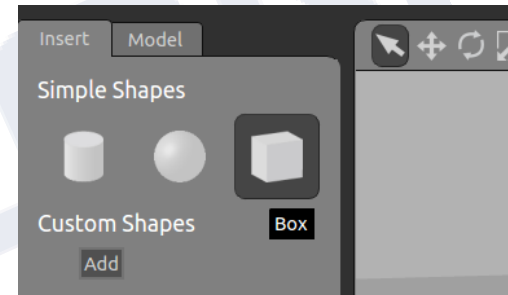
④ **Model tab**: 빌드하는 모델의 이름과 기본 파라미터를 설정할 수 있음. 모델에 있는 링크, 조인트, 중첩 모델 및 플러그인 목록이 표시됨

2. SLAM

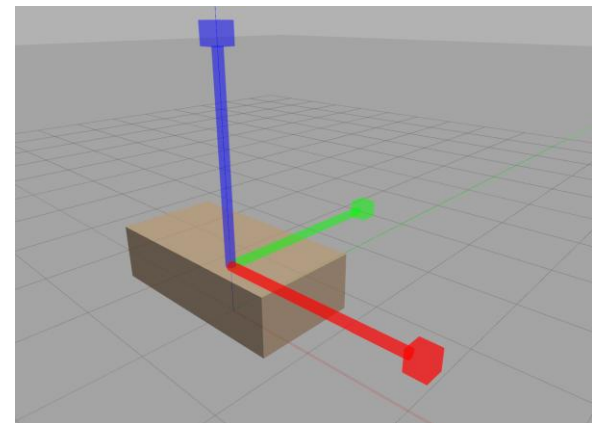
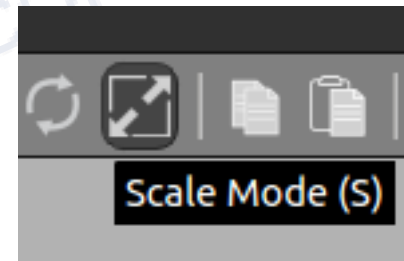
4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (1/16)

- 모빌리티 시뮬레이션 모델
- Left panel의 insert 탭에서 상자 아이콘을 클릭하여 생성



- 상자의 크기를 조정하여 차량 모양에 가깝게 보이게 조절
- 스케일 도구를 선택하여 색상 마커를 조정
- X축 - 빨강, Y축 - 초록, Z축 - 파랑

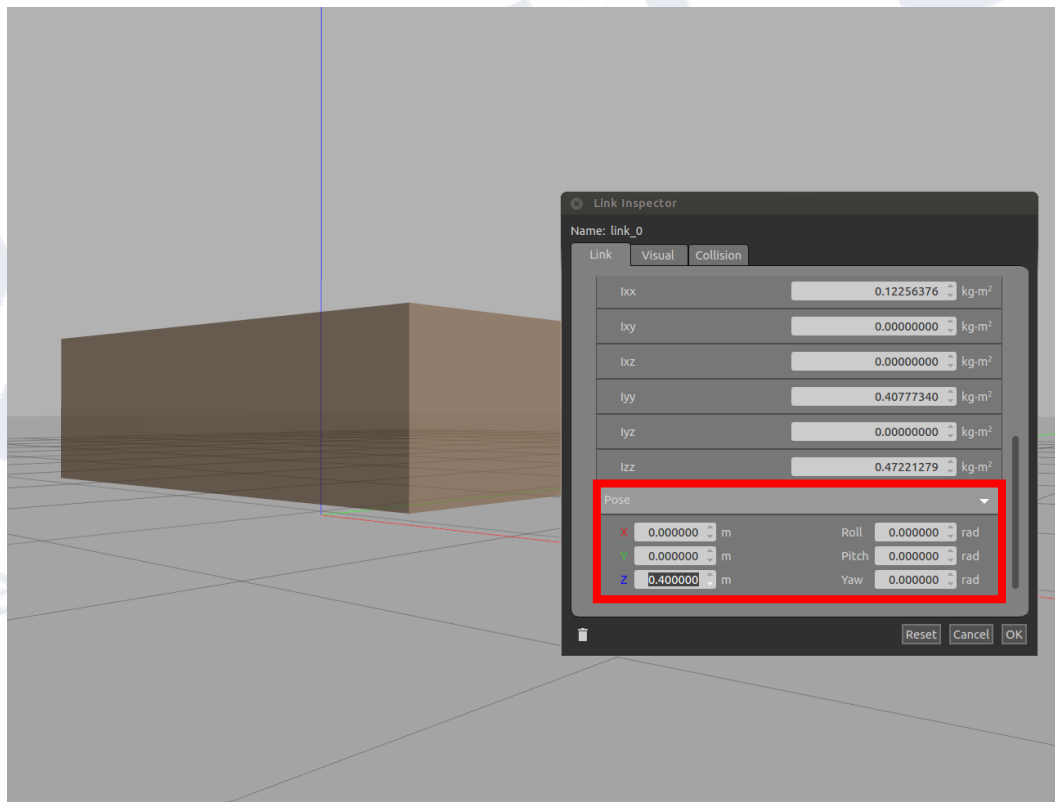


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (2/16)

- Link inspector를 통한 pose 변경
- 객체를 두 번 클릭하면 inspector를 불러올 수 있음
- Pose 항목에서 $x \rightarrow 0.0m$, $y \rightarrow 0.0m$, $z \rightarrow 0.4m$ 로 변경

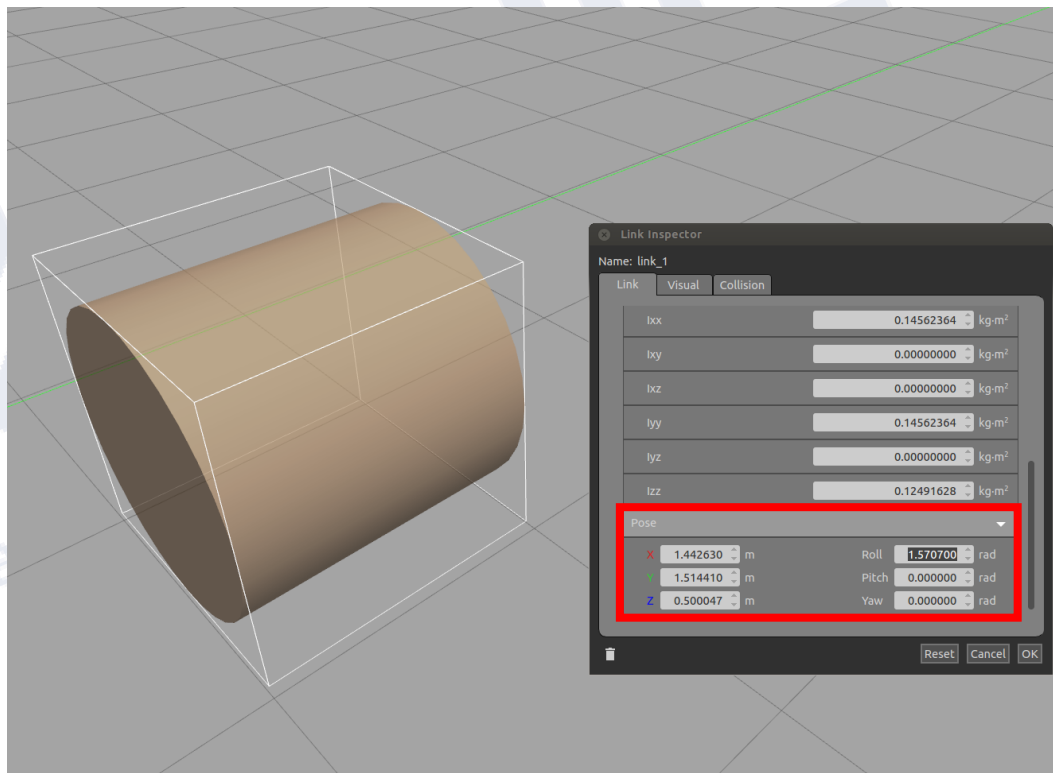


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (3/16)

- 앞 바퀴 생성
- 기본 방향의 원통 모델을 만들고, link inspector에서 X축을 따라 회전시키기 위해 Pose 항목에서 Roll $\rightarrow$ 1.57rad (90 deg) 로 변경

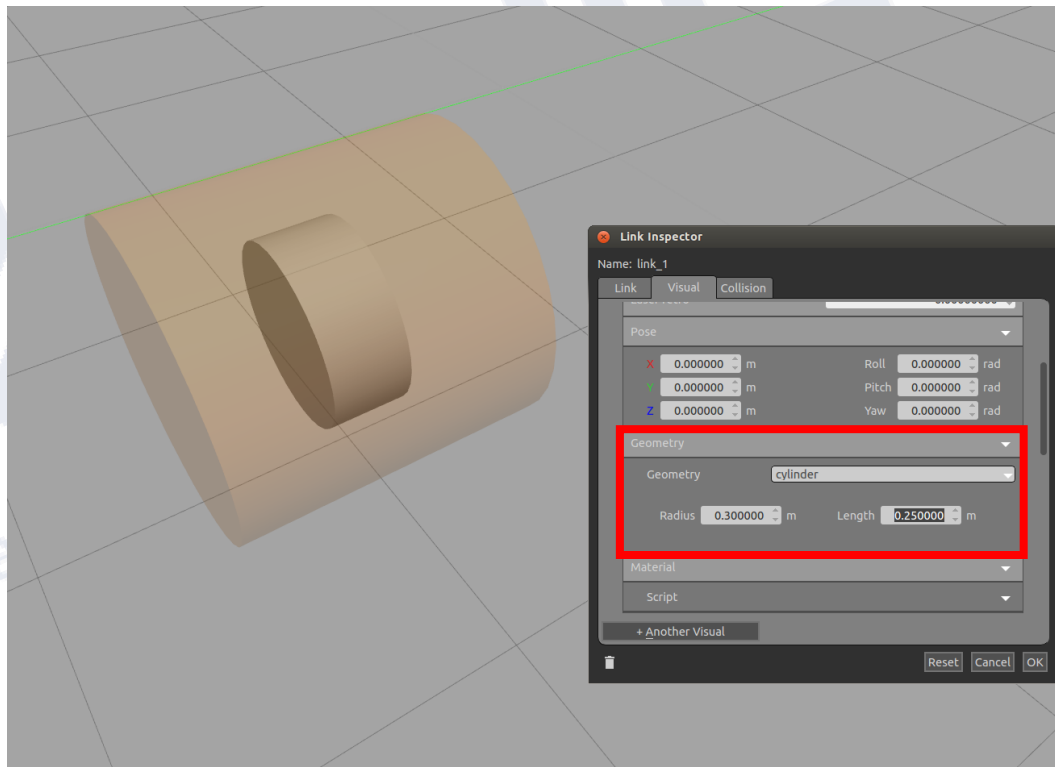


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (4/16)

- 정확한 치수를 지정하여 휠 크기 조정
- Visual 탭의 Geometry 항목에서 **Radius** → **0.3m**, **Length** → **0.25m** 로 변경
- 시각적 지오메트리만 변경된 상태라 collision 항목도 동일하게 변경해주어야 함

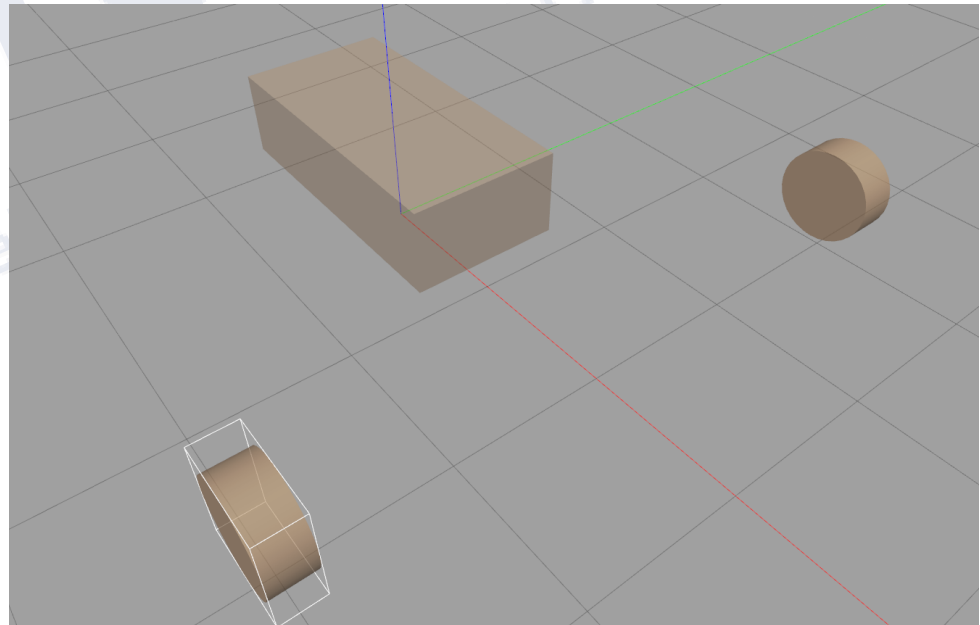
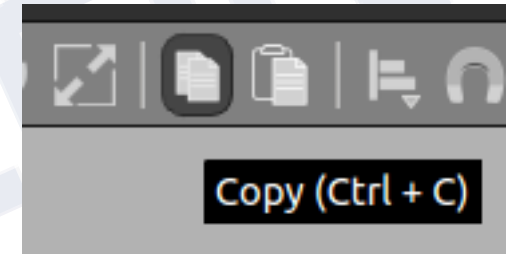


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (5/16)

- 휠 복제
- 휠을 선택하고 상단 툴바에서 복사 아이콘을 선택
- 붙여넣기 아이콘을 선택하고 복사본을 삽입



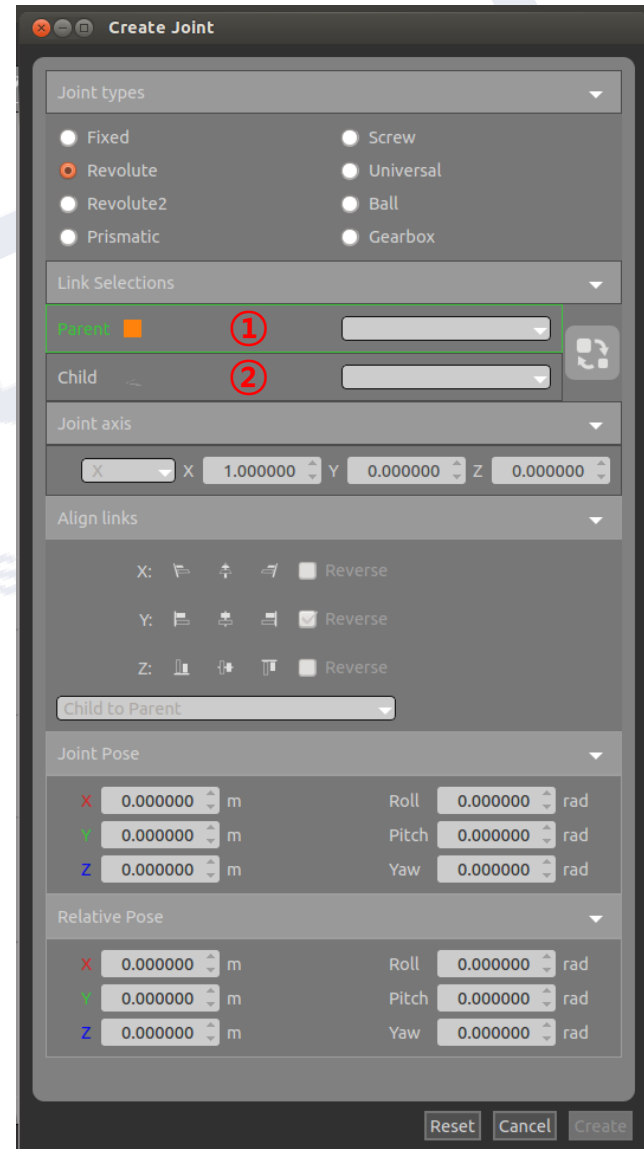
2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (6/16)

- Joint 추가

- ① 생성하려는 joint의 parent로 차체를 선택
- ② Child로 차량 휠을 선택
- ③ 주황색 선으로 차체와 휠이 연결되었는지 확인

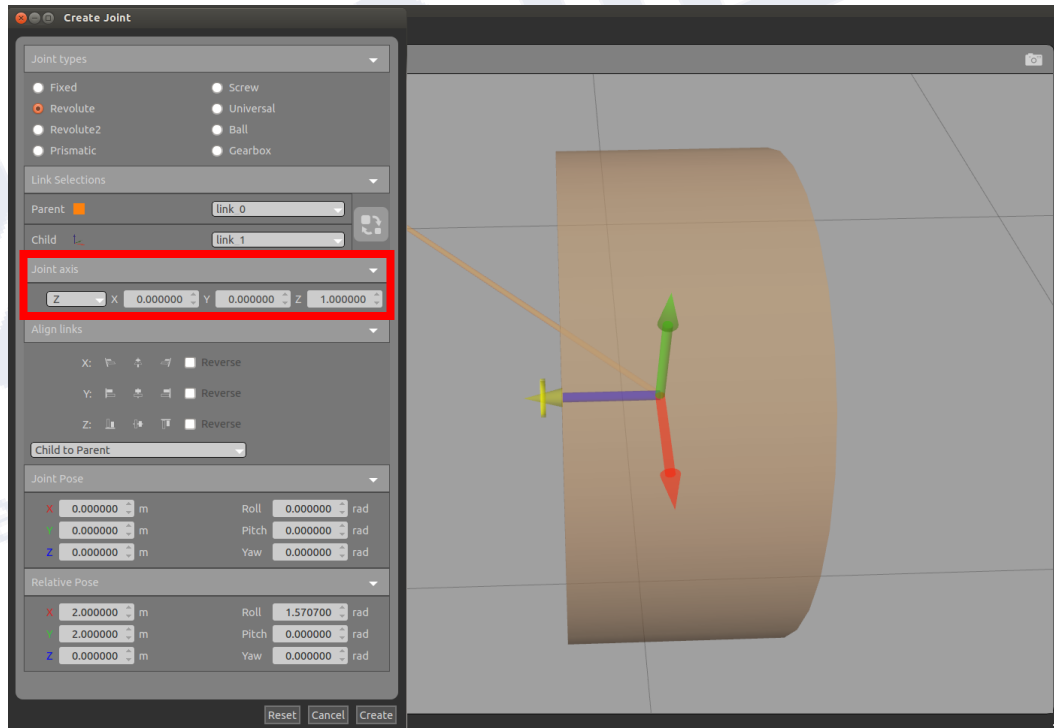


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (7/16)

- 휠 회전축 구성
- Joint axis 항목을 찾고 **파라미터를 (0, 0, 1) 또는 Z** 로 변경
- 노란색 링이 파란색 화살표 위에 있어야 정확한 회전축 설정이 완료된 것임

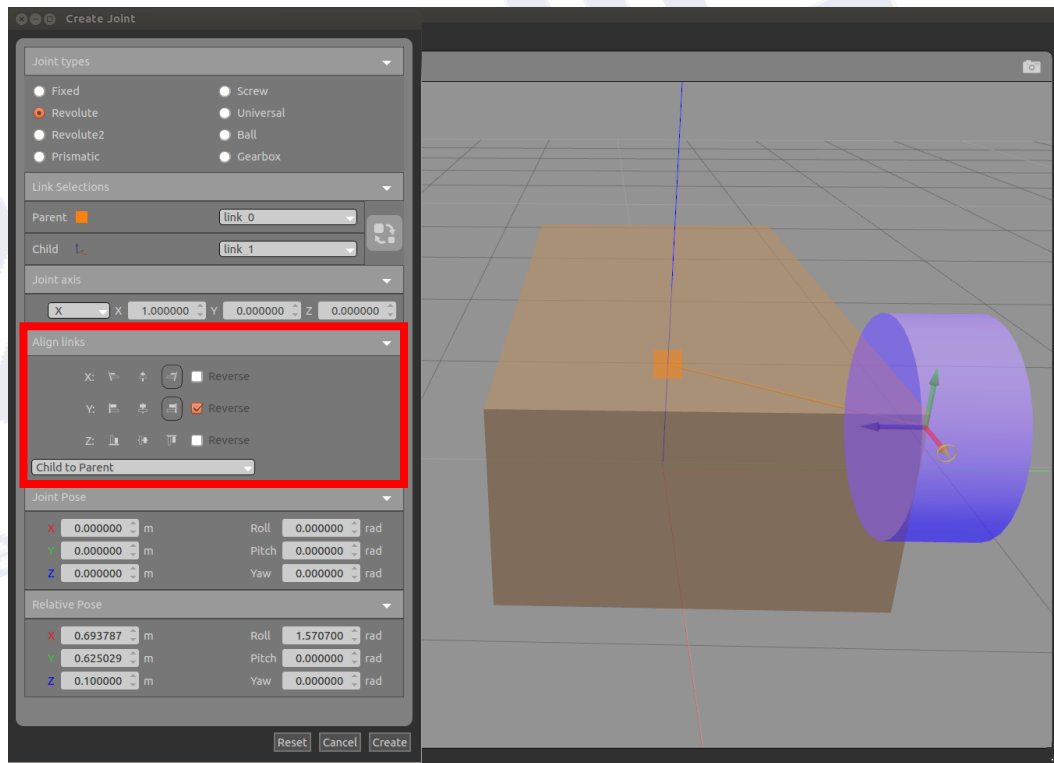


2. SLAM

4) Turtlebot3 SLAM 실습

● Gazebo 모델 생성 (8/16)

- 휠 포즈 정렬
- X축: 휠을 먼저 선택하고, **X Align Max** 선택
- Y축: **Y Align Max** 선택, **Reverse** 를 체크하여 child의 최대값에 정렬

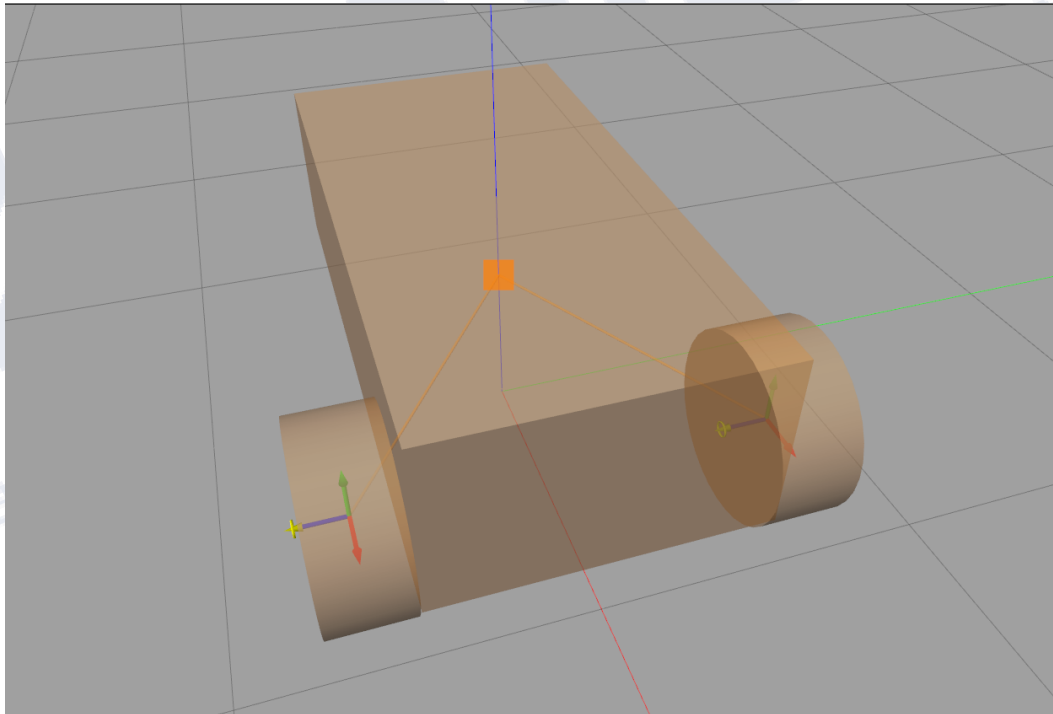


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (9/16)

- 휠의 link inspector를 열고 Pose 항목에서 $z \rightarrow 0.3\text{m}$ 로 변경 (휠의 반지름)
- 반대쪽 휠도 동일하게 설정
- 반대쪽 휠의 Y축은 Y Align Min 으로 해야함

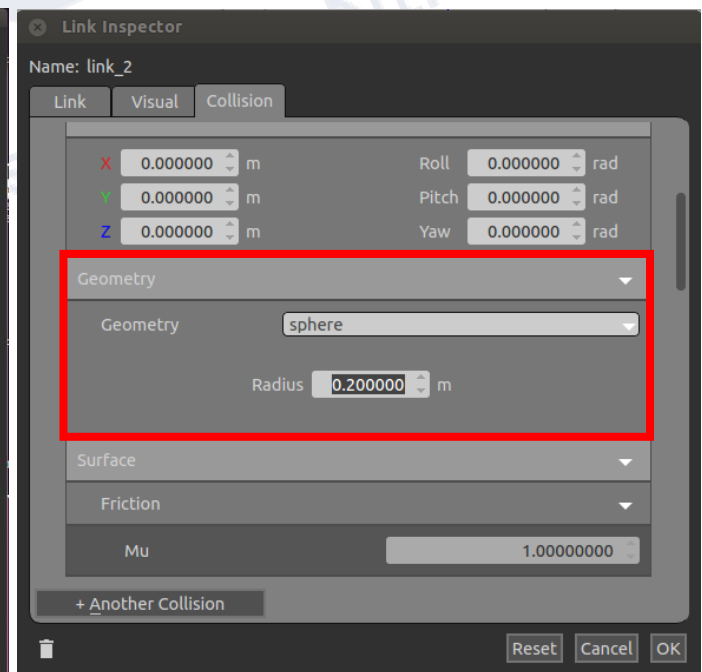
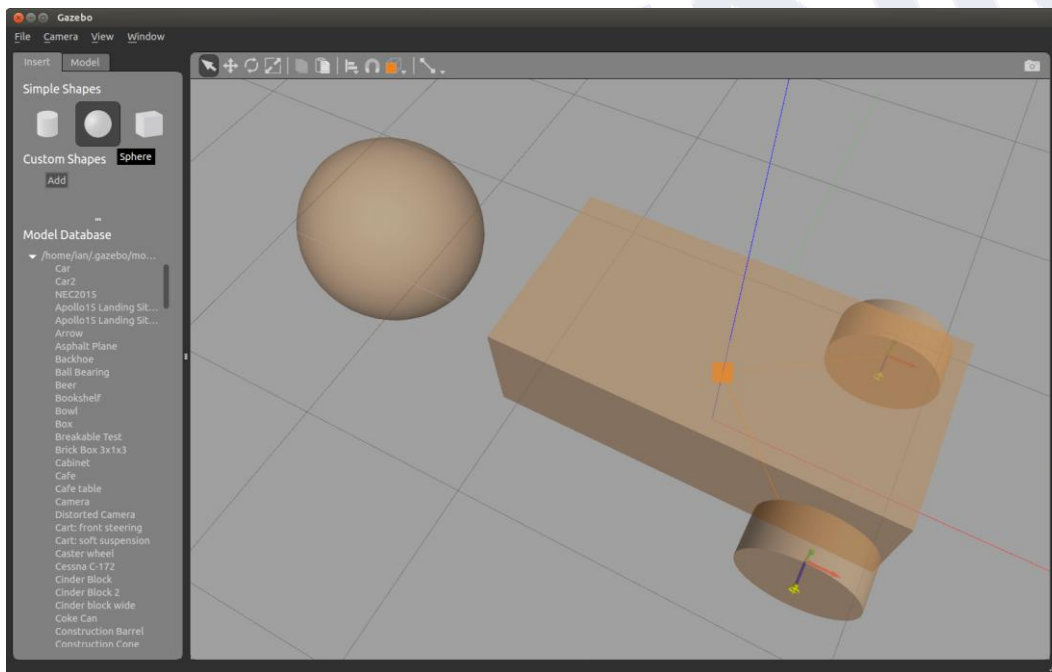


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (10/16)

- 캐스터 휠
- Left panel에서 구형 모델을 선택하고 생성
- Link inspector의 Visual 및 Collision 탭의 Geometry 항목에서 **Radius** → **0.2m** 로 변경

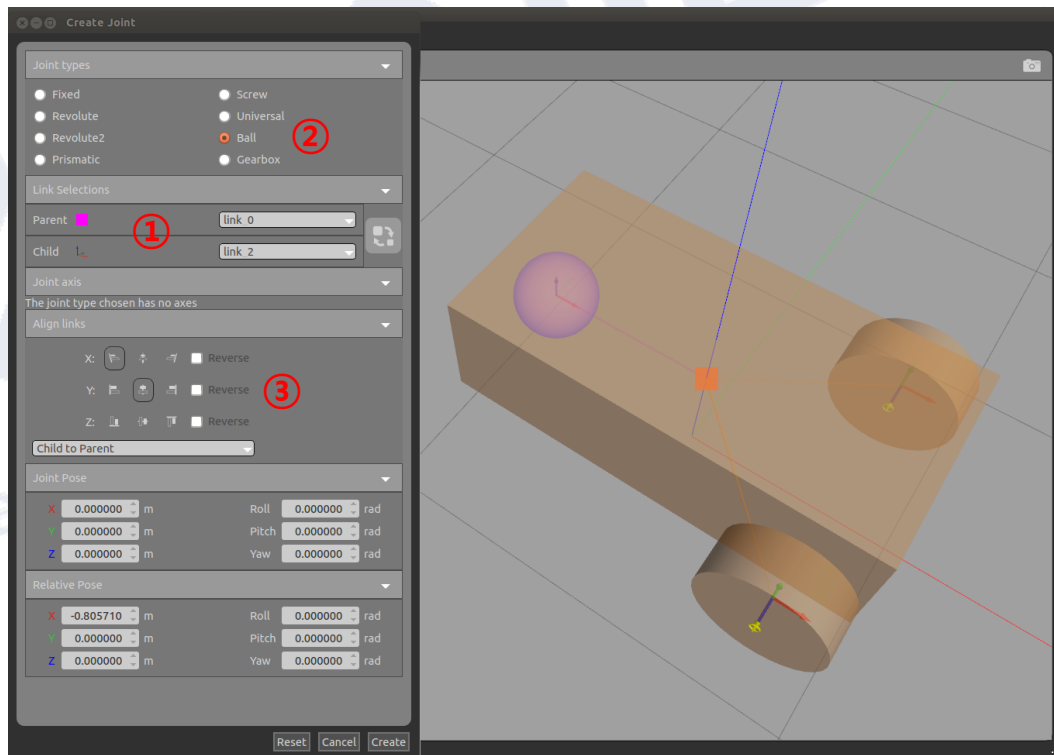


2. SLAM

4) Turtlebot3 SLAM 실습

● Gazebo 모델 생성 (11/16)

- ① Link Selections 항목에서 앞서 한 방식과 같이 joint를 설정
- ② Joint types 항목에서 **Ball** 로 선택
- ③ Align links 항목에서 **X Align Min, Y Align Center** 로 변경
- ④ Link inspector의 Pose 항목에서 **z → 0.2m** 로 변경 (캐스터 휠의 반지름)

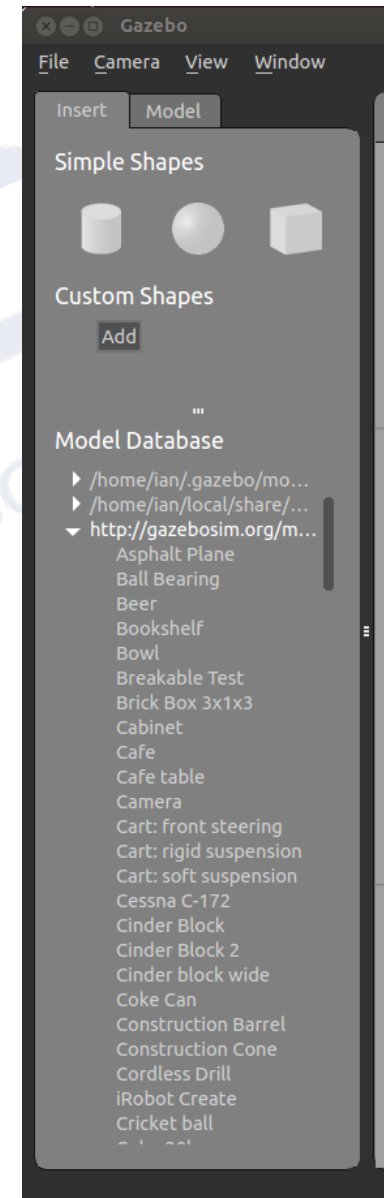
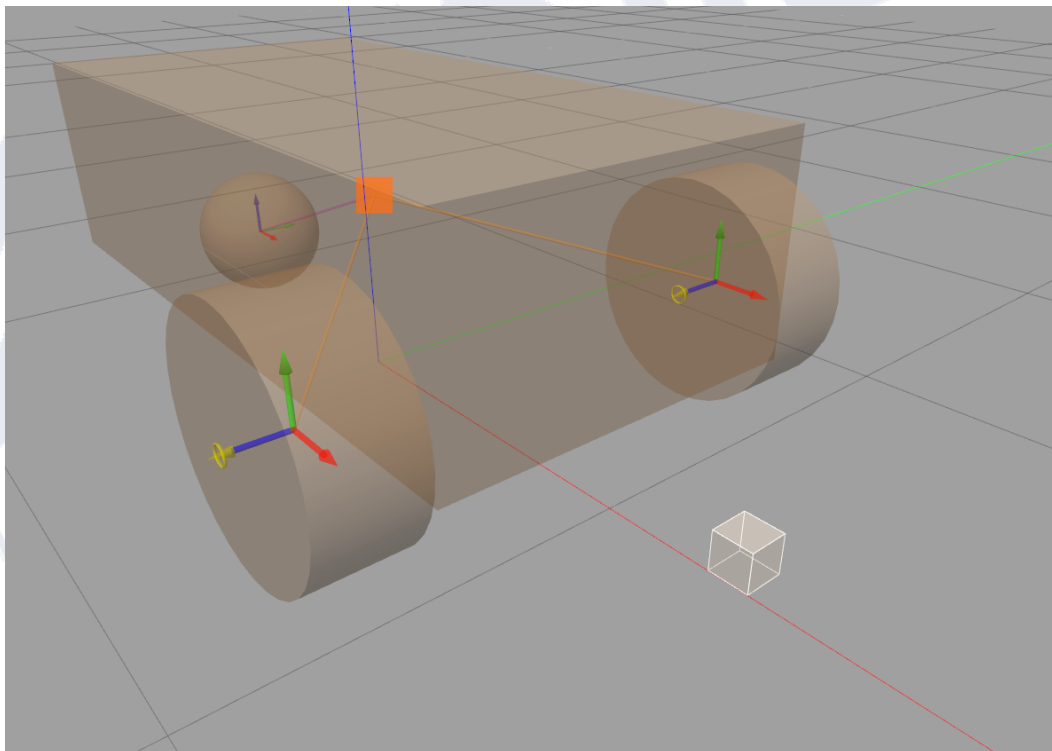


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (12/16)

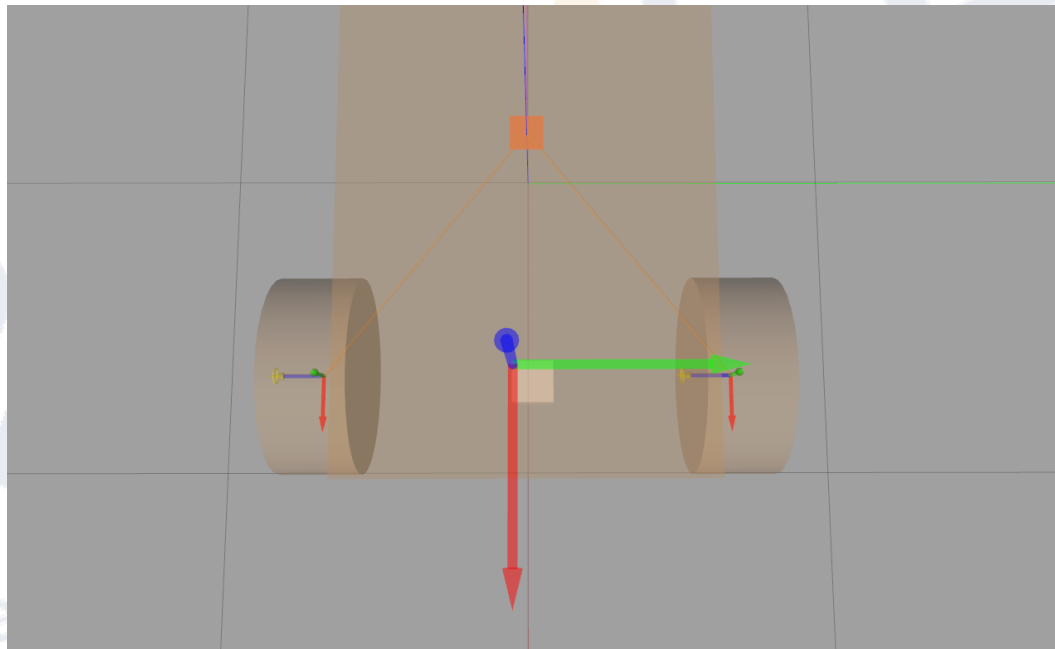
- 센서 추가
- Insert 탭의 Model Database에서 Depth Camera를 선택
- 빈 공간에 큐브 형태의 depth camera를 생성



2. SLAM

4) Turtlebot3 SLAM 실습

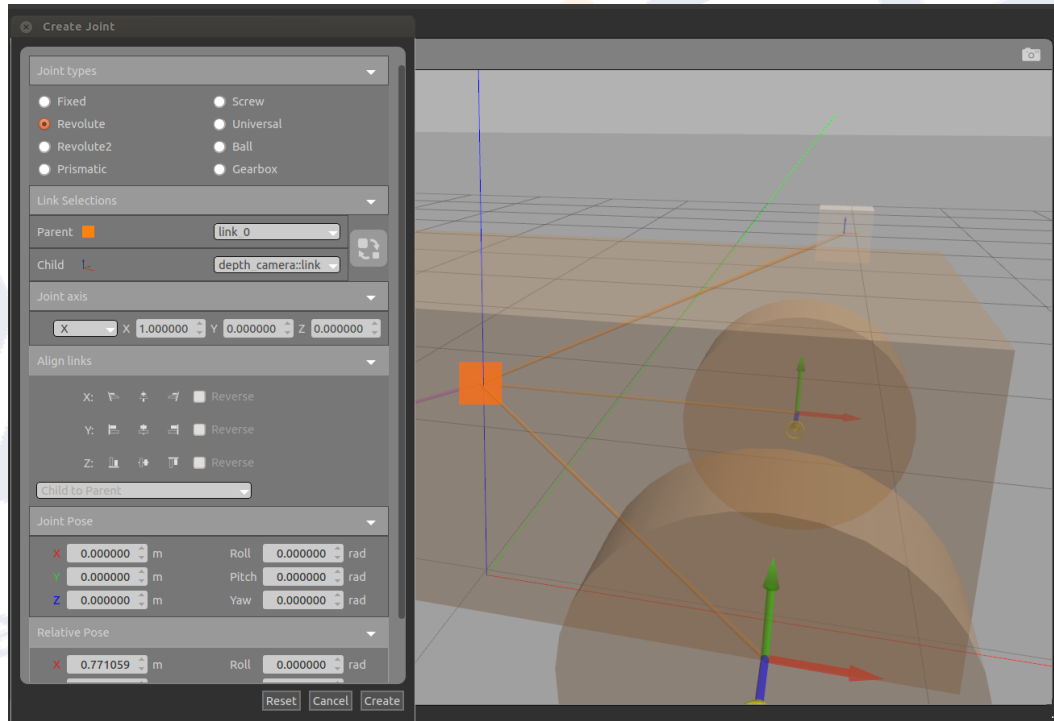
- Gazebo 모델 생성 (13/16)
 - 차체 앞쪽 대략적인 위치에 카메라를 이동



2. SLAM

4) Turtlebot3 SLAM 실습

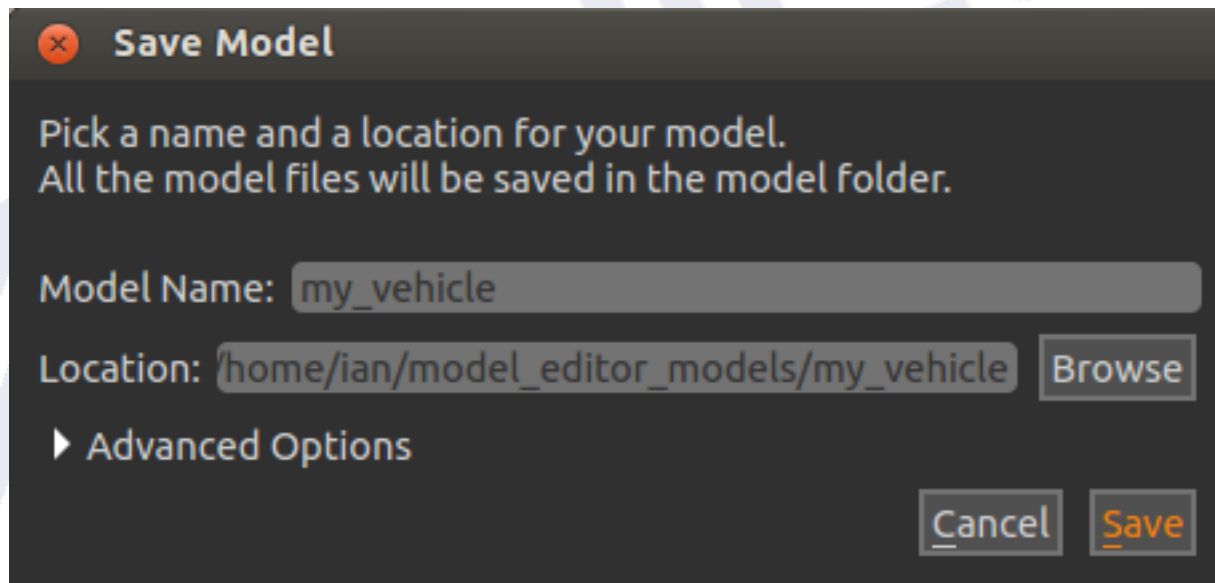
- Gazebo 모델 생성 (14/16)
 - 이전과 같이 조인트 링크를 생성하고, Joint types 을 **Fixed** 로 변경



2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (15/16)
 - 모델 저장
 - File 메뉴에서 모델 이름 및 위치를 설정하고 저장

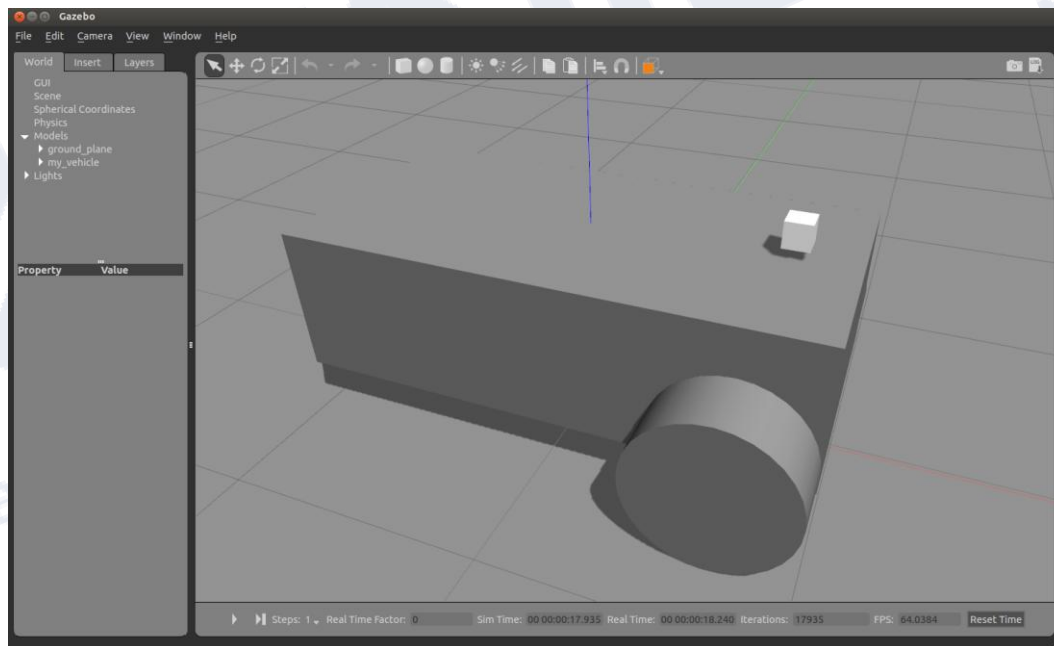


2. SLAM

4) Turtlebot3 SLAM 실습

- Gazebo 모델 생성 (16/16)

- 시뮬레이션 실행
- 모델 편집기를 종료. File 메뉴에서 **Exit Model Editor**를 선택
- **Play** 버튼을 눌러 시뮬레이션 실행 가능



2. SLAM

4) Turtlebot3 SLAM 실습

- Turtlebot3 패키지 설치

```
$ sudo apt update -y
$ sudo apt install ros-noetic-gmapping
$ sudo apt install ros-noetic-navigation
```

```
$ cd ~/catkin_ws/src
$ git clone -b noetic https://github.com/ROBOTIS-GIT/turtlebot3.git
$ git clone -b noetic https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
$ git clone -b noetic https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
$ cd ~/catkin_ws && catkin_make
```

- ~/.bashrc 에 터틀봇 모델을 불러오는 구문 추가

```
export TURTLEBOT3_MODEL=burger
or
export TURTLEBOT3_MODEL=waffle_pi
```

- bashrc 소싱 또는 터미널 재시작

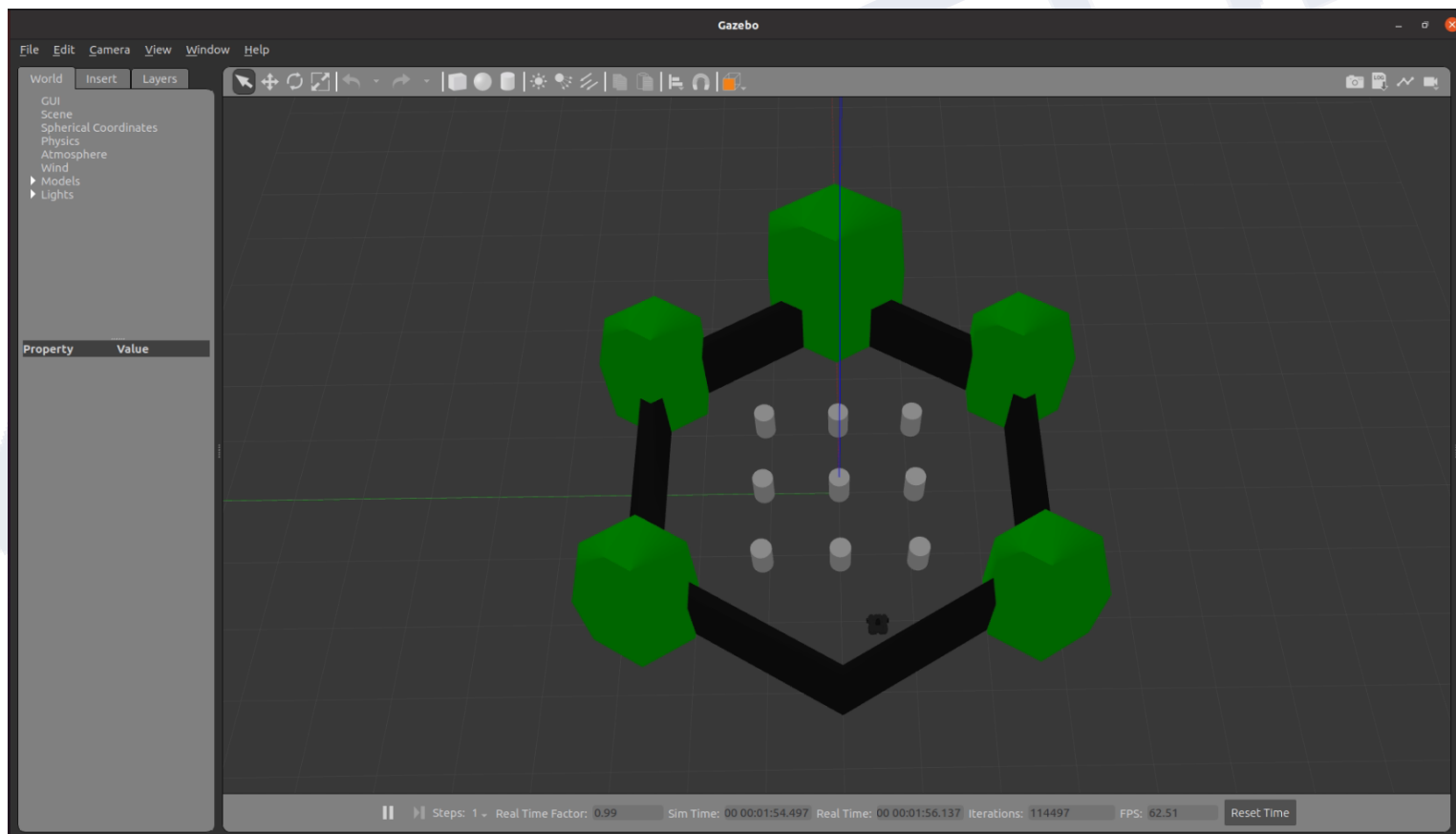
```
$ source ~/.bashrc
```

2. SLAM

4) Turtlebot3 SLAM 실습

- Turtlebot3 시뮬레이션 환경

```
$ roslaunch turtlebot3_gazebo turtlebot3_world.launch
```



4) Turtlebot3 SLAM 실습

- ## ● Turtlebot3 키보드 제어

```
$ roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

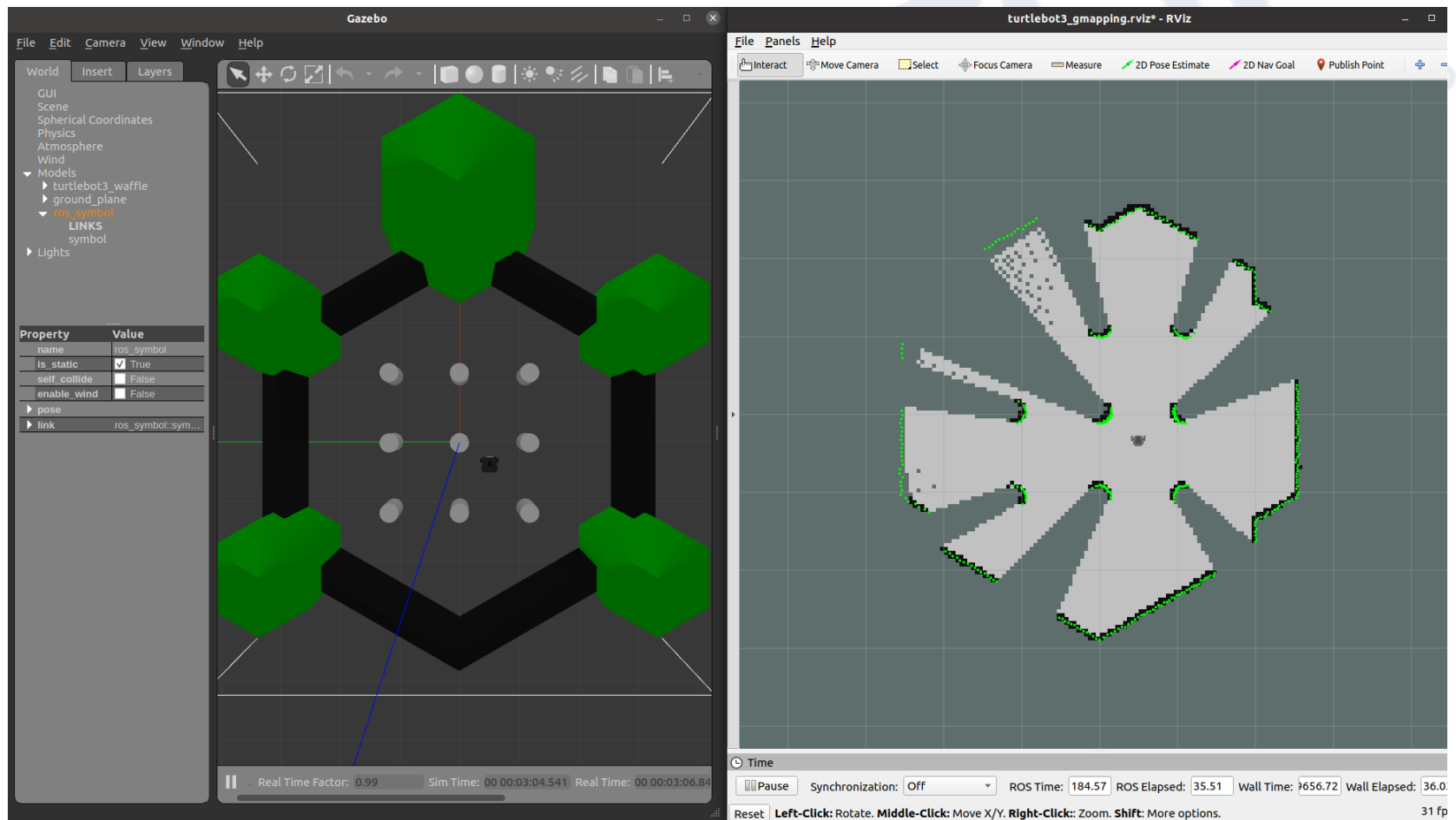


2. SLAM

4) Turtlebot3 SLAM 실습

- Turtlebot3 SLAM

```
$ roslaunch turtlebot3_slam turtlebot3_slam.launch
```

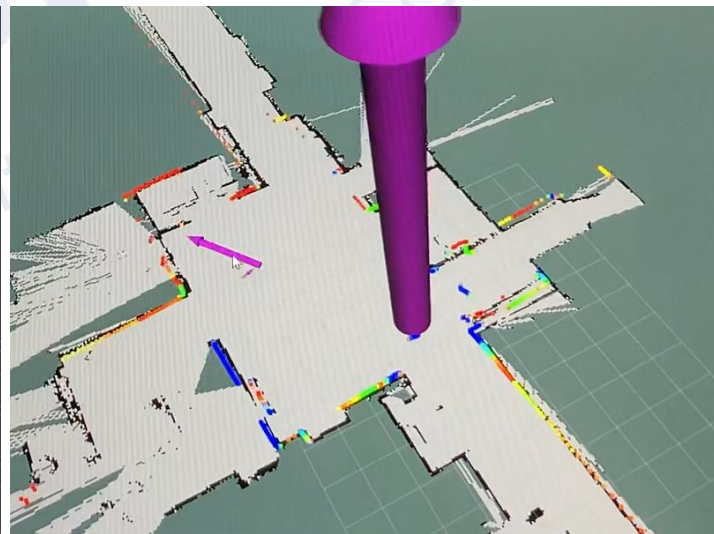
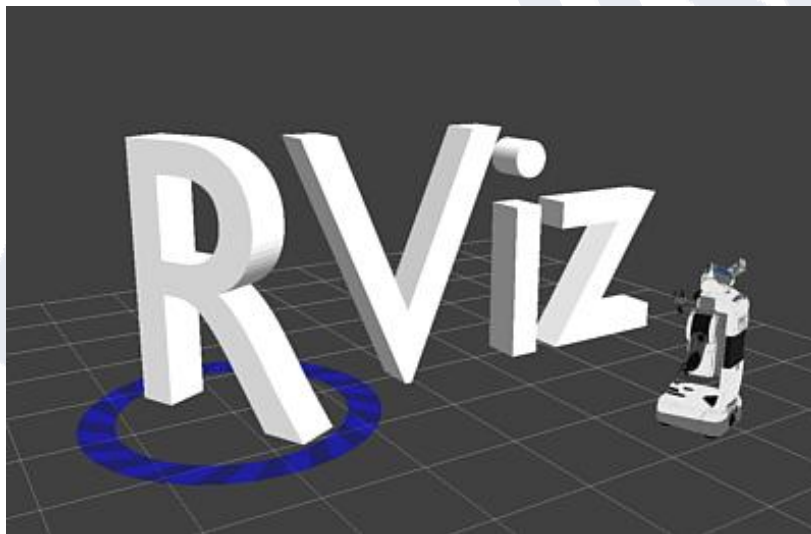


2. SLAM

4) Turtlebot3 SLAM 실습

- Rviz: Visualization tool

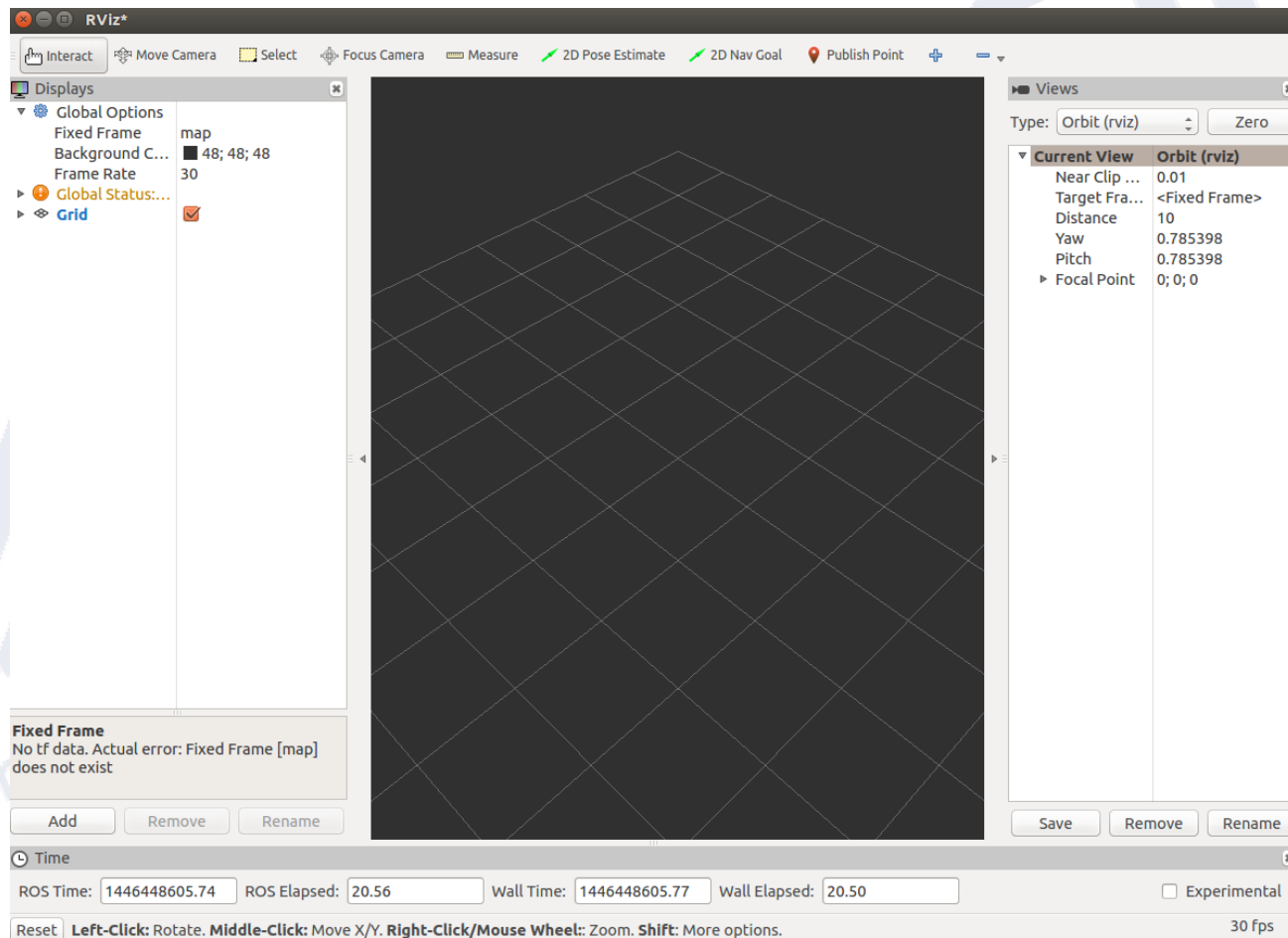
- ROS 3D 시각화 툴
- 다양한 센서(LiDAR, Radar, IMU, encoder 등)들의 데이터를 시각화
- 로봇의 자표 및 움직임을 나타내는 transform, URDF 표현 가능
- 모바일 로봇, 드론, 매니퓰레이터 등의 플랫폼을 원격으로 제어할 수 있음



2. SLAM

4) Turtlebot3 SLAM 실습

- Rviz: Visualization tool
 - 초기 모습

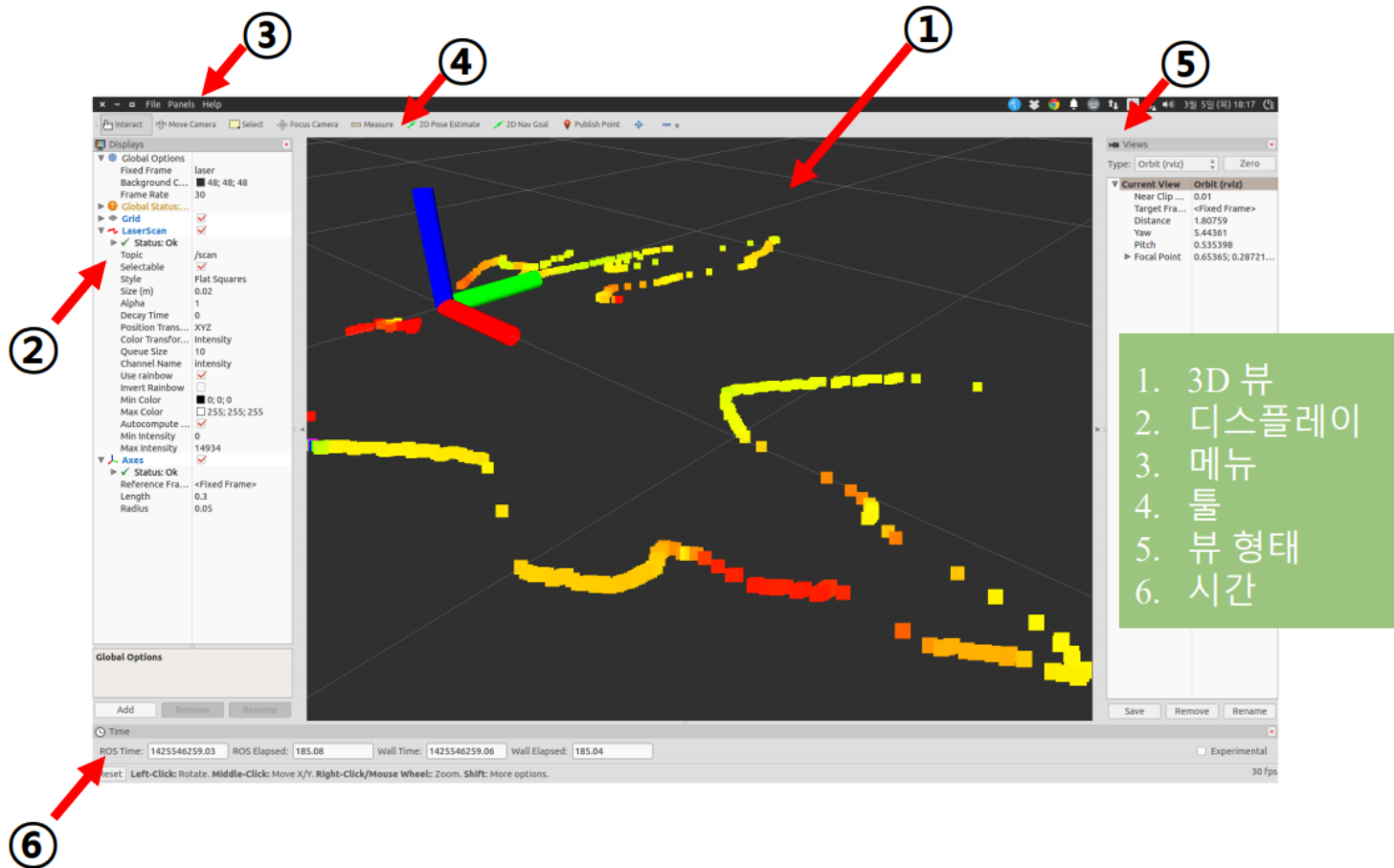


2. SLAM

4) Turtlebot3 SLAM 실습

- Rviz: Visualization tool

- 화면 구성

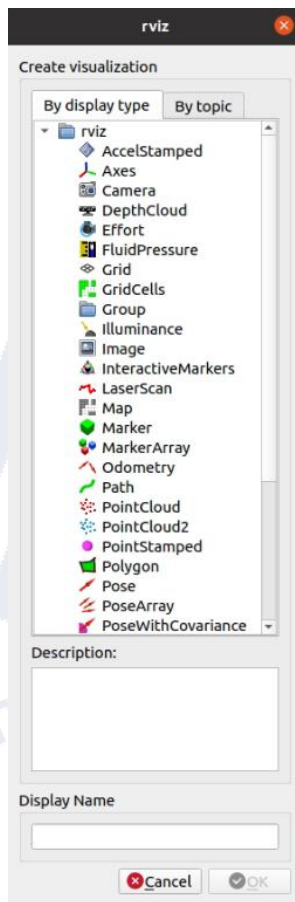


2. SLAM

4) Turtlebot3 SLAM 실습

- Rviz: Visualization tool

- 시각화 가능한 항목
- 좌측하단의 'Add' 버튼을 눌러 추가 가능



	xyz 축
	카메라 영상 오버레이
	거리영상에 카메라 영상을 입힘
	회전 관절의 힘
	유체 압력
	그리드
	그리드 셀 (지도에 이용)
	그룹
	조도
	영상
	인터랙티브 마커
	레이저 스캔
	지도
	마커
	마커 배열
	오도메트리

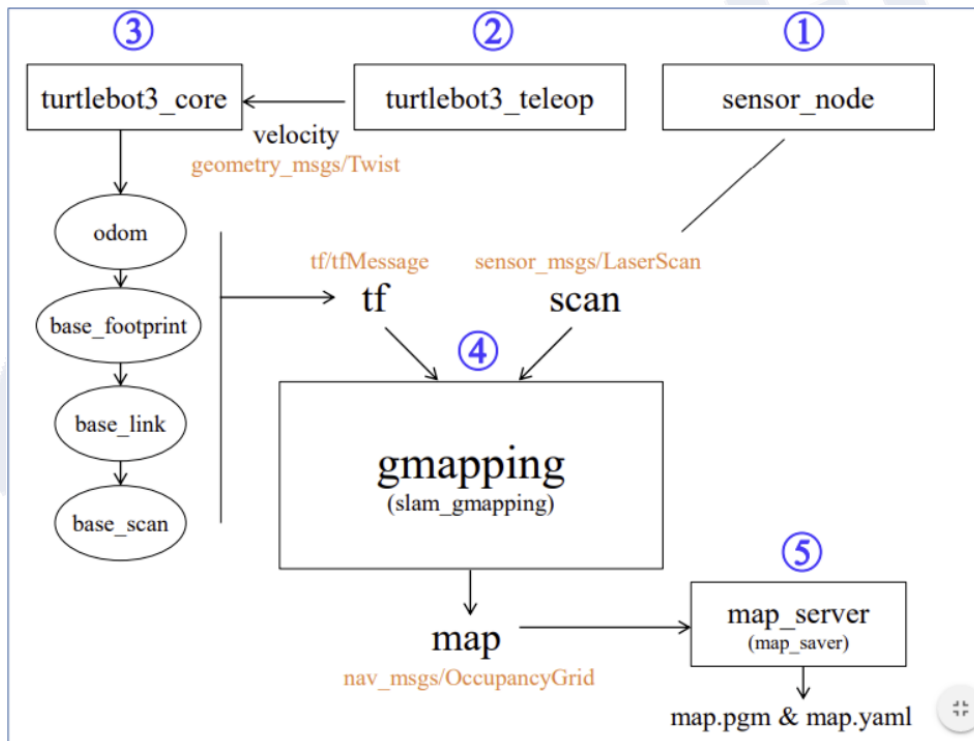
	경로
	포인트 클라우드
	포인트 클라우드2
	점
	폴리곤
	포즈
	포즈 배열
	범위
	상대 온도
	로봇 모델
	좌표 변환 값 (TF)
	상대 습도
	쥐어 돌림

2. SLAM

4) Turtlebot3 SLAM 실습

● Gmapping 구조

- Gmapping은 로봇이 얼마나 이동했는지를 추정하는 주행 데이터인 Odometry 데이터와 2D LiDAR를 통한 Laser scan 데이터, 그리고 Odometry의 추정 정확도를 향상시키기 위한 IMU 데이터를 입력으로 받으며, Occupancy Grid Map을 출력으로 도출함



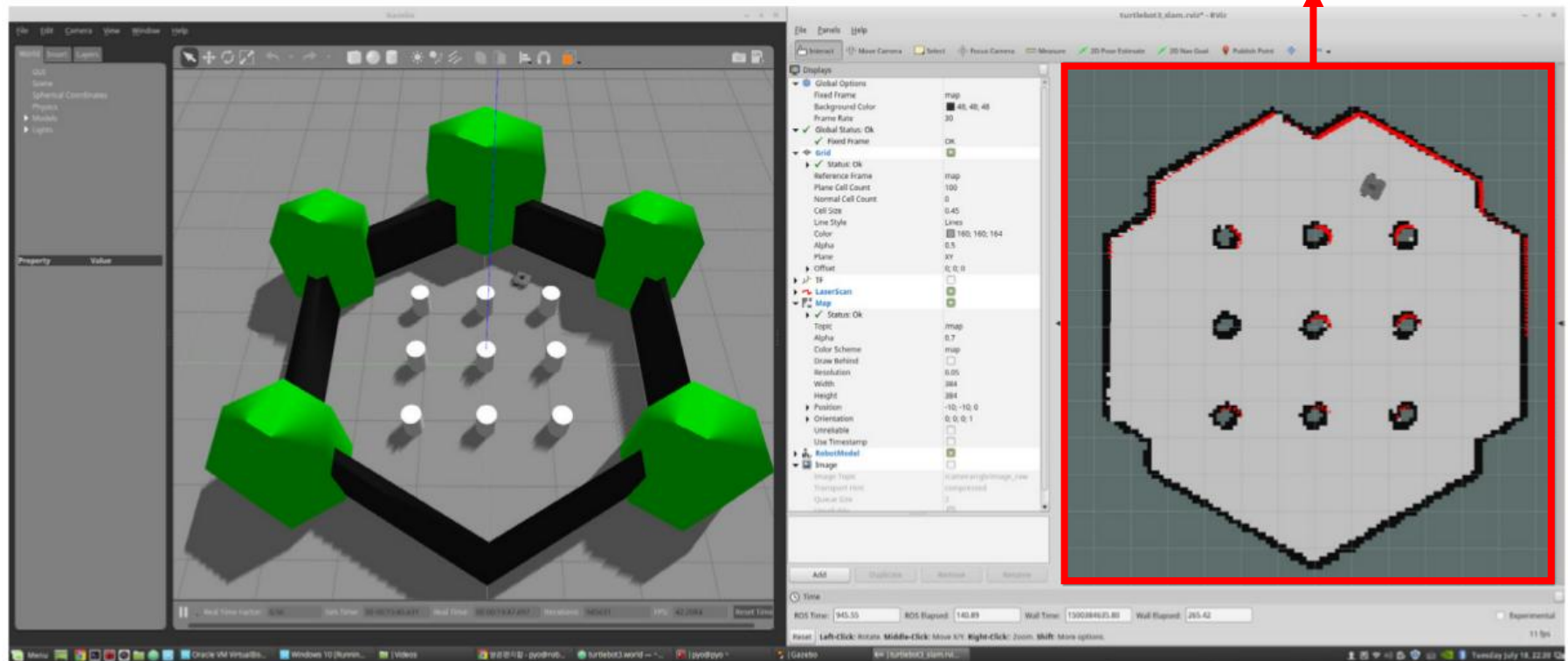
	Gmapping
개발사	OpenSLAM
라이선스	BSD-3-Clause
특징	2D
	개발 지원 중단
	Static한 환경에 유리
입력 데이터	Odometry 데이터
	Laser scan 데이터
	IMU 데이터
출력 데이터	Occupancy Grid Map 데이터

2. SLAM

4) Turtlebot3 SLAM 실습

- Occupancy grid map

“Occupancy Grid Map”



2. SLAM

4) Turtlebot3 SLAM 실습

- Occupancy grid map

- Occupancy grid map은 이산적인 셀로 구성된 지도로 각 셀은 특정 공간의 점유 상태를 나타냄
- 각 셀은 비어 있을 확률과 점유되어 있을 확률을 가지고 있으며, 이는 센서의 불확실성과 노이즈를 반영한 것
- SLAM 과정에서 지도는 지속적으로 업데이트되며, 새로운 센서 데이터가 수집될 때마다 수정 및 개선됨
- Occupancy grid map 을 구성하는 하나의 셀은 0부터 100까지의 정수 값으로 이루어짐.
- 0은 해당 셀이 완전한 **free space**임을 나타내며, 해당 셀 영역에 **장애물이 없다는 것**을 의미.
100은 해당 셀이 완전히 **점유되어 있음**을 나타내며, 해당 셀 영역에 **장애물이 있다는 것**을 의미.
-1 또는 **unknown**은 해당 셀의 **상태가 알려지지 않았음**을 나타내며, 로봇은 해당 셀 영역의 정보를 취득하지 못했거나 셀에 대한 확신이 없는 경우 해당 값을 사용함.

```
$ rostopic echo /map
```

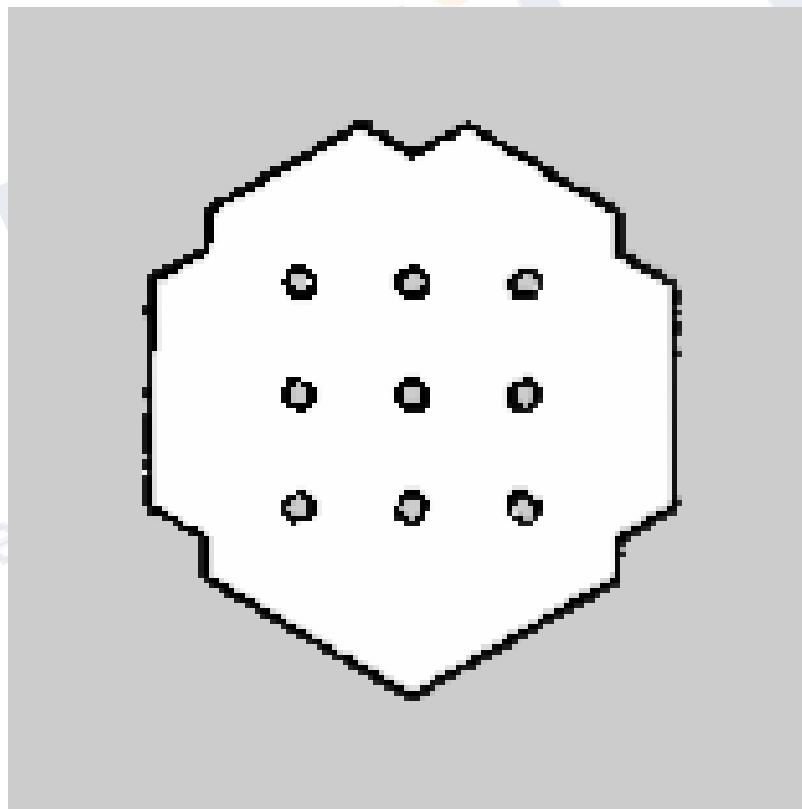
2. SLAM

4) Turtlebot3 SLAM 실습

- Turtlebot3 map save

```
$ rosrun map_server map_saver -f [file_name]
```

```
$ rosrun map_server map_saver -f ~/map
```



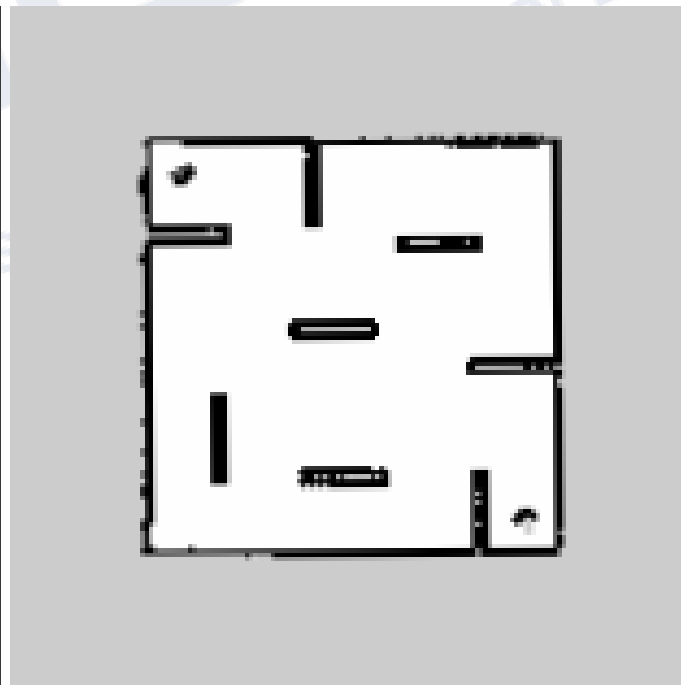
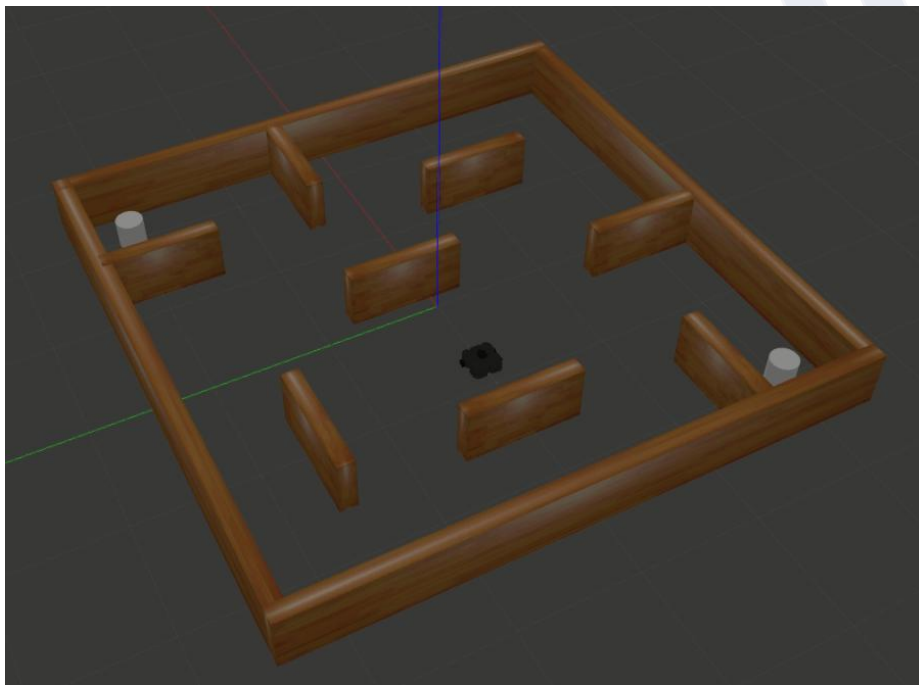
2. SLAM

4) Turtlebot3 SLAM 실습

- 연습 문제

- TurtleBot3 시뮬레이션 환경 중 stage4에 대한 지도를 작성

```
$ roslaunch turtlebot3_gazebo turtlebot3_stage_4.launch
```

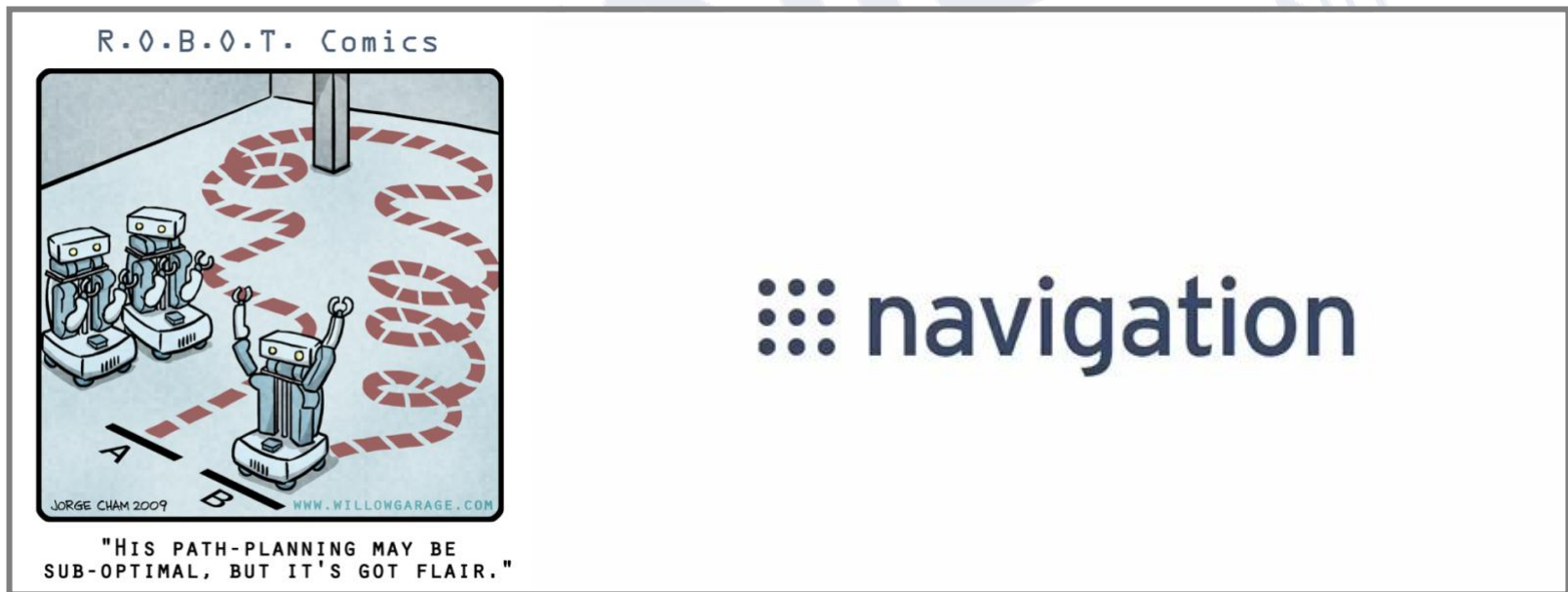


Navigation

3. Navigation

1) What is ROS navigation?

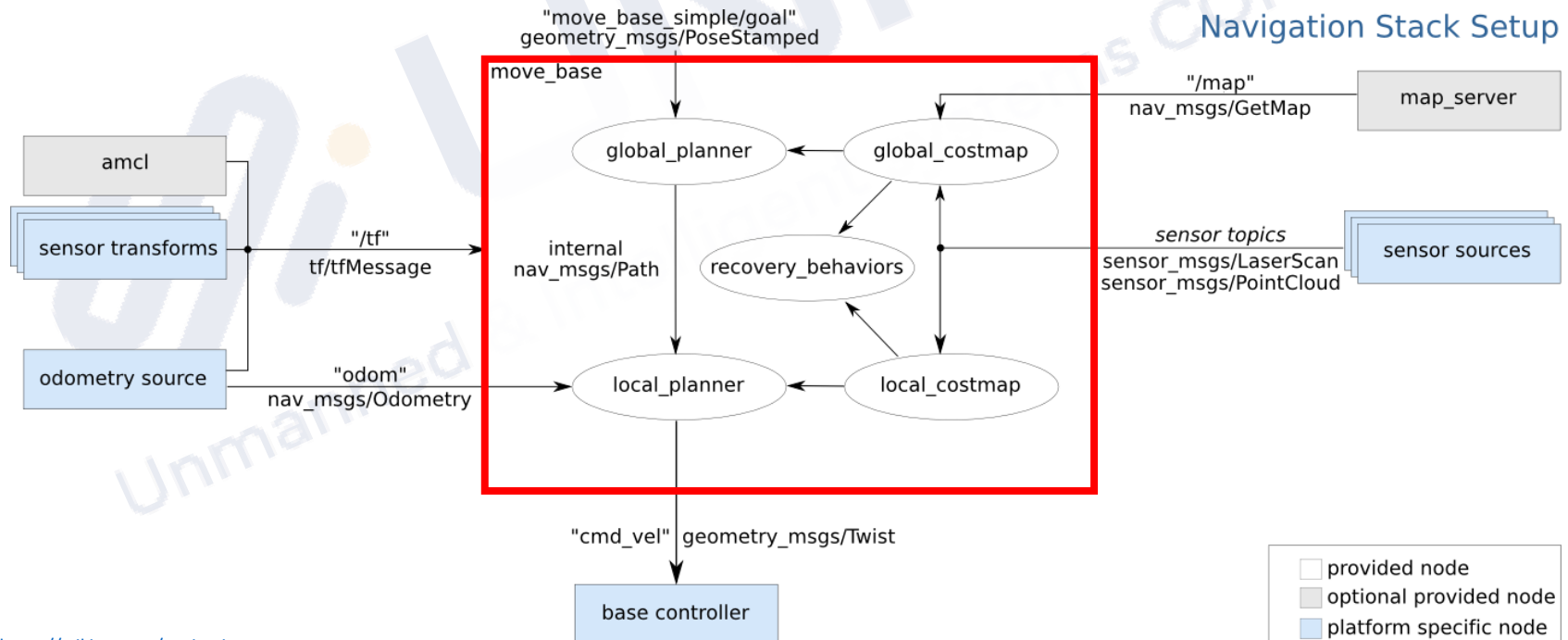
- ROS navigation stack은 로봇이 주변 환경을 인식하고, 계획된 경로를 따라 자율적으로 이동할 수 있도록 하는 데 사용되는 통합 소프트웨어 프레임워크
- ROS navigation stack은 센서 데이터의 통합, 위치 추정, 경로 계획, 장애물 회피, 그리고 실행까지의 전 과정을 관리하며, 특히 이동 로봇의 복잡한 탐색 문제를 해결하는 데 적합



3. Navigation

2) ROS navigation stack 구조

- ROS navigation stack은 **move_base** 노드를 중심으로 구성되어 있으며, move_base 노드를 통해 **로봇이 이동할 경로를 계획하거나 정적·동적 장애물 회피를 위한 제어 입력을 생성**
- move_base 노드는 **global_costmap, local_costmap, global_planner, local_planner**로 구성

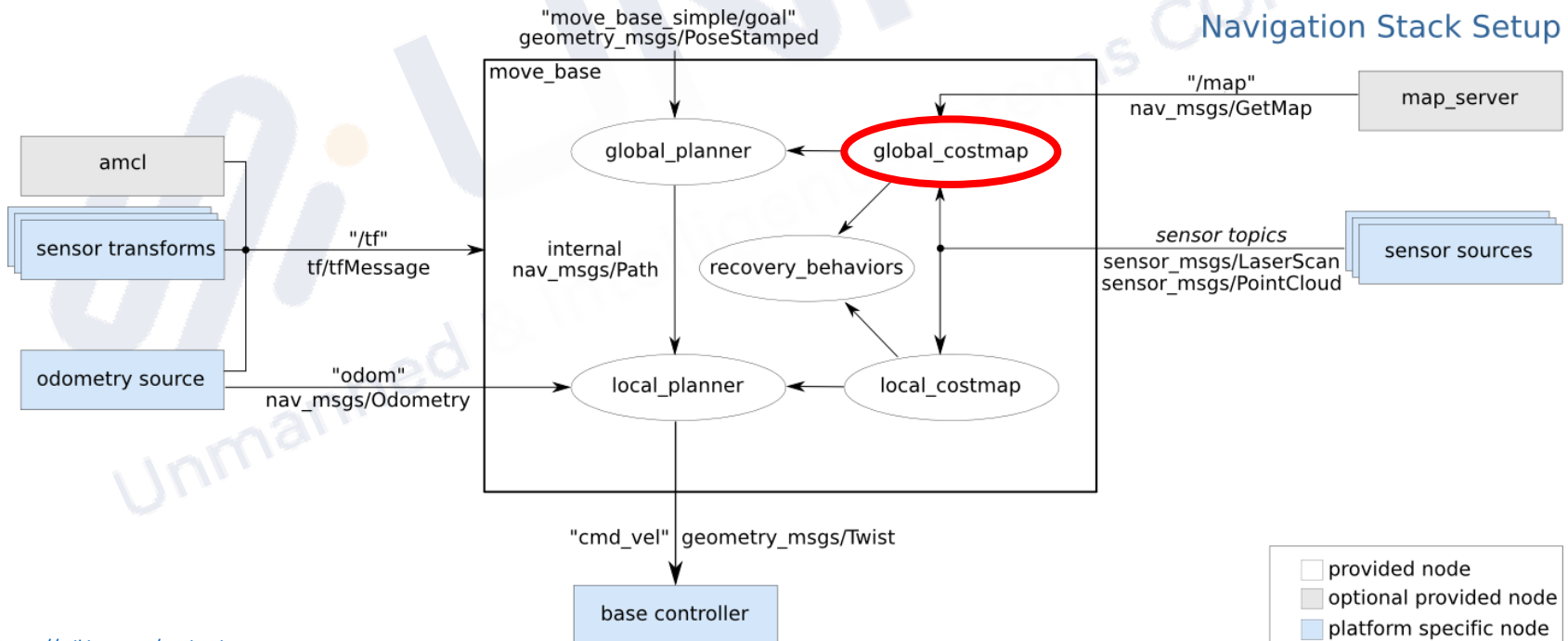


3. Navigation

2) ROS navigation stack 구조

- Global costmap

- Global costmap은 SLAM을 통해 생성한 occupancy grid map을 기반으로, 각 셀이 가지는 비용 값을 재 구성하여 global planner에게 제공

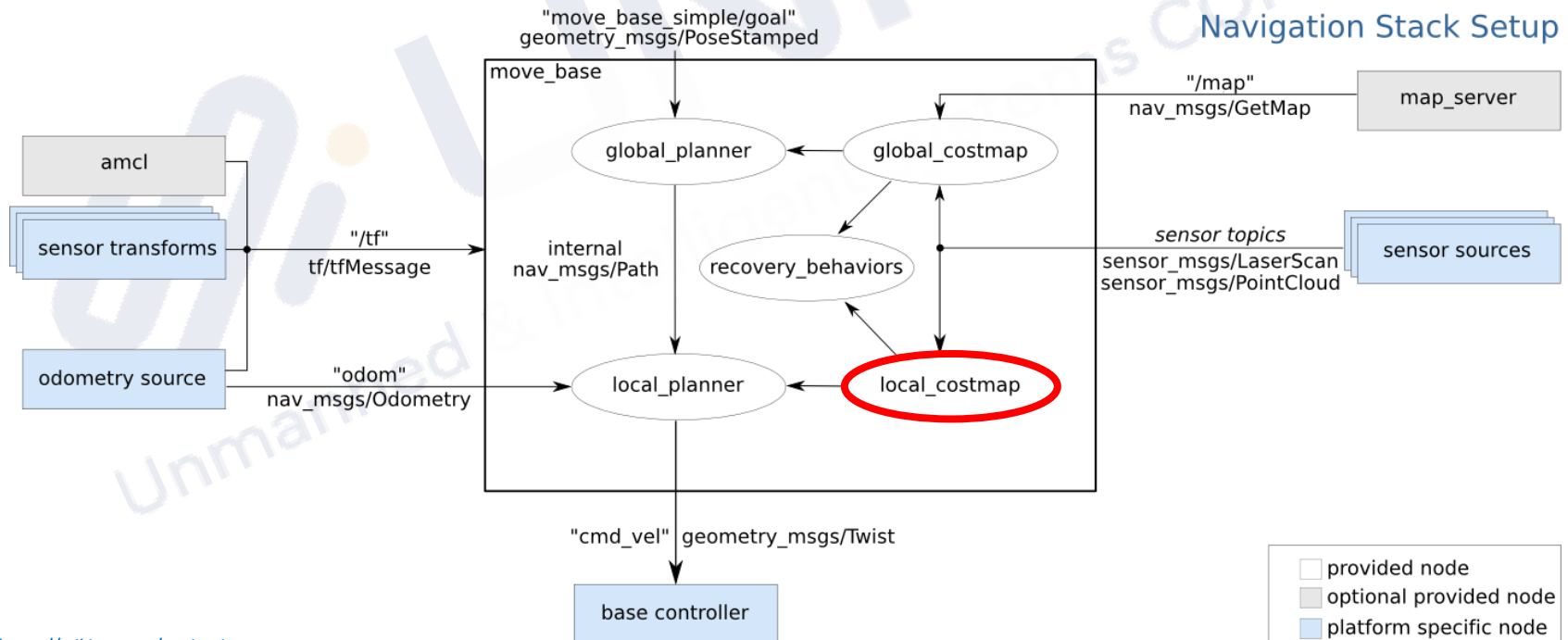


3. Navigation

2) ROS navigation stack 구조

● Local costmap

- Local costmap은 LiDAR 센서 등을 기반으로 로봇 주변의 국소적인 비용 지도를 생성하는 기법
- 생성된 local costmap은 local planner에 제공되며, 이를 통해 사람이나 다른 로봇과 같은 동적 장애물을 회피하기 위한 제어 입력 생성 가능

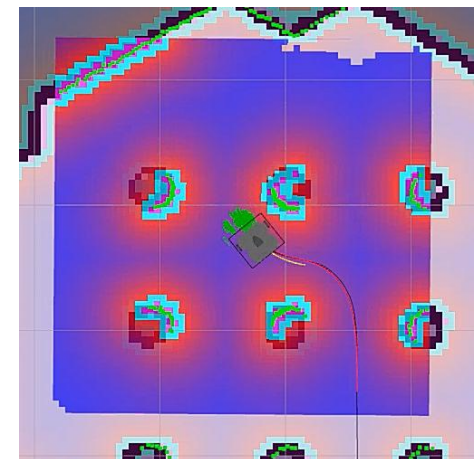
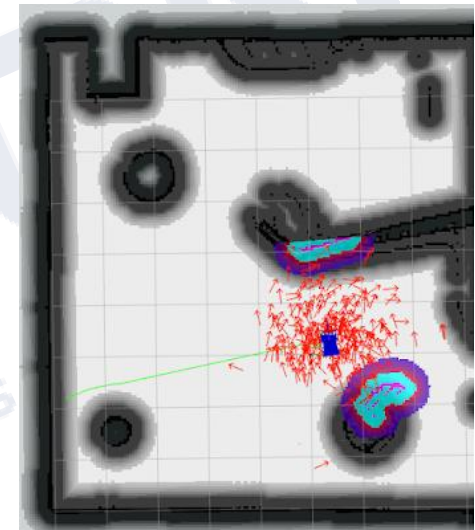


3. Navigation

2) ROS navigation stack 구조

● Costmap

- Costmap은 로봇이 안전하게 이동할 수 있는 공간을 그리드 셀로 표현한 지도
- 각 셀은 0에서 255 사이의 정수 비용 값을 가짐
 - 0: 자유 공간
 - 253: 내접 (셀의 중심이 실제 장애물에서 로봇의 내접 반경보다 가까운 경우를 의미. 따라서 로봇의 중심을 이 셀로 이동시키면 충돌 발생)
 - 254: 치명적 장애물
 - 255: 알 수 없는 정보
- Global planner는 장기 계획을 위해 global costmap을 사용하며, local planner는 단기 계획을 위해 local costmap을 사용함. Global costmap은 정적/저장된 지도로부터 생성되며, local costmap은 실시간 센서 데이터를 사용하여 생성됨. 오른쪽 그림에서 흑백 부분은 global costmap을, 색 부분은 local costmap을 나타냄

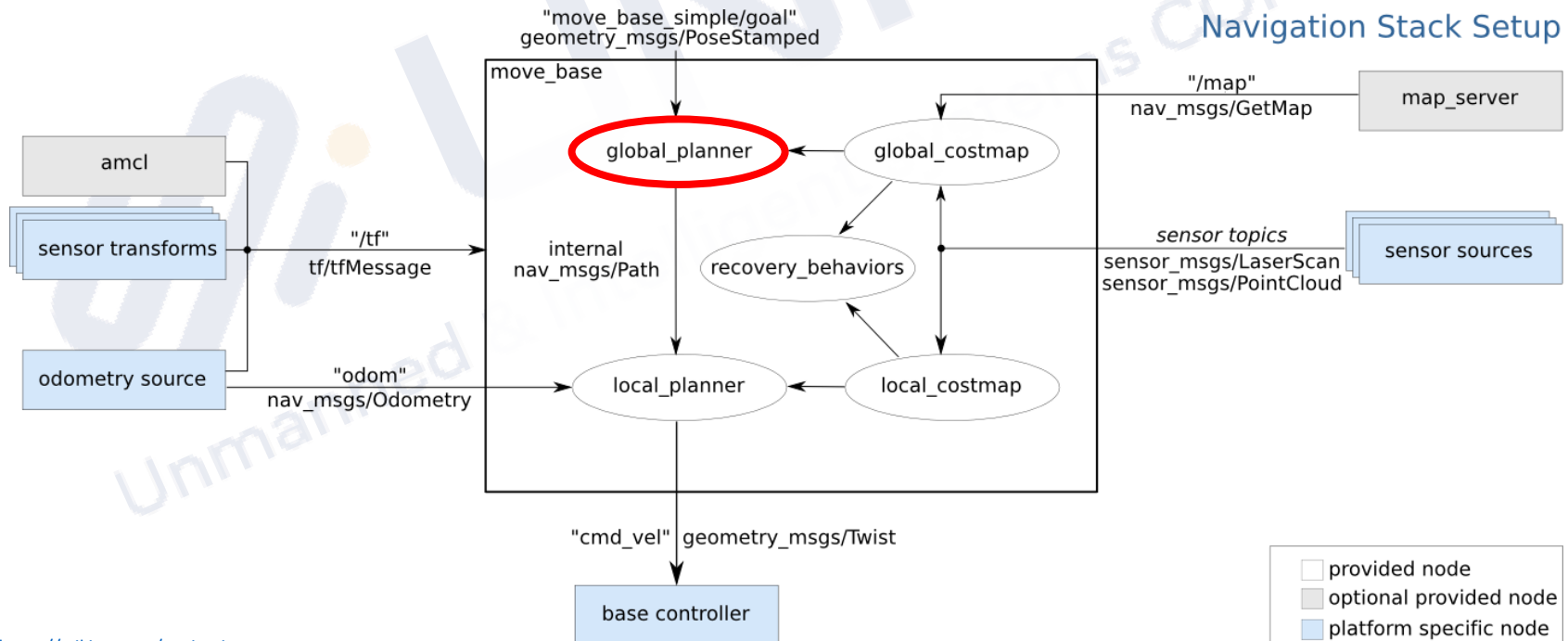


3. Navigation

2) ROS navigation stack 구조

- Global planner

- Global planner(Global path planning)는 앞서 생성된 global costmap을 바탕으로 로봇이 목표 지점까지 이동하기 위한 global path를 계산하여 도출
- 생성된 global path는 목표 위치가 변경되는 경우에만 다시 계산됨

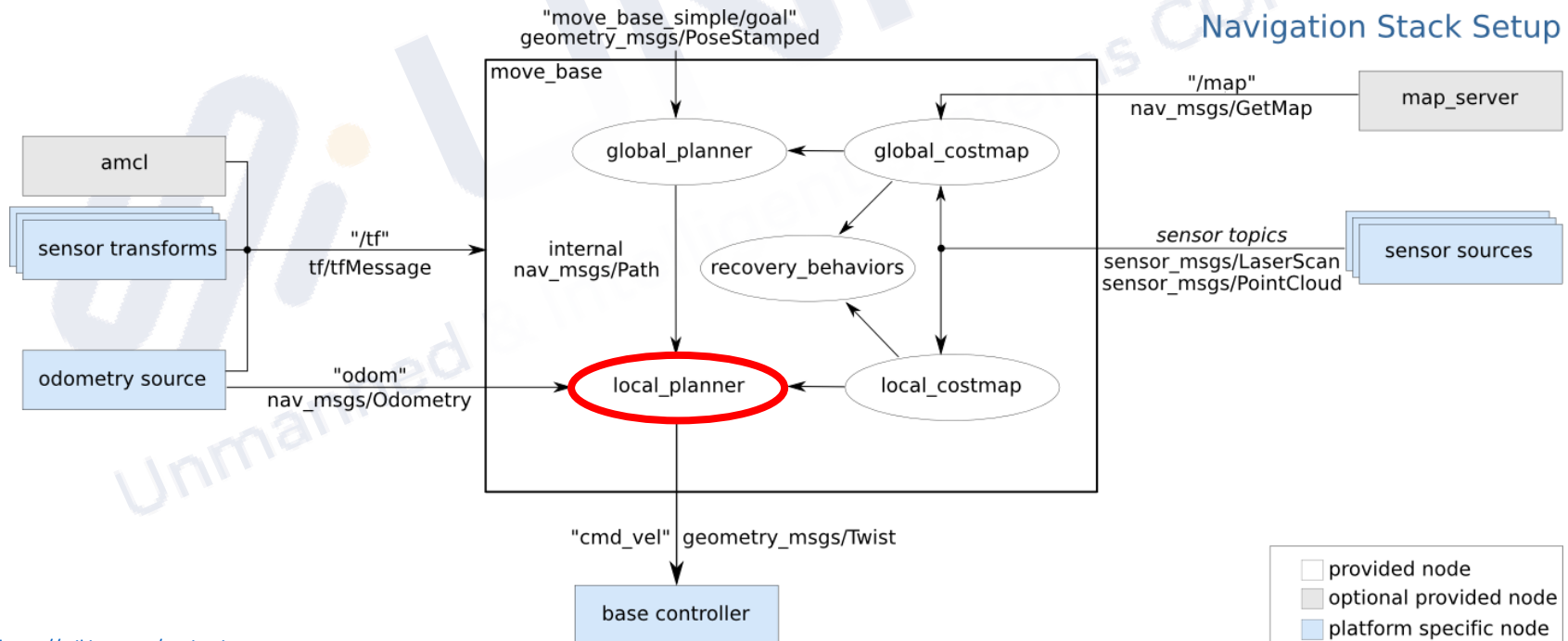


3. Navigation

2) ROS navigation stack 구조

- Local planner

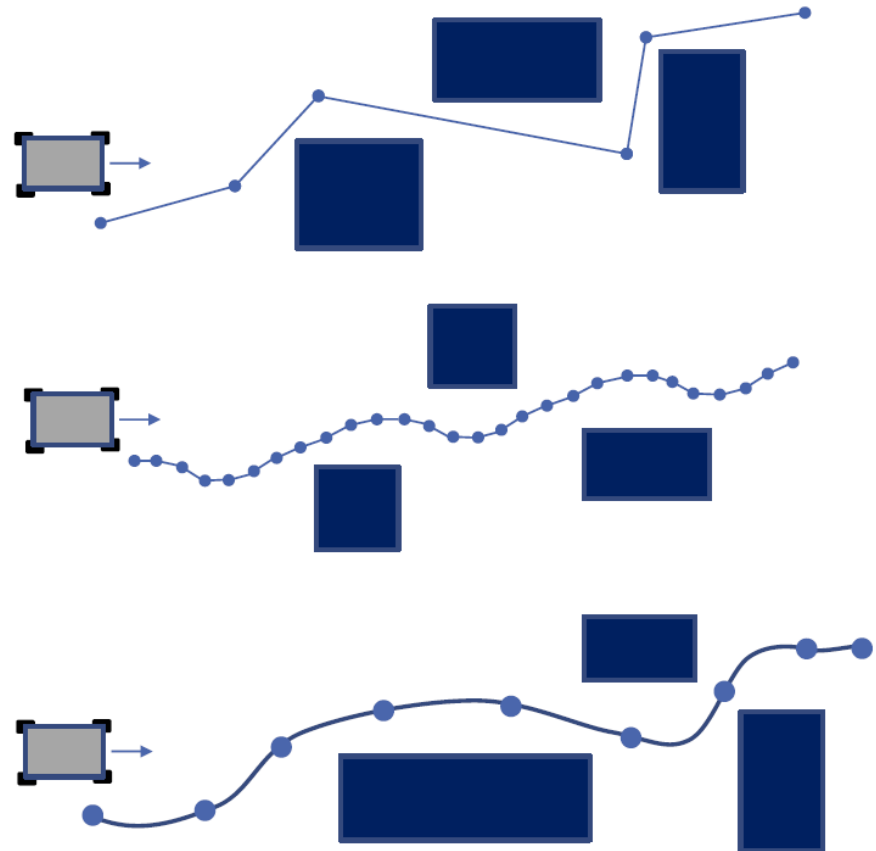
- Local planner(Local path planning)는 global planner를 통해 도출된 global path를 추종함과 동시에, 센서 데이터를 통해 생성된 local costmap을 바탕으로 정적·동적 장애물을 회피하기 위한 제어 입력을 생성



3. Navigation

2) ROS navigation stack 구조

- Path planning & Tracking control
 - Path generation
 - A*
 - D*
 - Dijkstra
 - Geometric controllers
 - Pure pursuit (carrot following)
 - Stanley
 - Dynamic controllers
 - MPC control
 - Other control systems
(Sliding mode, feedback linearization)



3. Navigation

2) ROS navigation stack 구조

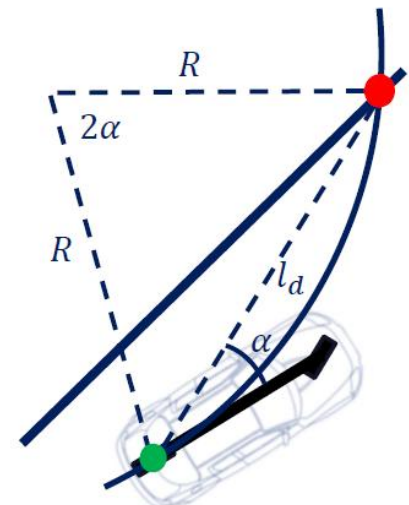
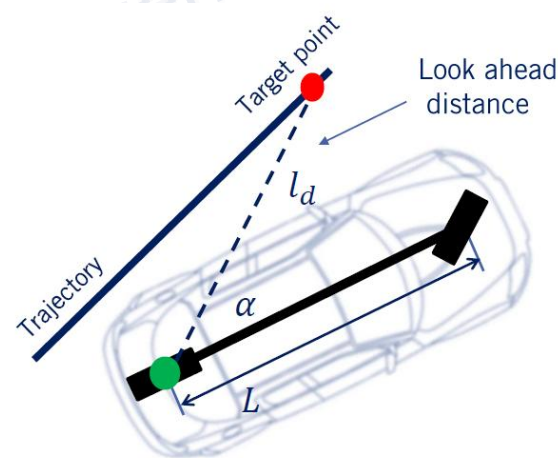
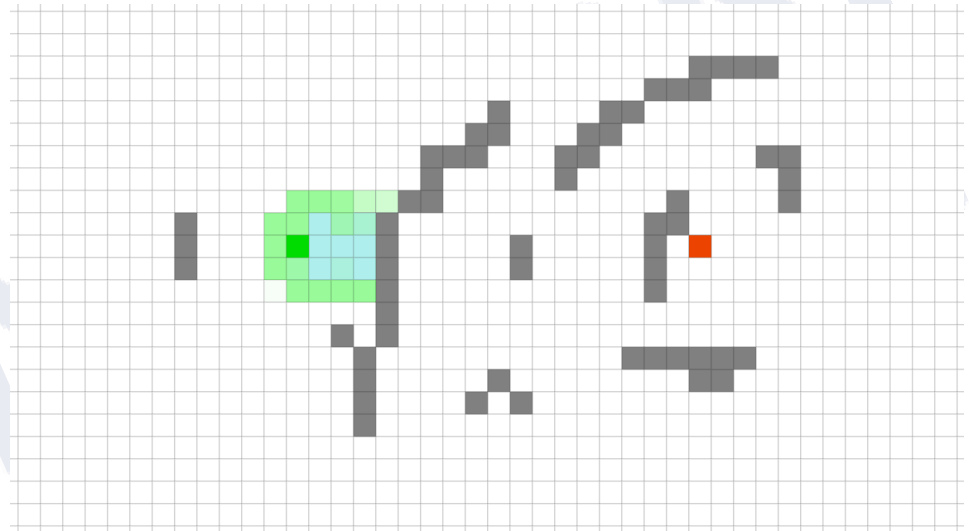
- Path planning & Tracking control

- Path generation

- A*

- Geometric controllers

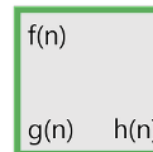
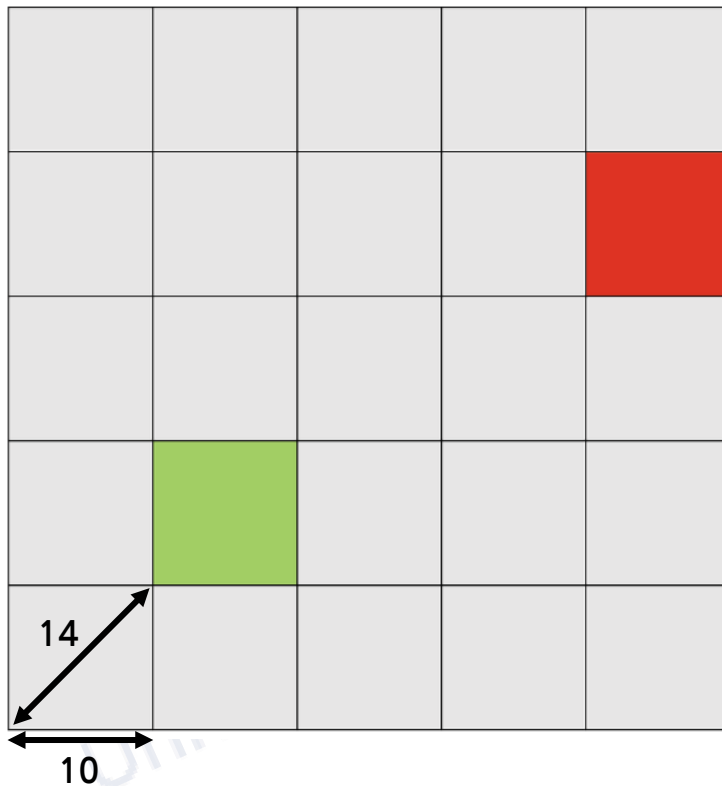
- Pure pursuit (carrot following)



3. Navigation

2) ROS navigation stack 구조

- Path planning - A*



$f(n)$: total cost
 $g(n)$: cost from start
 $h(n)$: cost to end

 Start node

 End node

- $g(n)$
- $h(n)$
- $f(n)$
- Parent node
- Open list
- Closed list

3. Navigation

2) ROS navigation stack 구조

- Path planning - A*

- $g(n)$

시작 노드로부터 현재 위치한 노드까지의 거리 값

- $h(n)$

현재 위치한 노드로부터 목표 노드까지의 거리 값

- $f(n)$

$g(n)$ 값과 $h(n)$ 값을 더한 값

- Parent node

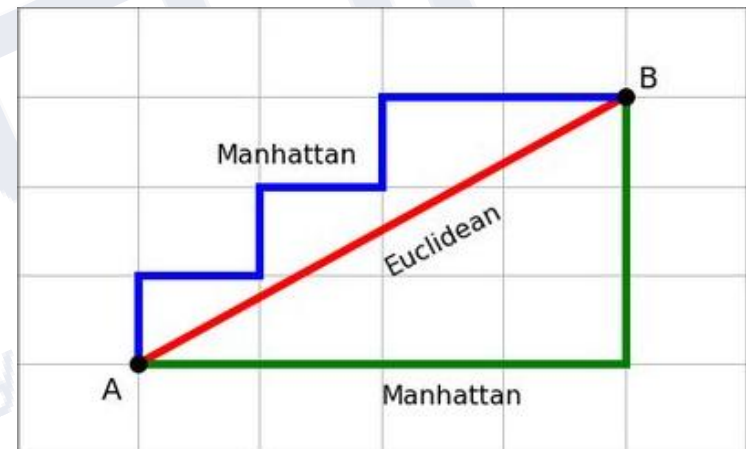
현재 노드를 탐색하기 전에 위치했던 노드

- Open list

탐색 중인(또는 탐색했던) 노드들을 저장하는 리스트

- Closed list

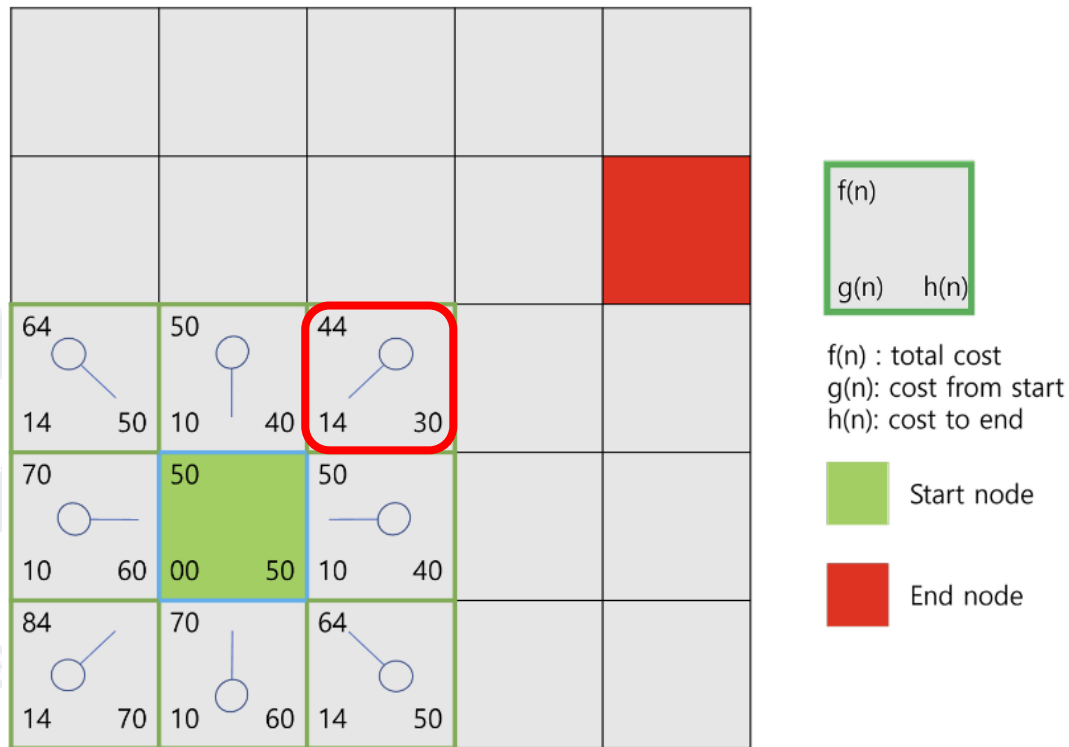
탐색한 노드 가운데, 가장 $f(n)$ 값이 작은 노드들을 저장하는 리스트



3. Navigation

2) ROS navigation stack 구조

- Path planning - A*



3. Navigation

2) ROS navigation stack 구조

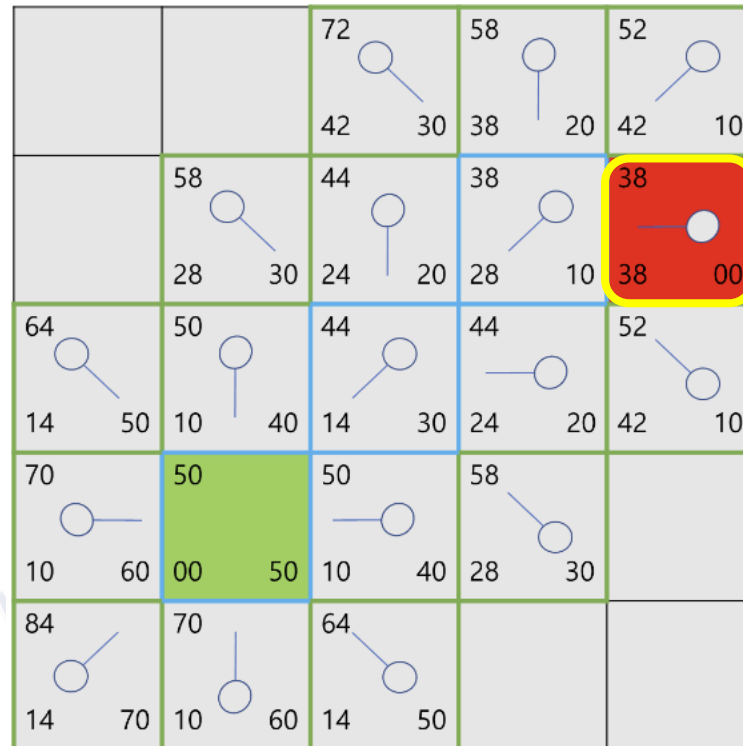
- Path planning - A*



3. Navigation

2) ROS navigation stack 구조

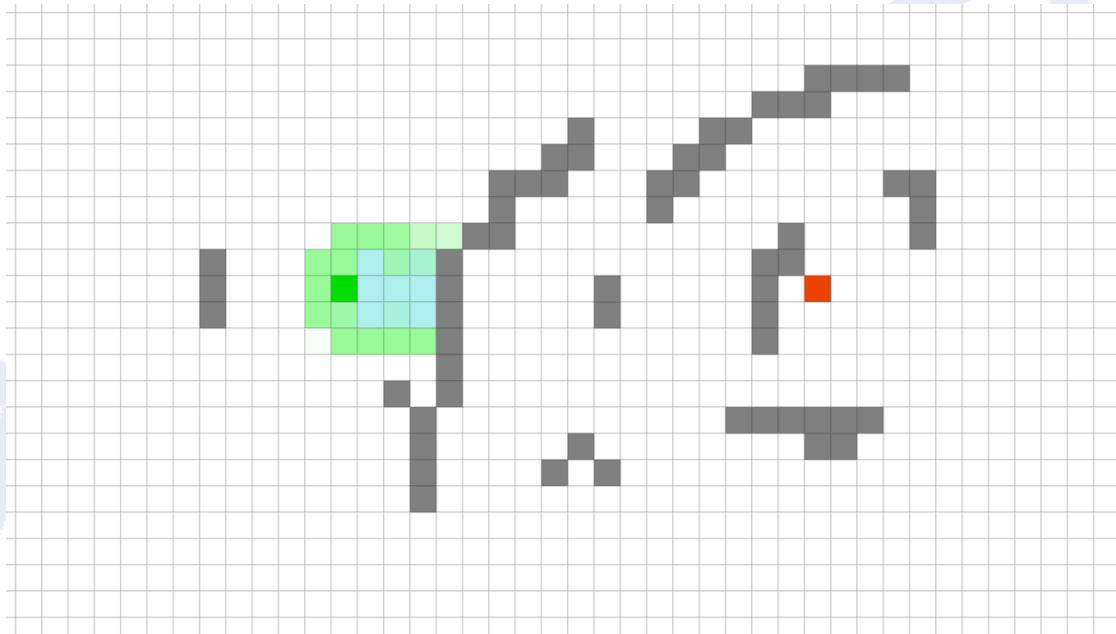
- Path planning - A*



3. Navigation

2) ROS navigation stack 구조

- Path planning - A*



- : Start position
- : Goal position
- : Static obstacle

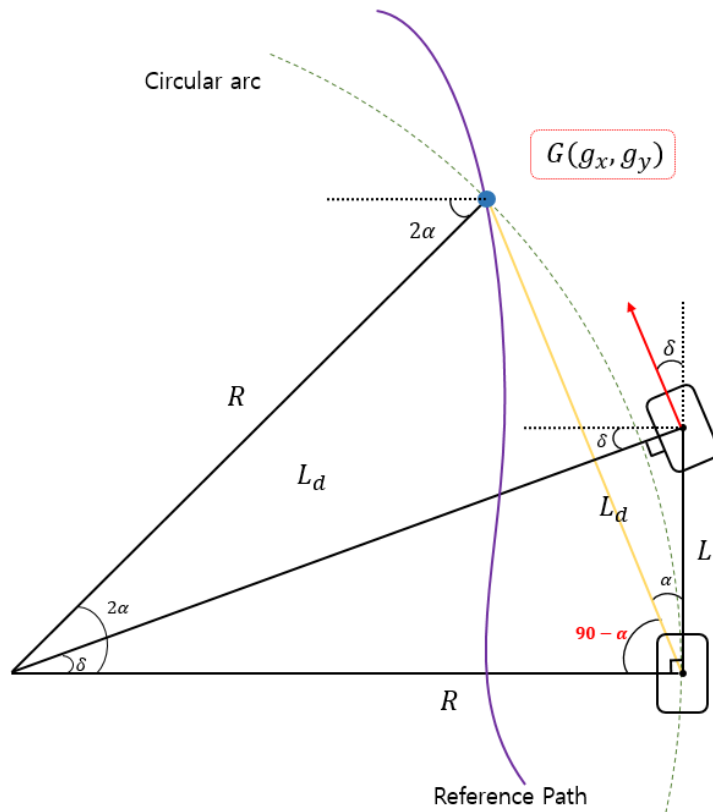
$$F(n) = G(n) + H(n)$$

$$H(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. Navigation

2) ROS navigation stack 구조

- Tracking control - Pure Pursuit



$$\frac{ld}{\sin 2\alpha} = \frac{R}{\sin(\frac{\pi}{2} - \alpha)}$$

$$\frac{ld}{2\sin\alpha\cos\alpha} = \frac{R}{\cos(\alpha)}$$

$$\frac{ld}{\sin\alpha} = 2R$$

$$k = \frac{1}{R} = \frac{2\sin\alpha}{ld}$$

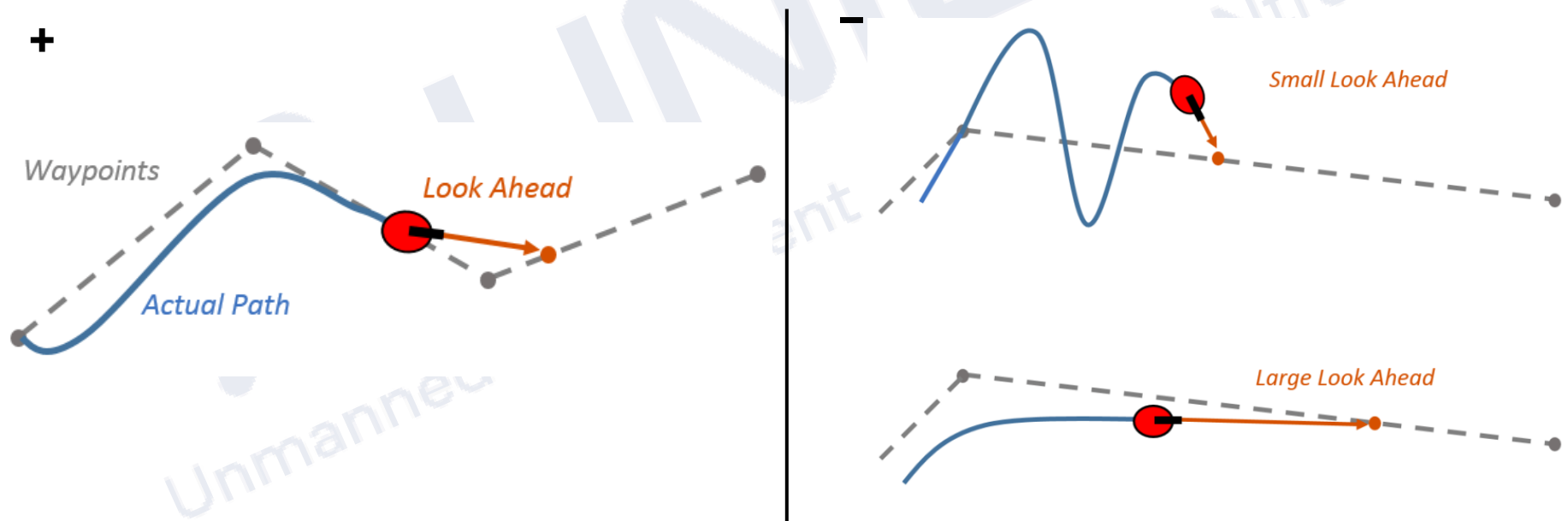
$$R = \frac{L}{\tan(\delta)}$$

$$\delta = \arctan\left(\frac{2L\sin\alpha}{ld}\right)$$

3. Navigation

2) ROS navigation stack 구조

- Tracking control - Pure Pursuit
 - Lookahead distance: 전방 주시 거리, 추종하고자 하는 목표지점까지의 거리



3. Navigation

2) ROS navigation stack 구조

- Localization - AMCL(Adaptive Monte Carlo Localization)
 - 각 파티클은 로봇의 자세를 나타내는 위치 및 방향 데이터를 저장
 - LiDAR, IMU, odometry, GPS, map 데이터를 퓨전
 - 필터링 과정을 통해 지도 상의 위치 보정

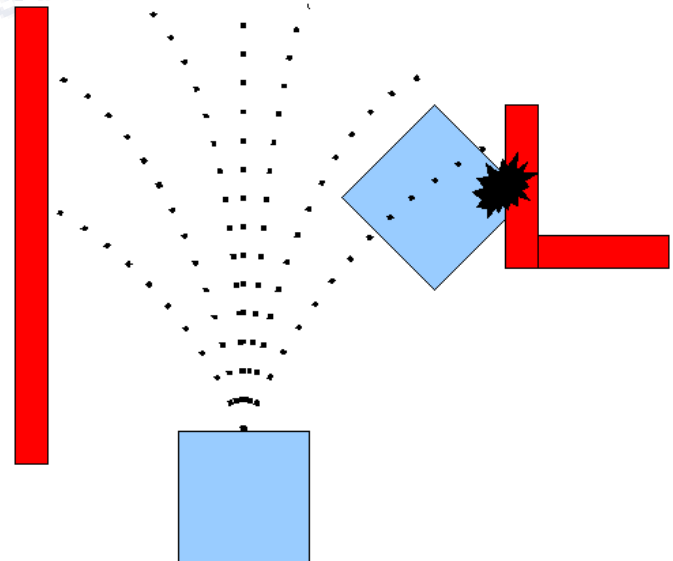
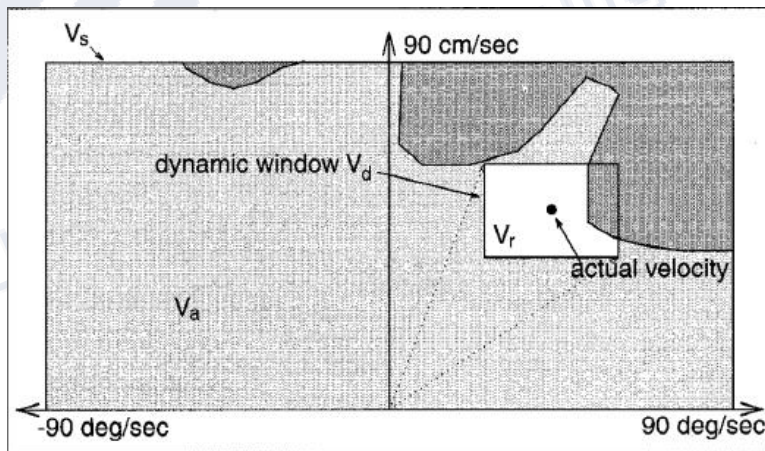


3. Navigation

2) ROS navigation stack 구조

- Obstacle avoidance - DWA(Dynamic Window Approach)

- 로봇의 상태를 고려한 최적의 원형 궤적을 나타내는 (v, ω) 를 찾는 기법
 - 목표지점까지 도달, 장애물 회피, 최적의 속도 및 가속도를 고려함
- ① 로봇의 제어 공간 $(dx, dy, d\theta)$ 에서 이산적으로 샘플링
 - ② 샘플링된 각 속도에 대해 로봇의 현재 상태에서 순방향 시뮬레이션을 수행하여 샘플링된 속도가 일정(짧은) 시간 동안 적용되면 어떤 일이 일어날지 예측
 - ③ 장애물과의 근접성, 목표와의 근접성, 광역 경로와의 근접성, 속도와 같은 특성을 통합하는 지표를 사용하여 전방 시뮬레이션에서 발생하는 각 궤적을 평가하고 이때 장애물과 충돌하는 궤적은 제외
 - ④ 가장 높은 점수의 궤적을 선택하고 속도 명령 생성
 - ⑤ 반복 수행

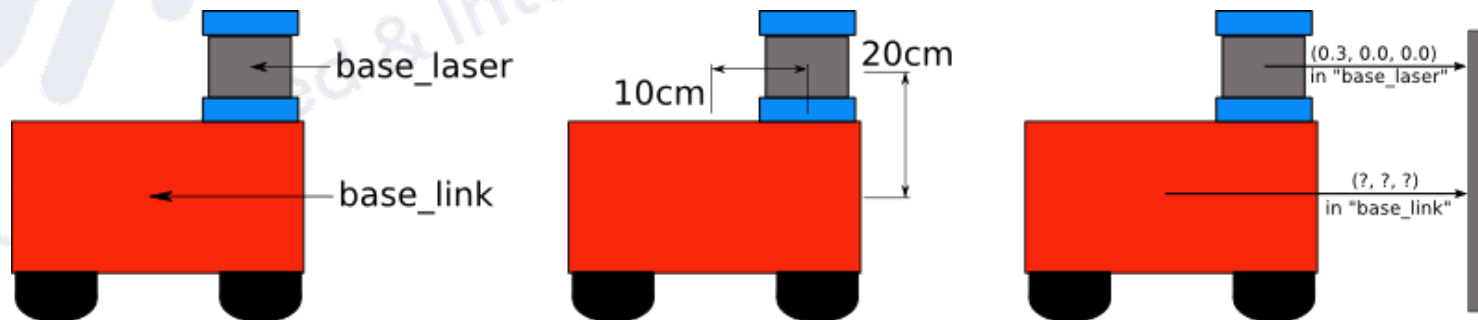


3. Navigation

3) Transform

● TF configuration

- TF 트리는 서로 다른 좌표 프레임 간의 변환 및 회전 측면에서 오프셋을 정의함.
- 예를 들어 라이다 센서가 장착된 로봇을 고려할 때, 로봇에서 두 개의 좌표 프레임을 정의할 수 있는데 하나는 로봇 베이스의 중심점에 해당하고 다른 하나는 베이스 위에 장착된 레이저의 중심점에 해당함.
- 모바일 로봇에 연결된 좌표 프레임을 "base_link", 라이다에 연결된 좌표 프레임을 "base_laser" 라고 정의하고, 라이다가 모바일 로봇의 중심점에서 10cm 앞쪽, 20cm 위에 장착되어 있다고 가정함.
- 이때 "base_link" 프레임에서 "base_laser" 프레임으로 데이터를 가져오려면 ($x: 0.1m, y: 0.0m, z: 0.2m$)의 변환을 적용해야 하고, "base_laser" 프레임에서 "base_link" 프레임으로 데이터를 가져오려면 반대 변환($x: -0.1m, y: 0.0m, z: -0.2m$)을 적용해야 함.

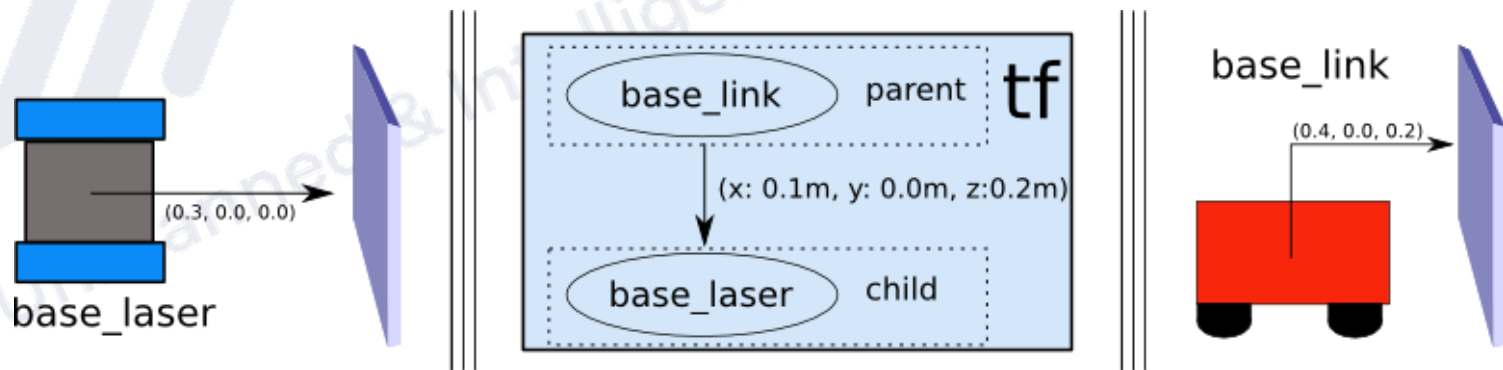


3. Navigation

3) Transform

- TF configuration

- 센서의 수가 증가하고, 좌표 프레임의 수가 증가함에 따라 각각의 좌표 변환 관계를 다 정의하기가 어려움.
- TF 라이브러리를 사용하여 "base_link" 와 "base_laser" 간의 관계를 정의하고 두 좌표 프레임 간의 변환을 관리하게 할 수 있음.
- TF는 트리 구조를 사용하여 두 좌표 프레임을 연결하고 모든 변환이 부모에서 자식으로 이동한다고 가정함. 여기서는 "base_laser" 프레임에서 수신한 라이다 데이터를 "base_link" 프레임 기준으로 변환.
- 로봇은 이 정보를 사용하여 "base_link" 프레임의 라이다 데이터에 대해 추론하고 주변 장애물을 안전하게 피할 수 있음.



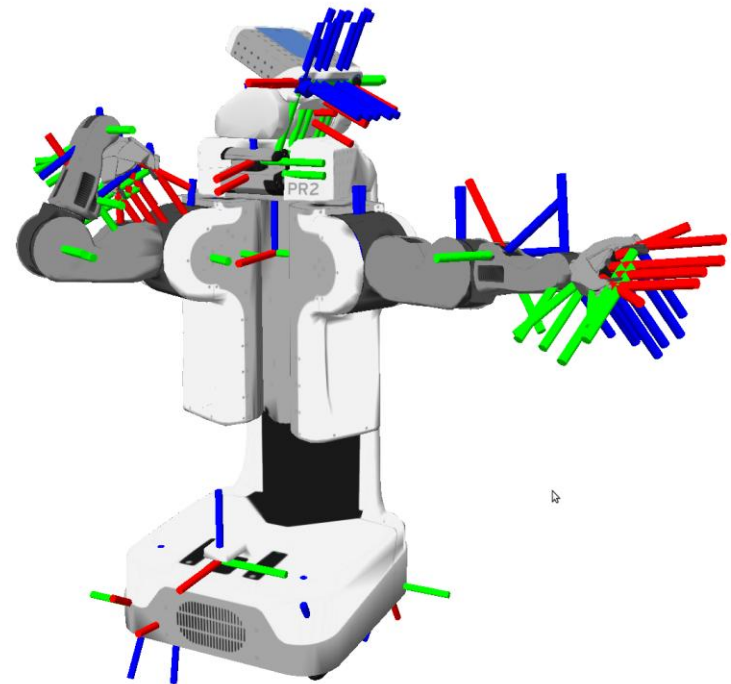
3. Navigation

3) Transform

- TF2

- TF2 라이브러리는 TF 변환 라이브러리 2세대를 의미함.
- 시간 경과에 따른 여러 좌표 프레임을 사용자가 확인할 수 있게 해주는 변환 툴.
- 라이브러리를 활용하면 아래 질문에 대한 답을 얻을 수 있음.

- Where was the head frame relative to the world frame, 5 seconds ago?
- What is the pose of the object in my gripper relative to my base?
- What is the current pose of the base frame in the map frame?



3. Navigation

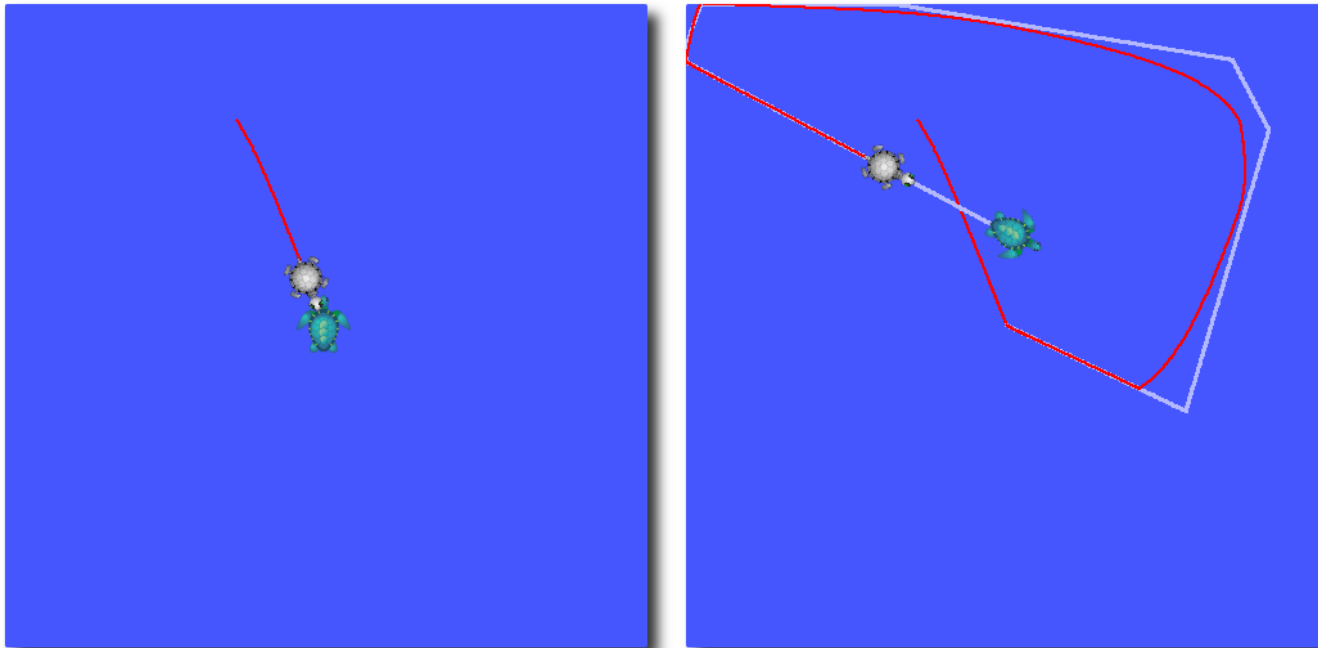
3) Transform

- TF2 demo
 - Dependencies

```
$ sudo apt install ros-noetic-turtle-tf2 ros-noetic-tf2-tools ros-noetic-tf
```

- Running the demo

```
$ roslaunch turtle_tf2 turtle_tf2_demo_cpp.launch
```

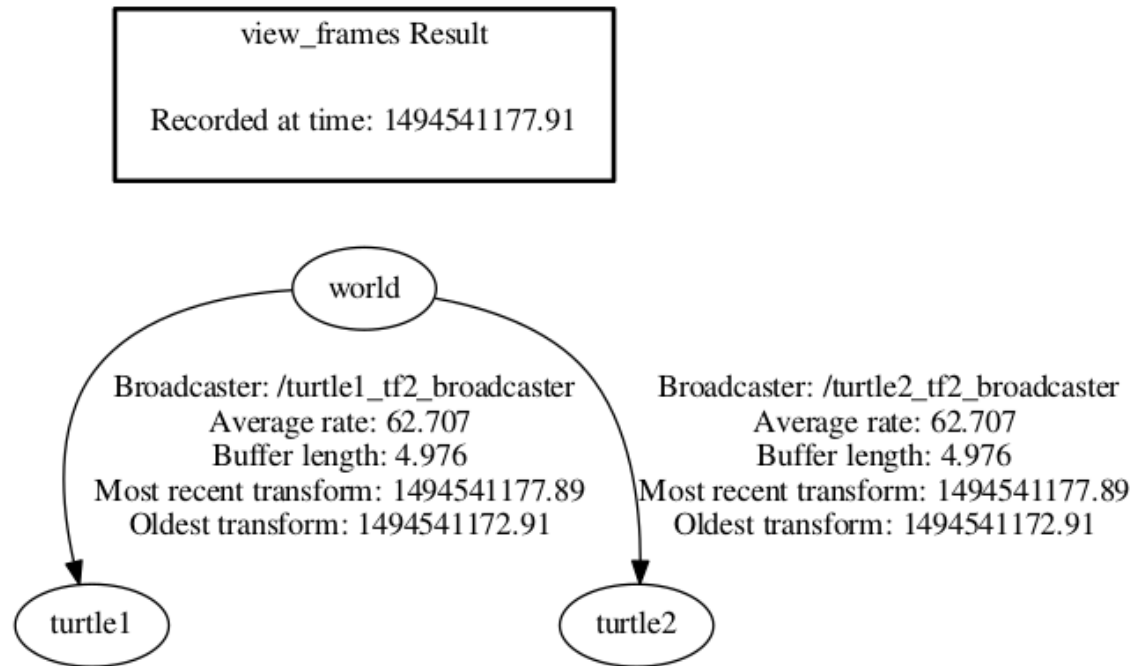


3. Navigation

3) Transform

- TF2 demo
 - Using view_frames

```
$ rosrn tf2_tools view_frames.py  
$ evince frames.pdf
```



3. Navigation

3) Transform

- TF2 demo
 - Using tf_echo

```
roslaunch tf2_demo tf_echo [reference_frame] [target_frame]
$ roslaunch tf2_demo tf_echo turtle1 turtle2
```

```
[ INFO] 1253585683.529245000: Started node [/tf2_echo_1253585683508144000], pid [24418], bo
und on [aqp], xmlrpc port [41125], tcpport [57369], logging to [~/ros/roscat/log/tf2_echo_
1253585683508144000_24418.log], using [real] time
Exception thrown:Frame id /turtle1 does not exist! When trying to transform between /turtle
2 and /turtle1.
The current list of frames is:

Success at 1253585684.557003974
[0.000000 0.000000 0.140754 0.990045] Euler(0.282446 -0.000000 0.000000)
Translation: [-0.000036 -0.000010 0.000000]
Success at 1253585685.544698953
[0.000000 0.000000 0.140754 0.990045] Euler(0.282446 -0.000000 0.000000)
Translation: [-0.000036 -0.000010 0.000000]
Success at 1253585686.557049989
[0.000000 0.000000 0.140754 0.990045] Euler(0.282446 -0.000000 0.000000)
Translation: [-0.000036 -0.000010 0.000000]
Success at 1253585687.552628993
[0.000000 0.000000 0.140754 0.990045] Euler(0.282446 -0.000000 0.000000)
Translation: [-0.000036 -0.000010 0.000000]
Success at 1253585688.553683042
[0.000000 0.000000 0.140754 0.990045] Euler(0.282446 -0.000000 0.000000)
Translation: [-0.000036 -0.000010 0.000000]
Success at 1253585688.910640001
[0.000000 0.000000 0.140754 0.990045] Euler(0.282446 -0.000000 0.000000)
Translation: [-0.000036 -0.000010 0.000000]
```

3. Navigation

3) Transform

- TF2 demo

- rviz and tf2

```
$ rviz
```

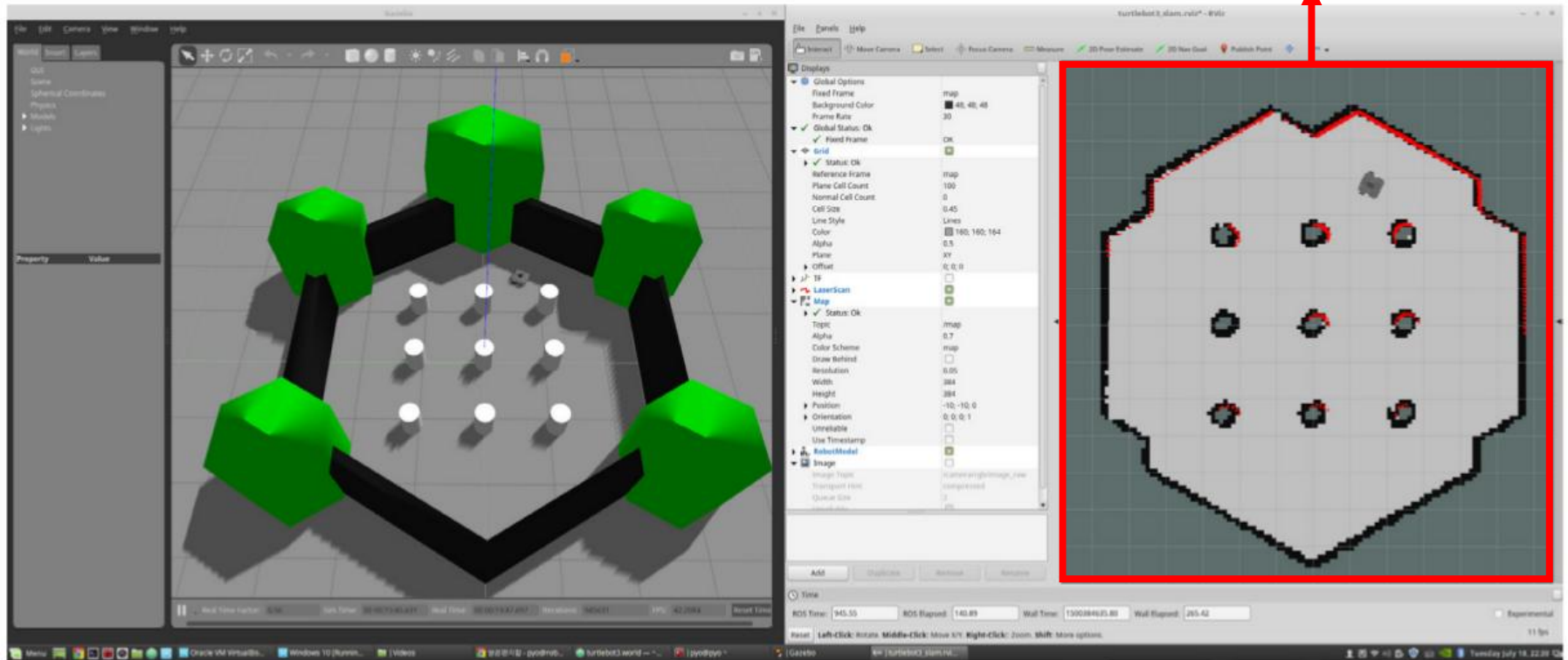
- **Fixed Frame**을 'world' 로 변경
- 'add' 버튼을 눌러 TF 추가

3. Navigation

4) Turtlebot3 navigation 실습

- SLAM map

“Occupancy Grid Map”



3. Navigation

4) Turtlebot3 navigation 실습

- SLAM map

- Occupancy grid map은 이산적인 셀로 구성된 지도로 각 셀은 특정 공간의 점유 상태를 나타냄
- 각 셀은 비어 있을 확률과 점유되어 있을 확률을 가지고 있으며, 이는 센서의 불확실성과 노이즈를 반영한 것
- SLAM 과정에서 지도는 지속적으로 업데이트되며, 새로운 센서 데이터가 수집될 때마다 수정 및 개선됨
- Occupancy grid map 을 구성하는 하나의 셀은 0부터 100까지의 정수 값으로 이루어짐.
- 0은 해당 셀이 완전한 **free space**임을 나타내며, 해당 셀 영역에 **장애물이 없다는 것**을 의미.
100은 해당 셀이 완전히 **점유되어 있음**을 나타내며, 해당 셀 영역에 **장애물이 있다는 것**을 의미.
-1 또는 **unknown**은 해당 셀의 **상태가 알려지지 않았음**을 나타내며, 로봇은 해당 셀 영역의 정보를 취득하지 못했거나 셀에 대한 확신이 없는 경우 해당 값을 사용함.

```
$ rostopic echo /map
```

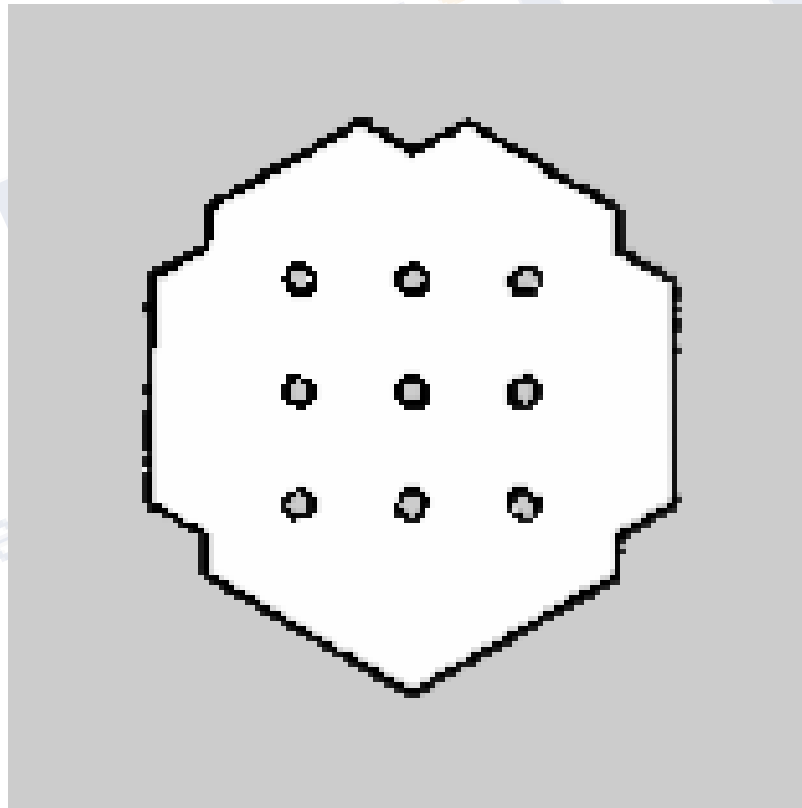
3. Navigation

4) Turtlebot3 navigation 실습

- SLAM map

```
$ rosrun map_server map_saver -f [file_name]
```

```
$ rosrun map_server map_saver -f ~/map
```



3. Navigation

4) Turtlebot3 navigation 실습

- Navigation map 설정

- map 폴더로 map.pgm 과 map.yaml 파일을 복사

```
$ cp ~/map.* ~/catkin_ws/src/turtlebot3/turtlebot3_navigation/maps
```

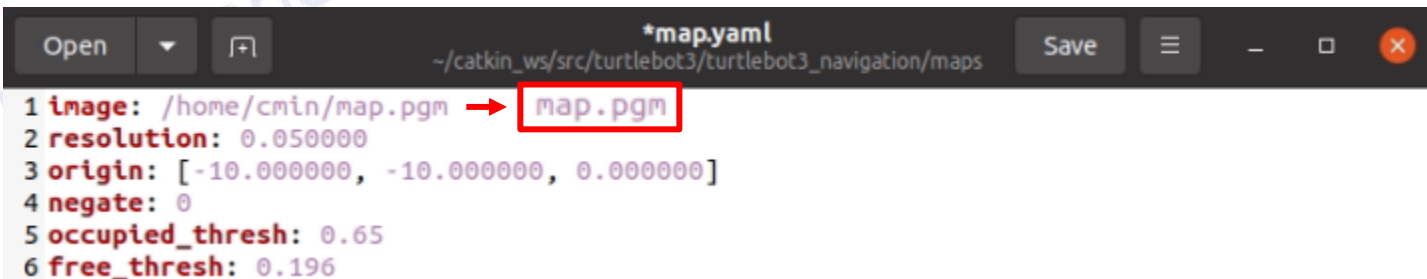
- map.pgm 과 map.yaml 파일이 복사된 것을 확인

```
$ roscd turtlebot3_navigation  
$ cd maps  
$ ls
```

```
cmin@cmin:~/catkin_ws/src/turtlebot3/turtlebot3_navigation/maps$ ls  
map.pgm  map.yaml
```

- map.yaml 수정

```
$ gedit map.yaml
```



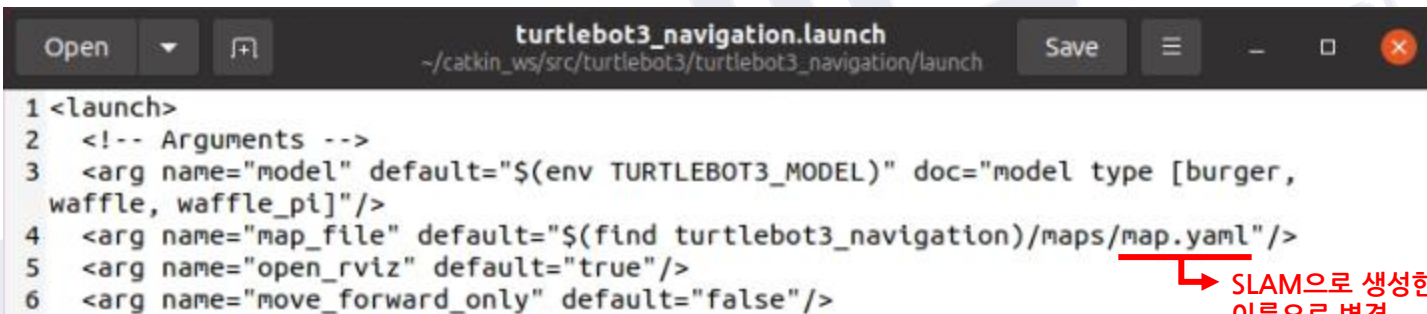
3. Navigation

4) Turtlebot3 navigation 실습

- Navigation map 설정

- turtlebot3_navigation.launch 수정

```
$ roscd turtlebot3_navigation
$ cd launch
$ gedit turtlebot3_navigation.launch
```



```
turtlebot3_navigation.launch
~/catkin_ws/src/turtlebot3/turtlebot3_navigation/launch

1 <launch>
2   <!-- Arguments -->
3   <arg name="model" default="$(env TURTLEBOT3_MODEL)" doc="model type [burger,
   waffle, waffle_pi]"/>
4   <arg name="map_file" default="$(find turtlebot3_navigation)/maps/map.yaml"/>
5   <arg name="open_rviz" default="true"/>
6   <arg name="move_forward_only" default="false"/>
```

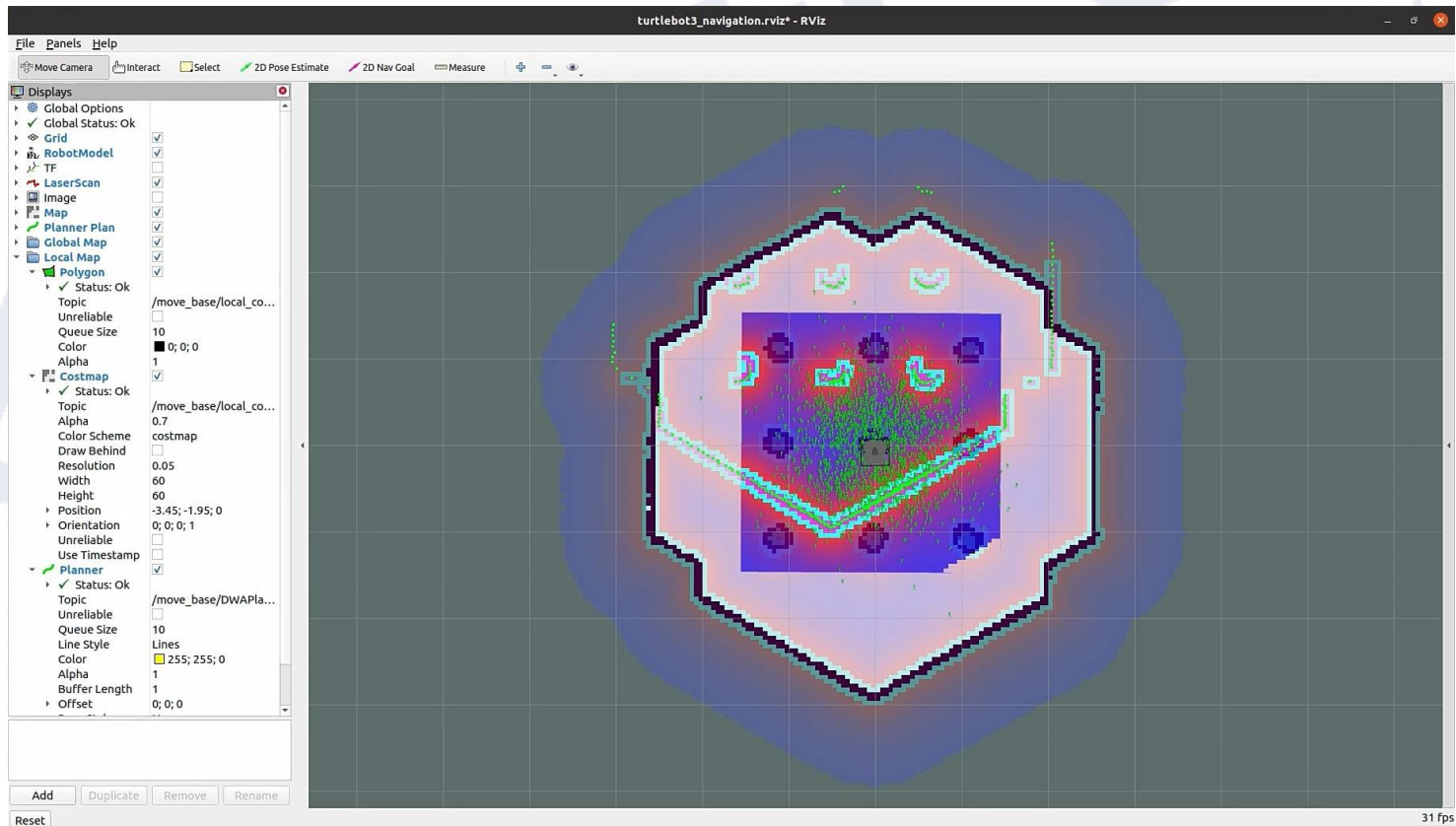
SLAM으로 생성한 맵 파일
이름으로 변경

3. Navigation

4) Turtlebot3 navigation 실습

- Turtlebot3 navigation

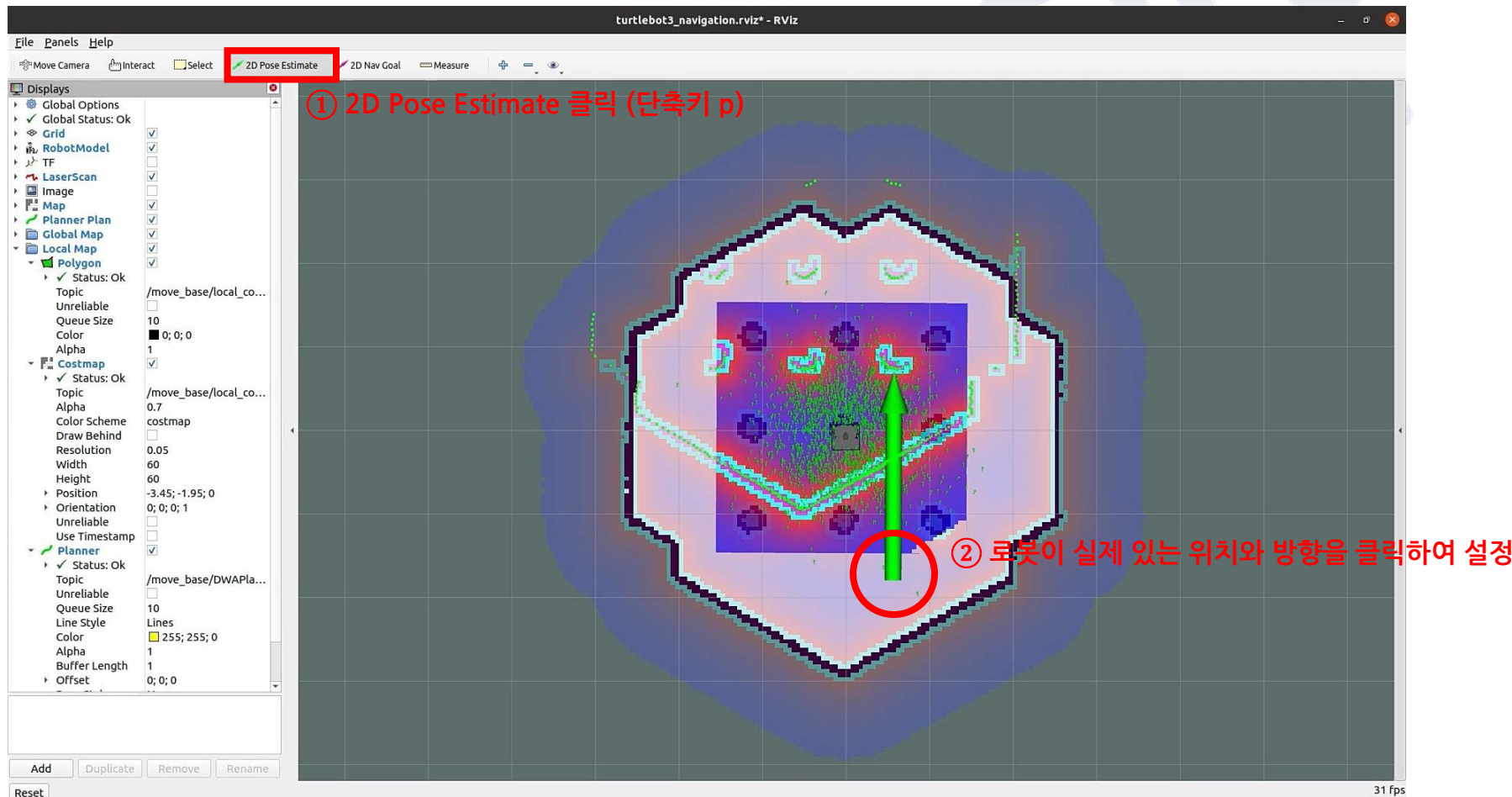
```
$ roscore
$ roslaunch turtlebot3_gazebo turtlebot3_world.launch
$ roslaunch turtlebot3_navigation turtlebot3_navigation.launch
```



3. Navigation

4) Turtlebot3 navigation 실습

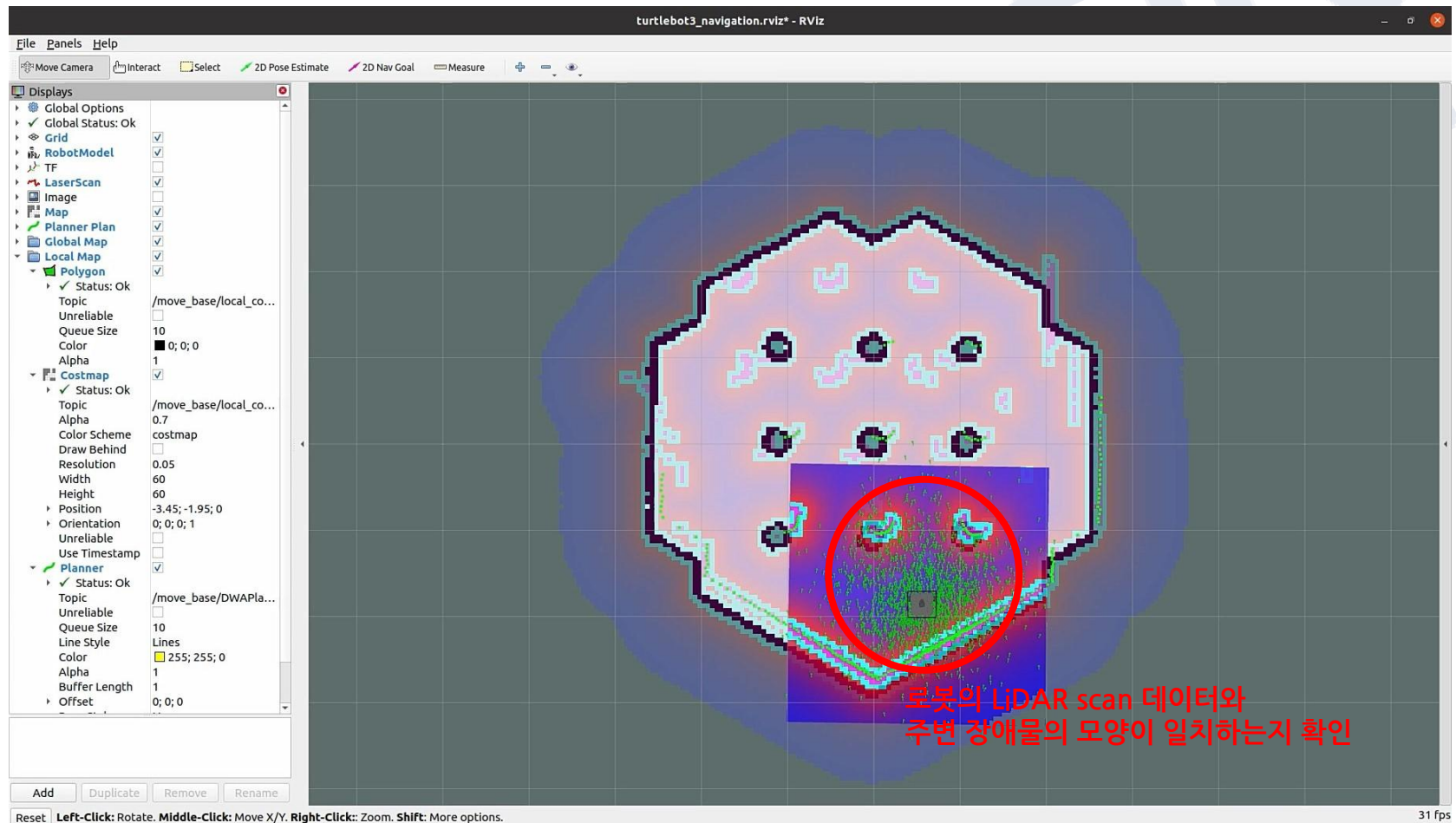
- Turtlebot3 navigation



3. Navigation

4) Turtlebot3 navigation 실습

- Turtlebot3 navigation

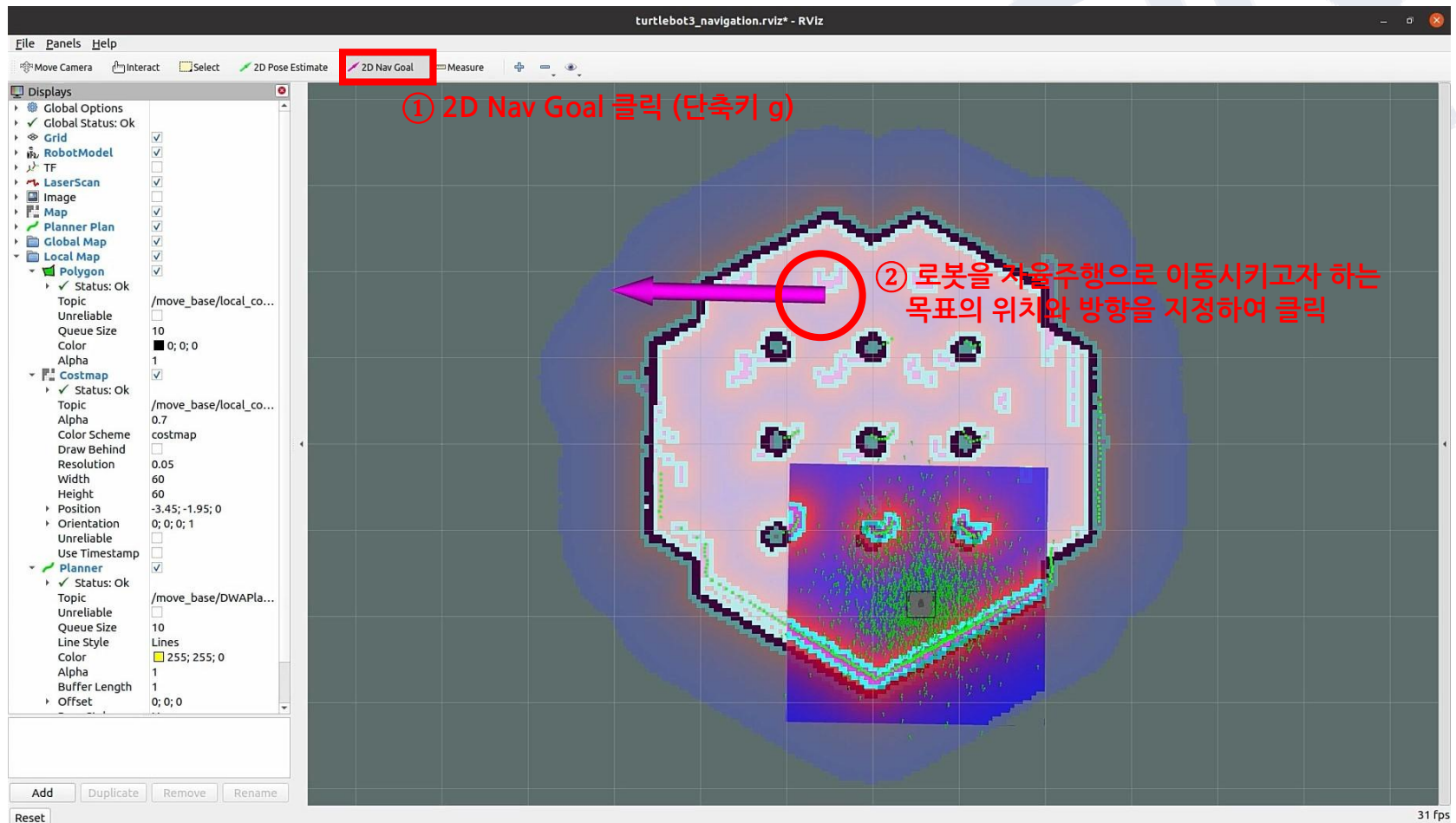


로봇의 LIDAR scan 데이터와
주변 장애물의 모양이 일치하는지 확인

3. Navigation

4) Turtlebot3 navigation 실습

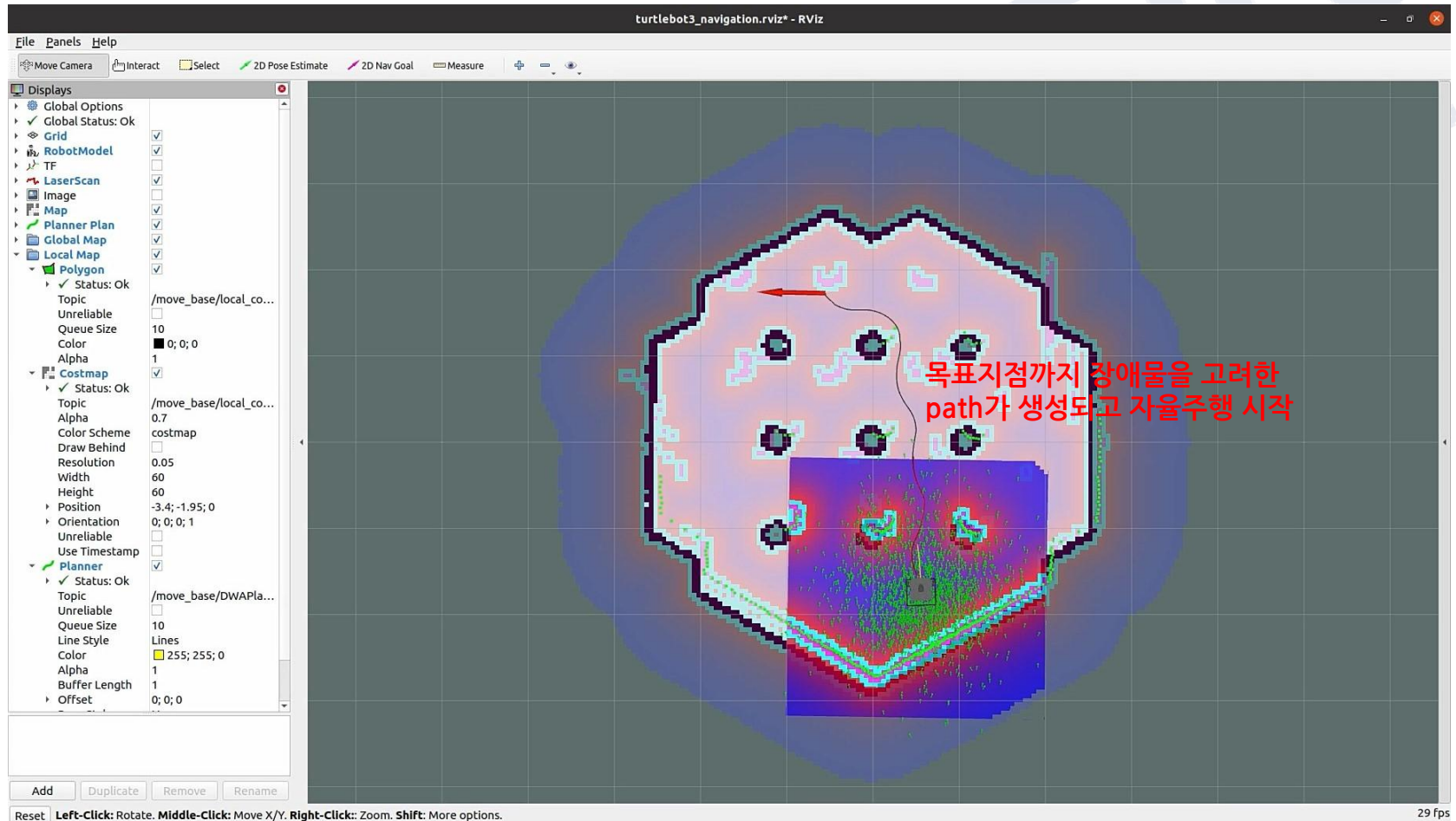
- Turtlebot3 navigation



3. Navigation

4) Turtlebot3 navigation 실습

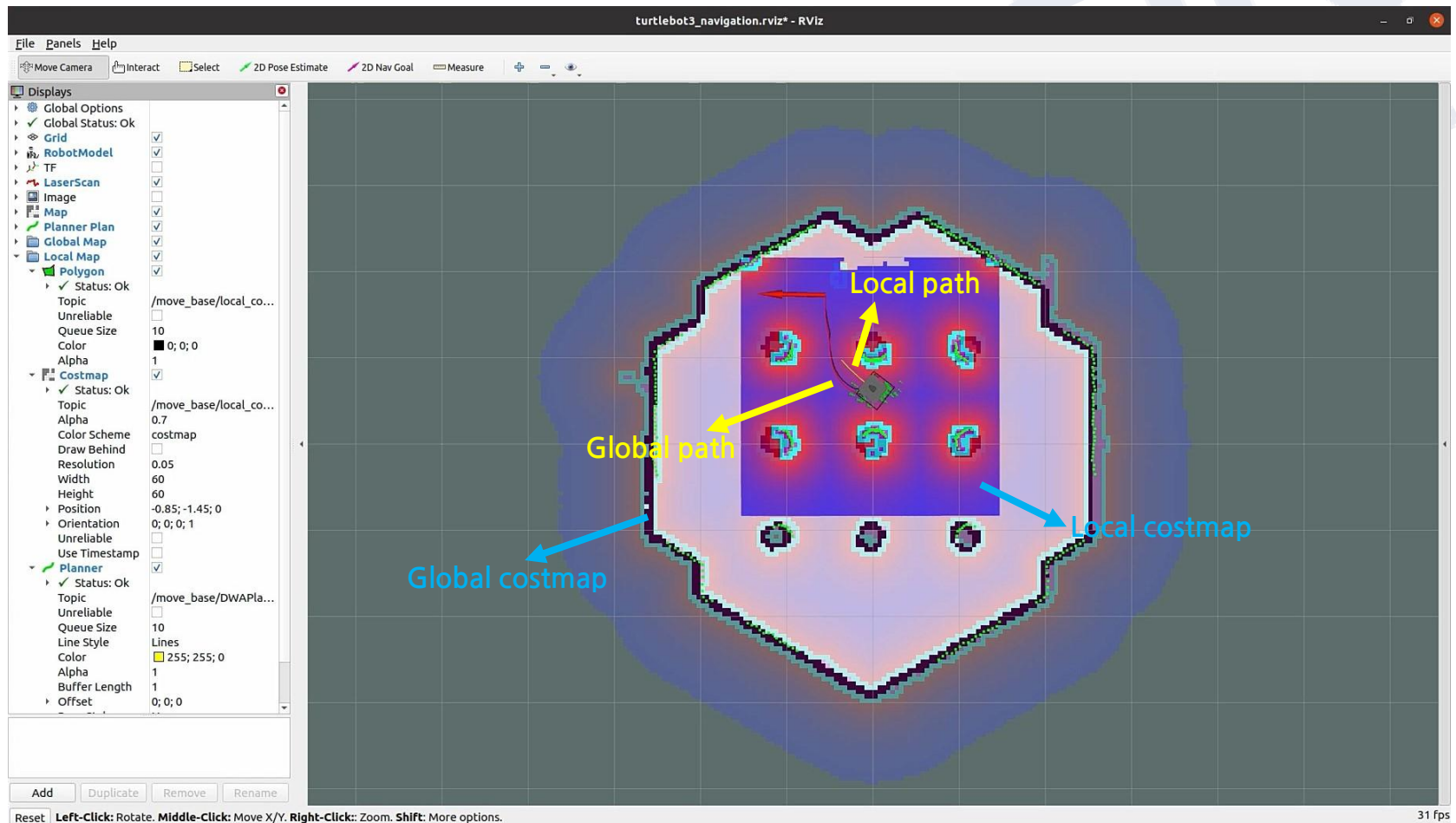
- Turtlebot3 navigation



3. Navigation

4) Turtlebot3 navigation 실습

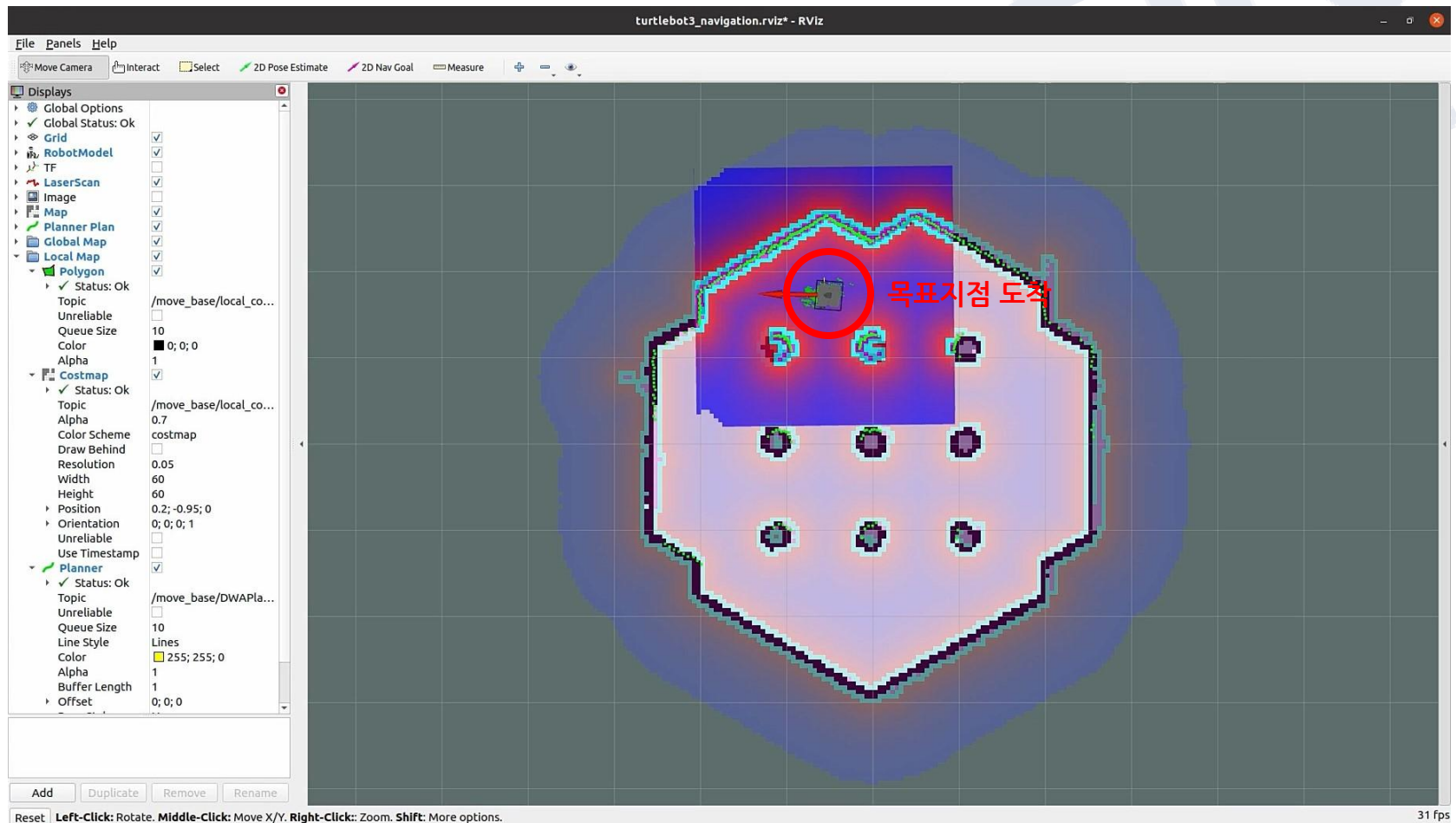
- Turtlebot3 navigation



3. Navigation

4) Turtlebot3 navigation 실습

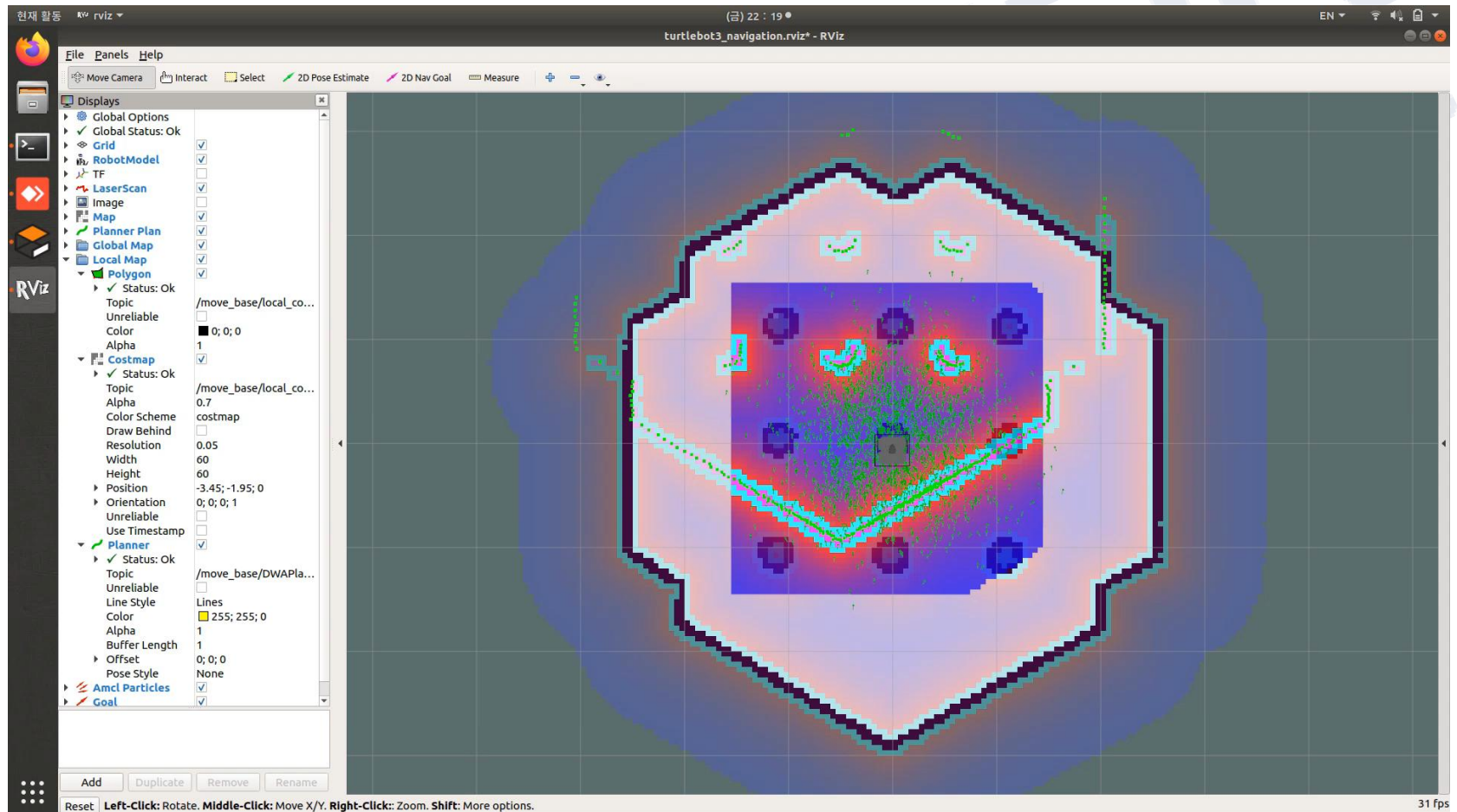
- Turtlebot3 navigation



3. Navigation

4) Turtlebot3 navigation 실습

- Turtlebot3 navigation



3. Navigation

4) Turtlebot3 navigation 실습

- Turtlebot3 navigation

```
$ roscd turtlebot3_navigation/param
$ gedit dwa_local_planner_params_{TURTLEBOT3_MODEL}.yaml
```

- TurtleBot의 선속도, 회전속도 등을 조절해서 동작 확인
- goal_tolerance를 조절하여 도착지점 도달 여부 확인

```
1  DWAPlanerROS:
2
3  # Robot Configuration Parameters
4  max_vel_x: 0.26
5  min_vel_x: -0.26
6
7  max_vel_y: 0.0
8  min_vel_y: 0.0
9
10 # The velocity when robot is moving in a straight line
11 max_vel_trans: 0.26
12 min_vel_trans: 0.13
13
14 max_vel_theta: 1.82
15 min_vel_theta: 0.9
```

```
16
17 acc_lim_x: 2.5
18 acc_lim_y: 0.0
19 acc_lim_theta: 3.2
20
21 # Goal Tolerance Parameters
22 xy_goal_tolerance: 0.05
23 yaw_goal_tolerance: 0.17
24 latch_xy_goal_tolerance: false
25
26 # Forward Simulation Parameters
27 sim_time: 2.0
28 vx_samples: 20
29 vy_samples: 0
30 vth_samples: 40
31 controller_frequency: 10.0
```

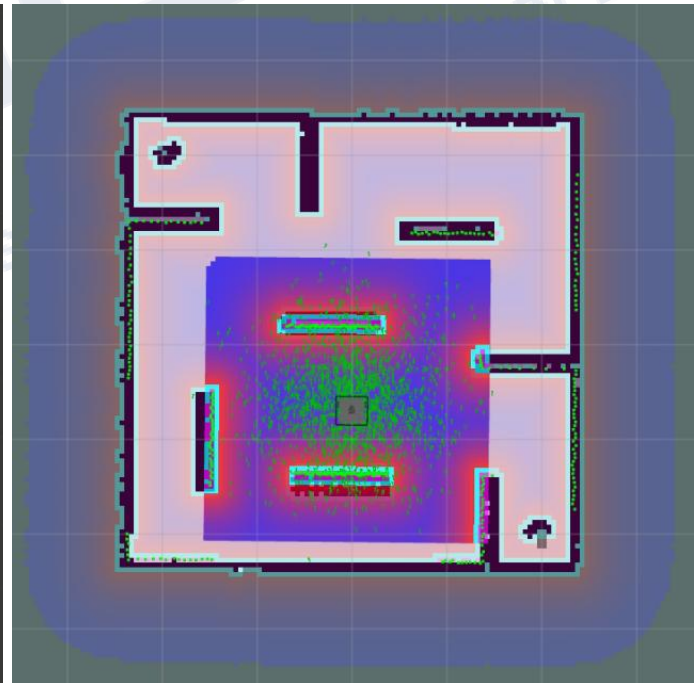
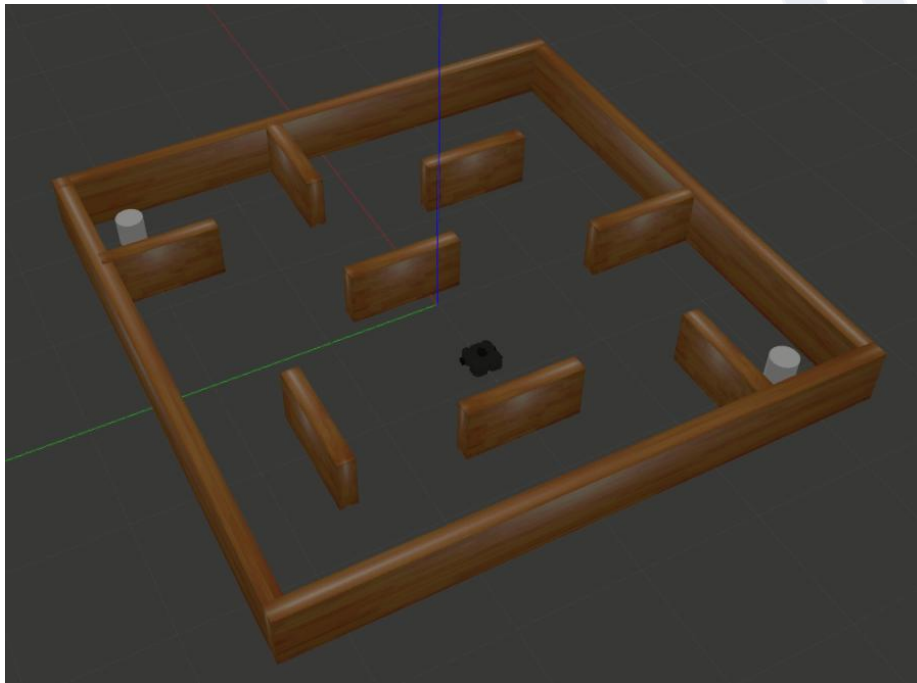
3. Navigation

4) Turtlebot3 navigation 실습

- 연습 문제

- TurtleBot3 시뮬레이션 환경 중 stage4에 대한 자율주행 테스트

```
$ roslaunch turtlebot3_gazebo turtlebot3_stage_4.launch
```

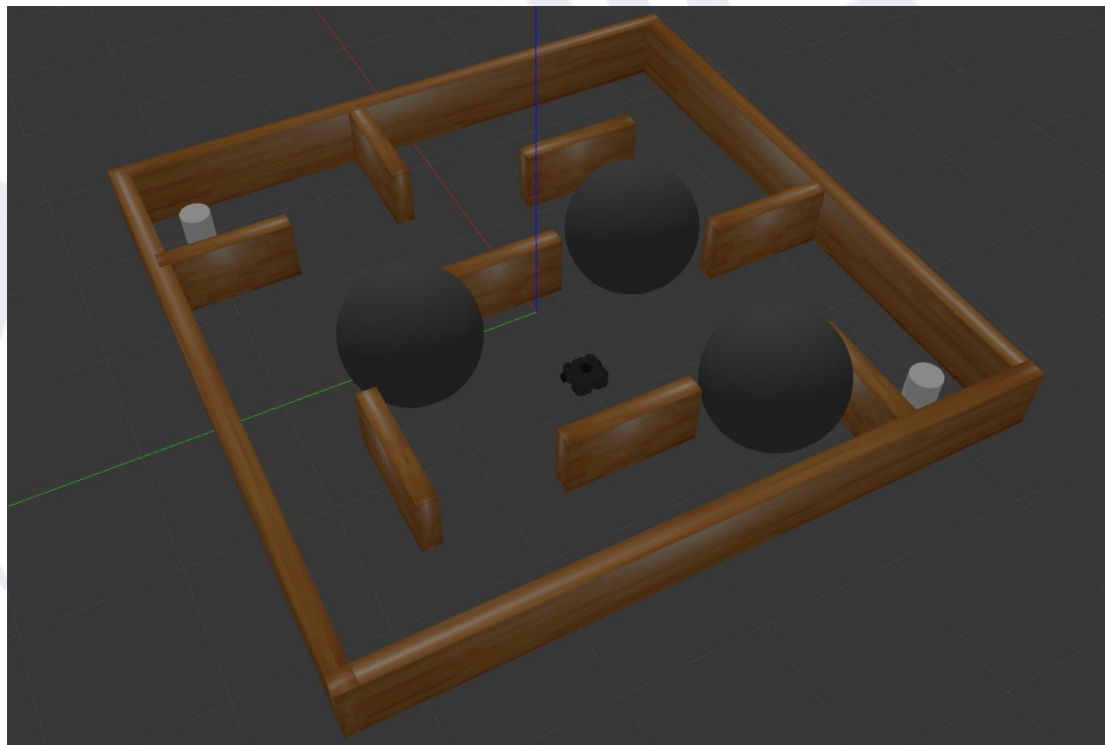


3. Navigation

4) Turtlebot3 navigation 실습

- 연습 문제

- TurtleBot3 자율주행 시뮬레이션 중 gazebo에서 구형 장애물 생성
- 광역/지역 경로의 변화 확인
- 회피 동작 확인



Advanced ROS Techniques

Unmanned & Intelligent Systems Control LAB

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP란?



- GIMP(GNU Image Manipulation Program)는 GNU/Linux, macOS, Windows 등 여러 운영체제에서 사용할 수 있는 크로스 플랫폼 이미지 편집기.
- GIMP 편집기를 통해 이미지를 원하는 대로 편집 가능.
- 앞서 SLAM을 통해 도출한 occupancy grid map을 GIMP 편집기를 사용하여 보정, 병합 등 다양하게 편집 가능

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기를 통한 보정 시 이점

- SLAM을 수행하였던 기존 환경이 변하는 경우, 재 mapping을 할 필요 없이 간단한 편집 과정을 통해 occupancy grid map을 수정 가능.
- SLAM을 수행하여 도출한 occupancy grid map 중, 일부 지역으로 mobility가 이동하지 못하도록 편집함으로써 위험을 사전에 방지 가능.
- SLAM을 통해 도출한 occupancy grid map이 완전히 mapping 되지 않아도, 보정 작업을 통해 어느 정도 완전한 map이 될 수 있도록 편집 가능.

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 설치

```
$ sudo apt-get update
```

```
$ sudo add-apt-repository ppa:otto-kesselgulasch/gimp-edge
```

```
$ sudo apt-get update
```

```
$ sudo apt install gimp
```

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 설치

❖ 만약 설치 과정에서 아래와 같은 오류 메시지가 나온다면,

```
unicon3@unicon3:~$ sudo apt install gimp
Reading package lists... Done
Building dependency tree
Reading state information... Done
Some packages could not be installed. This may mean that you have
requested an impossible situation or if you are using the unstable
distribution that some required packages have not yet been created
or been moved out of Incoming.
The following information may help to resolve the situation:

The following packages have unmet dependencies:
 gimp : Depends: libgimp2.0 (>= 2.10.18) but it is not going to be installed
        Depends: libgimp2.0 (<= 2.10.18-2) but it is not going to be installed
        Depends: libgegl-0.4-0 (>= 0.4.22) but it is not going to be installed
E: Unable to correct problems, you have held broken packages.
```

\$ **sudo apt install ppa-purge && sudo ppa-purge ppa:otto-kesselgulasch/gimp (수행 후)**

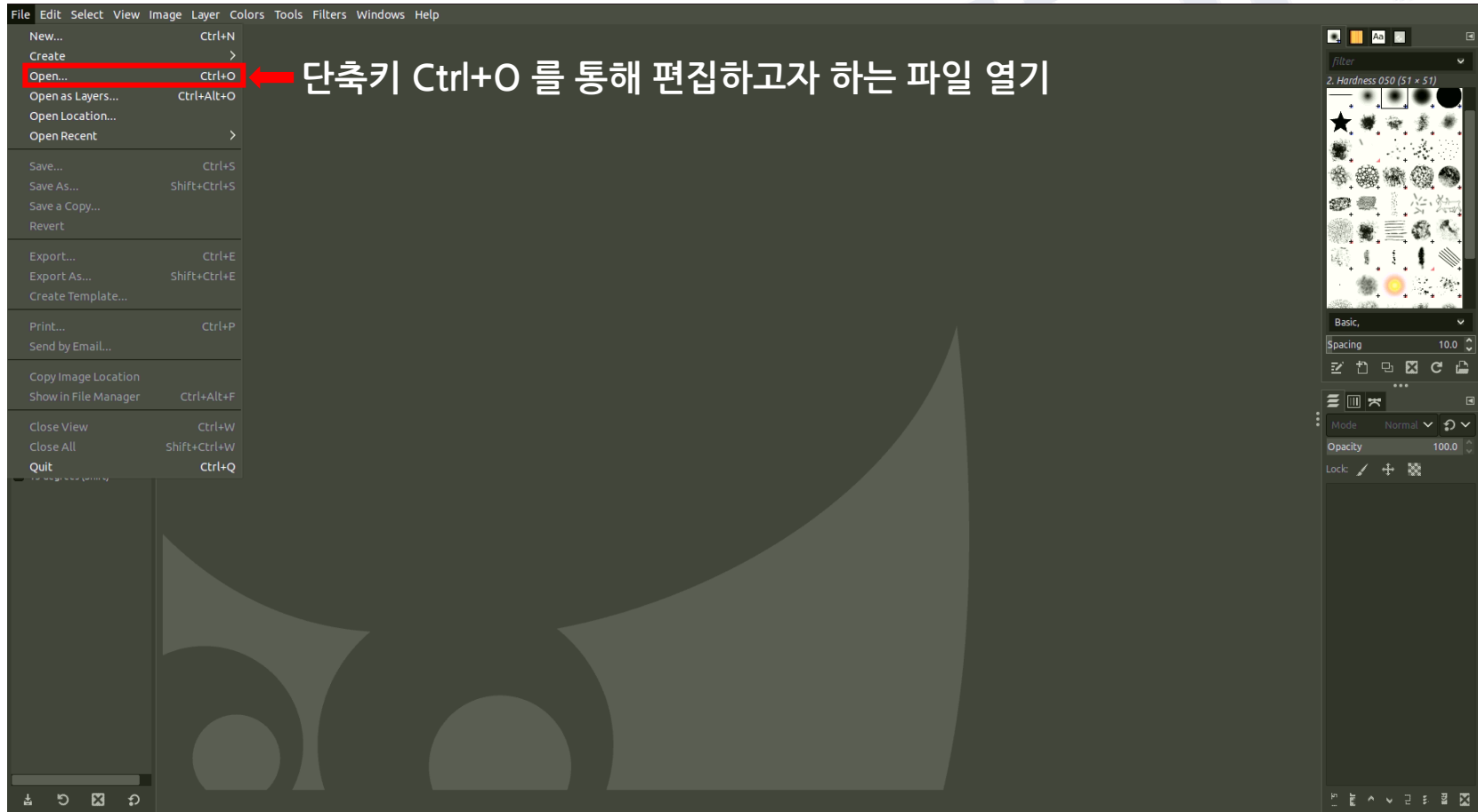
\$ sudo apt install gimp (재 수행)

\$ gimp (GIMP 실행)

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

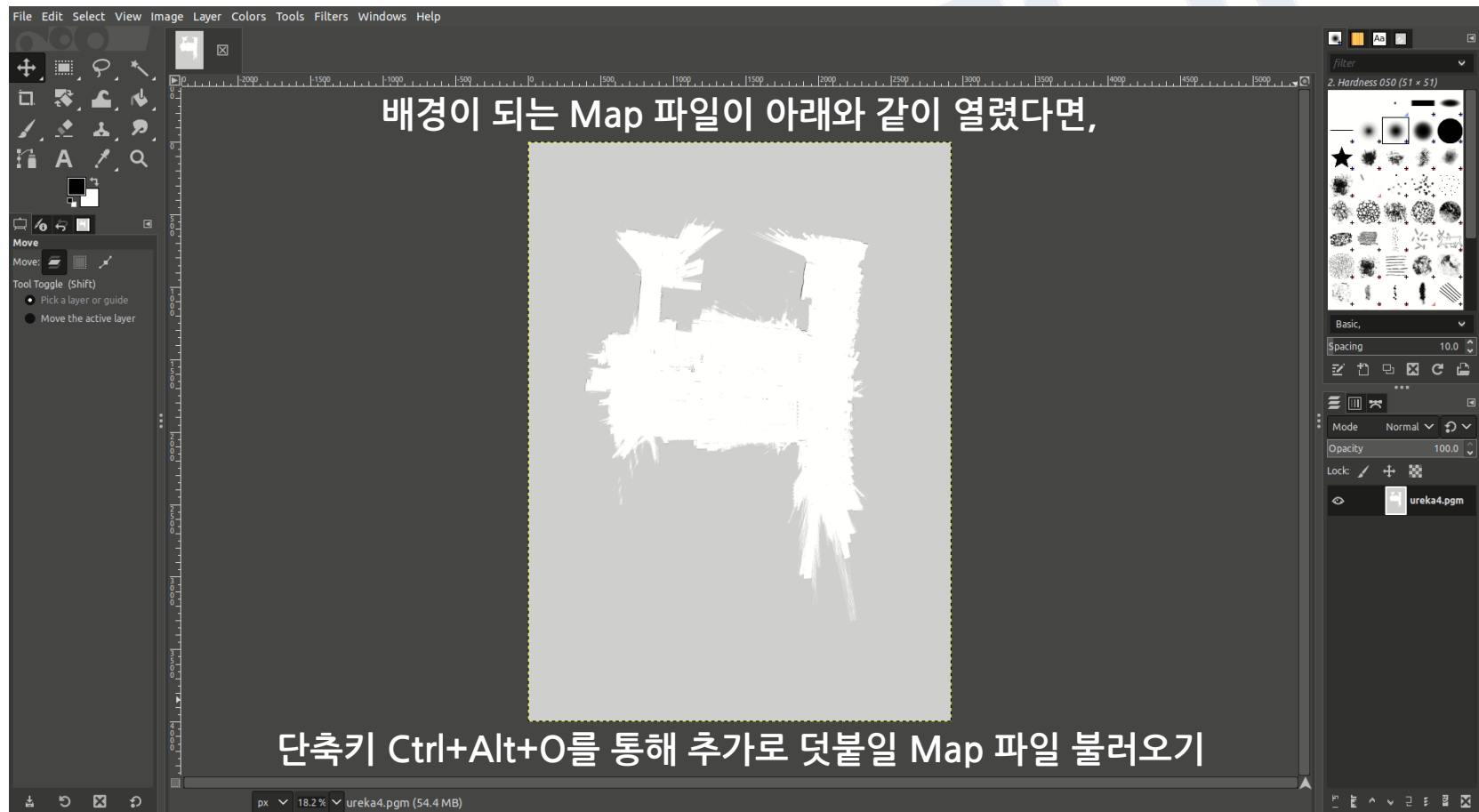
GIMP 편집기 사용 방법 - Map 파일 불러오기



4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

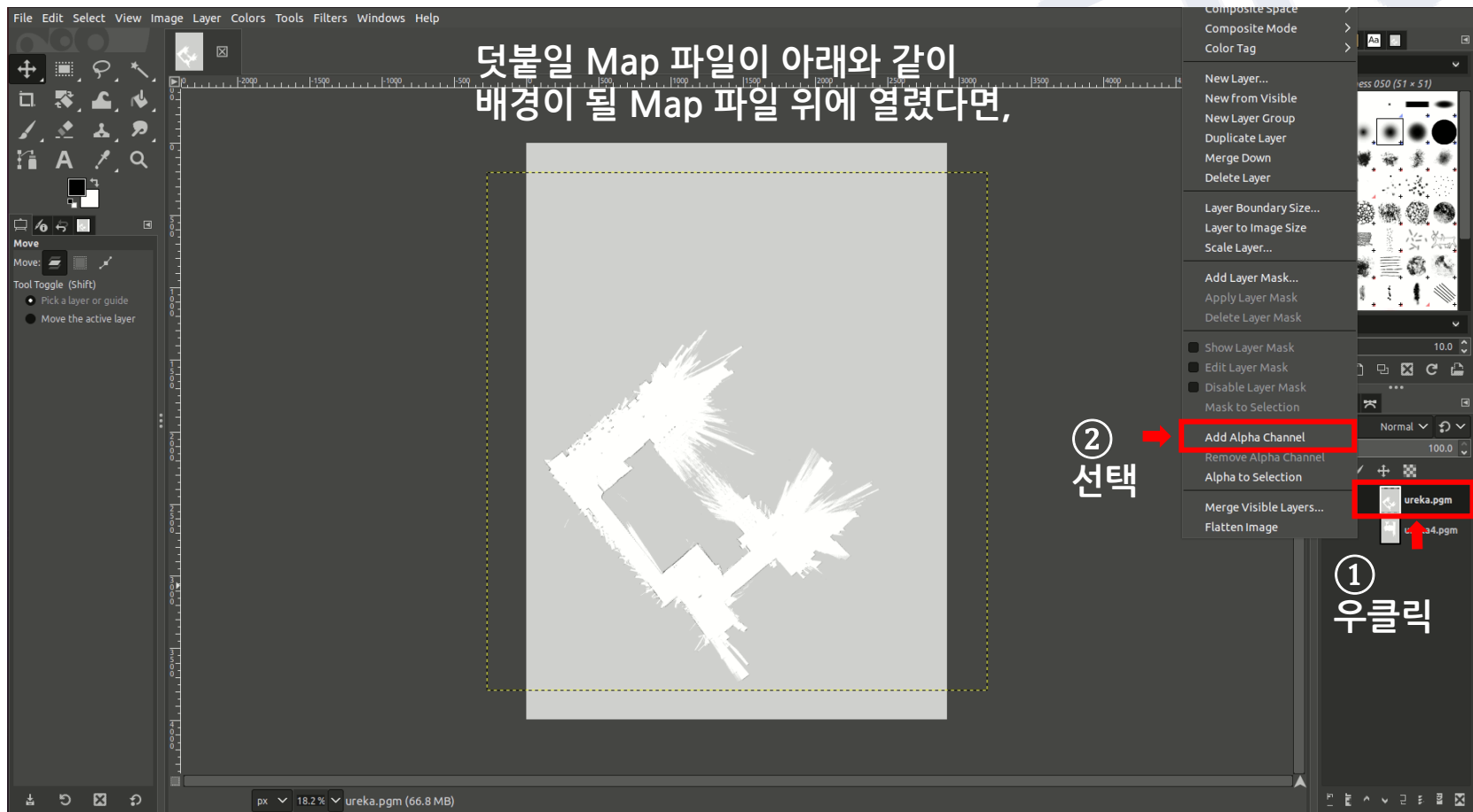
GIMP 편집기 사용 방법 - Map 파일 불러오기



4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 병합 (2개 이상의 Map 병합)

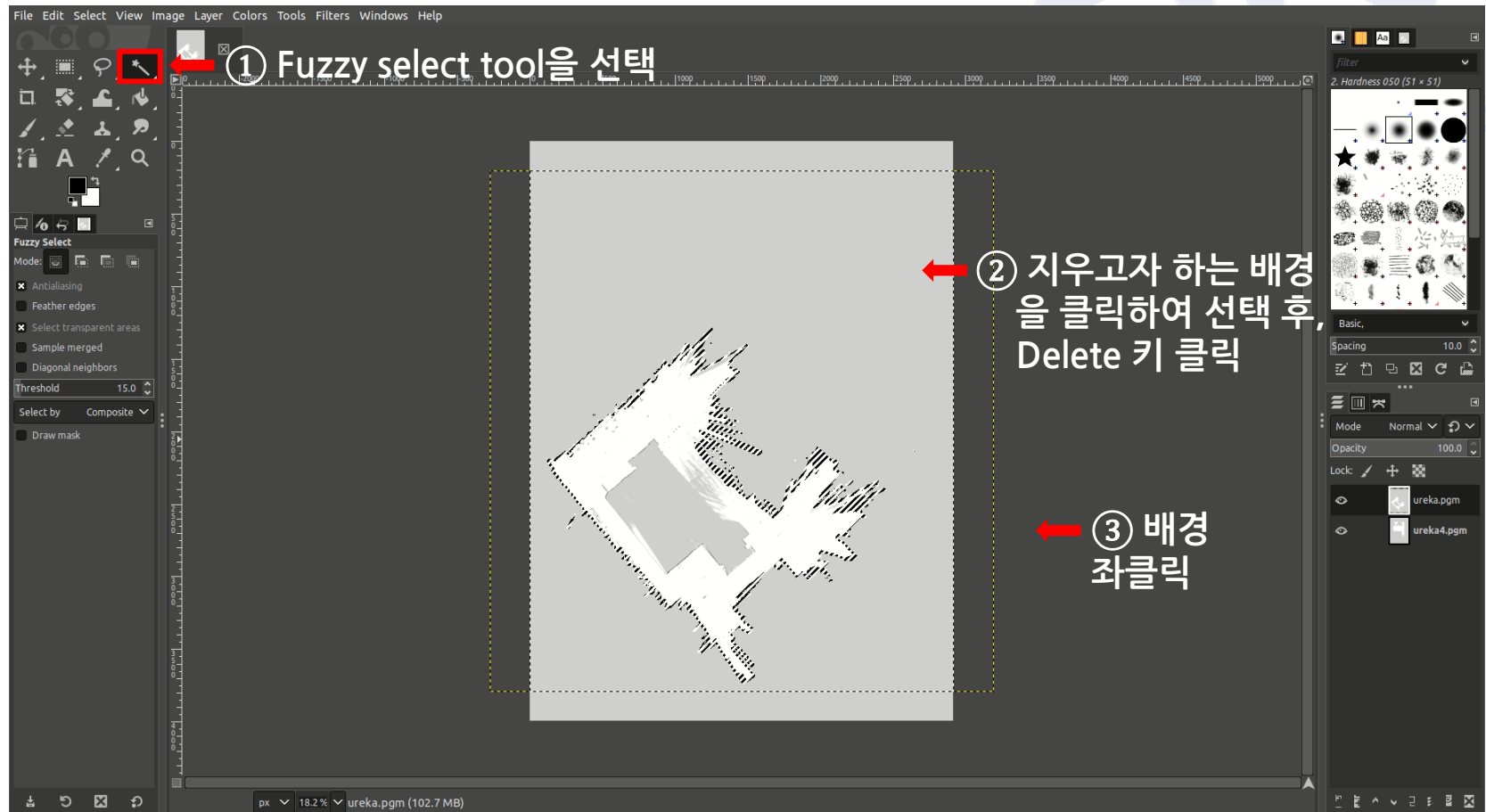


➤ ①~② 과정은 덧붙일 Map 파일의 배경을 제거하기 위한 전 처리 과정

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 병합 (2개 이상의 Map 병합)

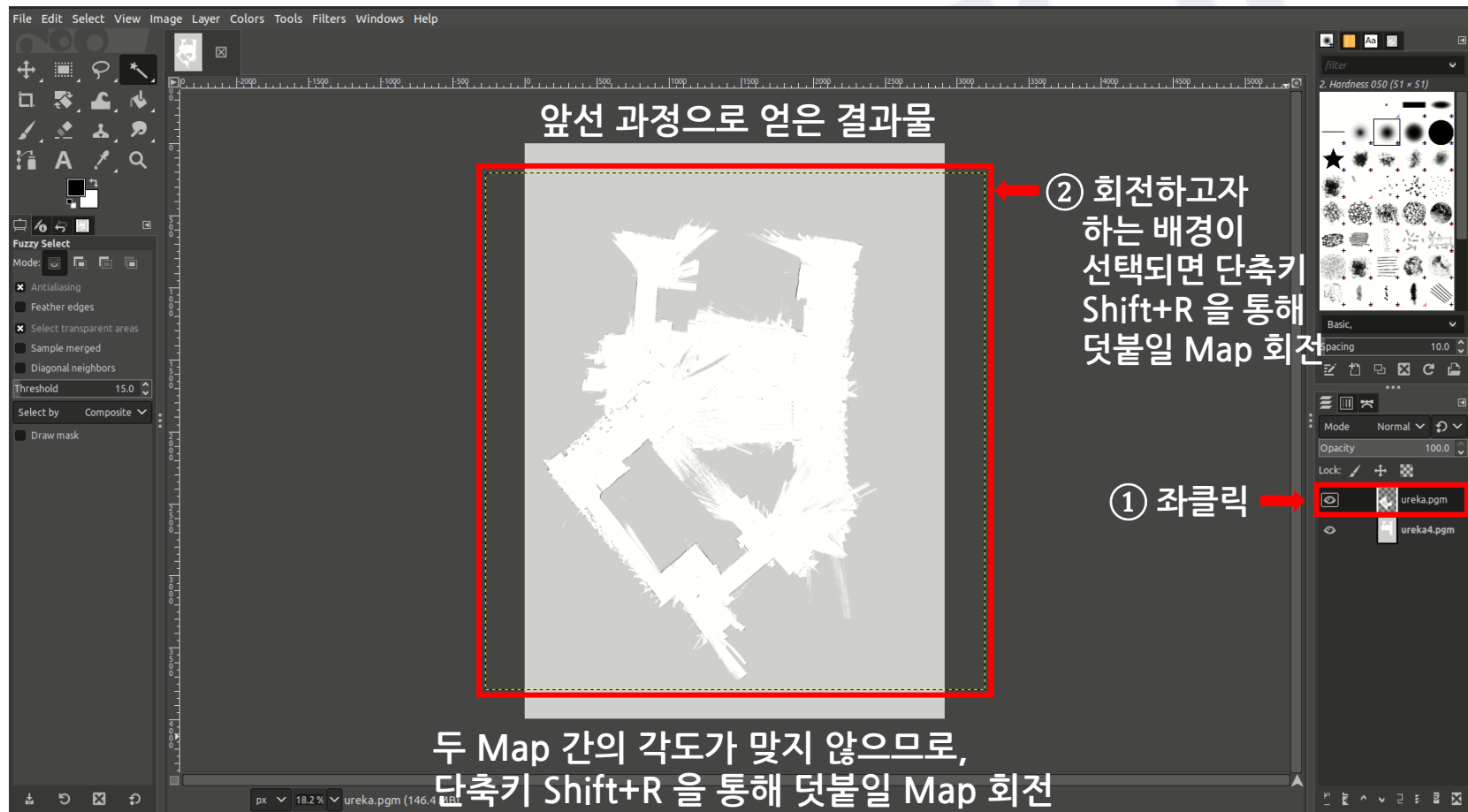


➤ ①~③ 과정을 통해 덧붙일 Map 파일의 배경 제거

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

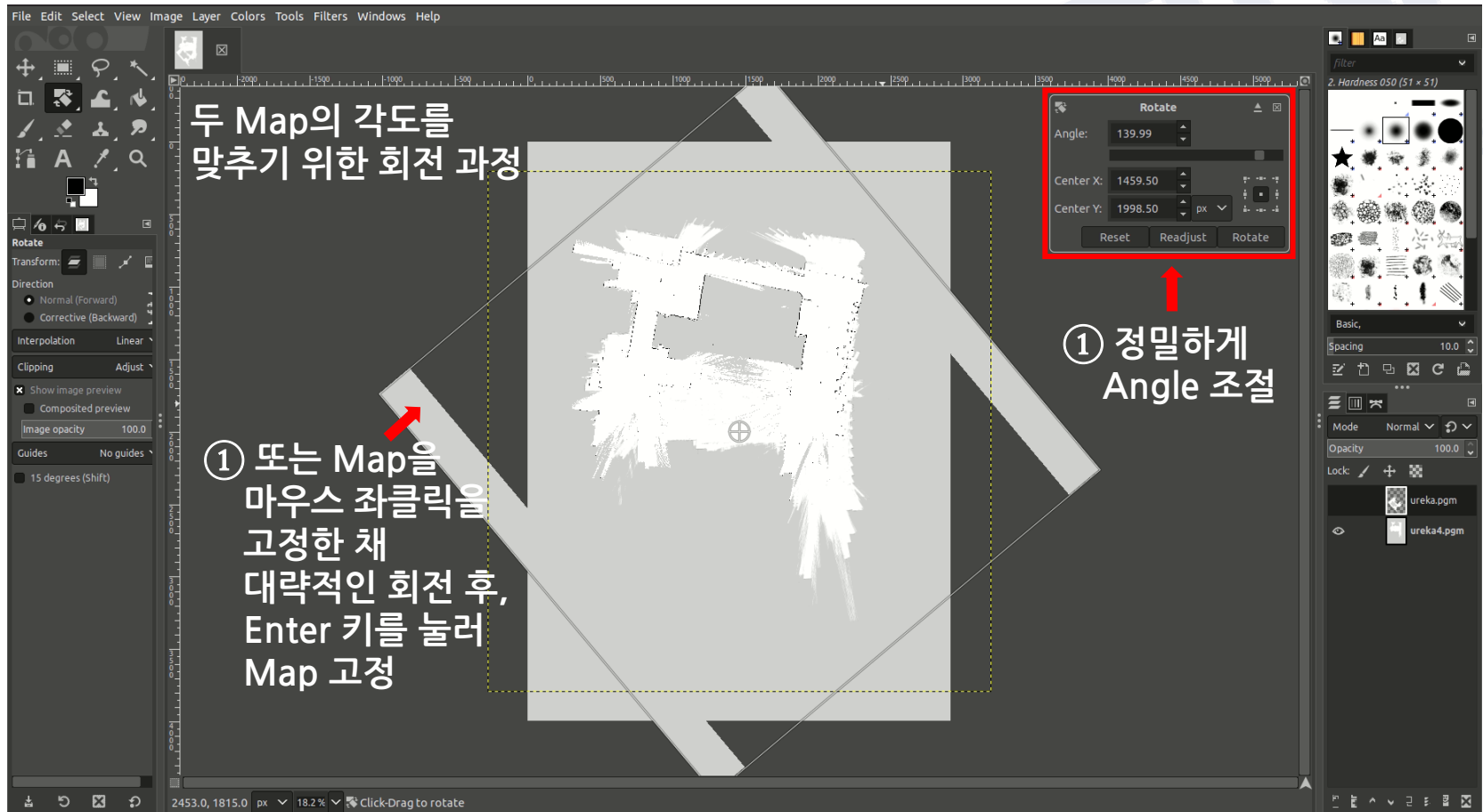
GIMP 편집기 사용 방법 - Map 병합 (2개 이상의 Map 병합)



4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 병합 (2개 이상의 Map 병합)

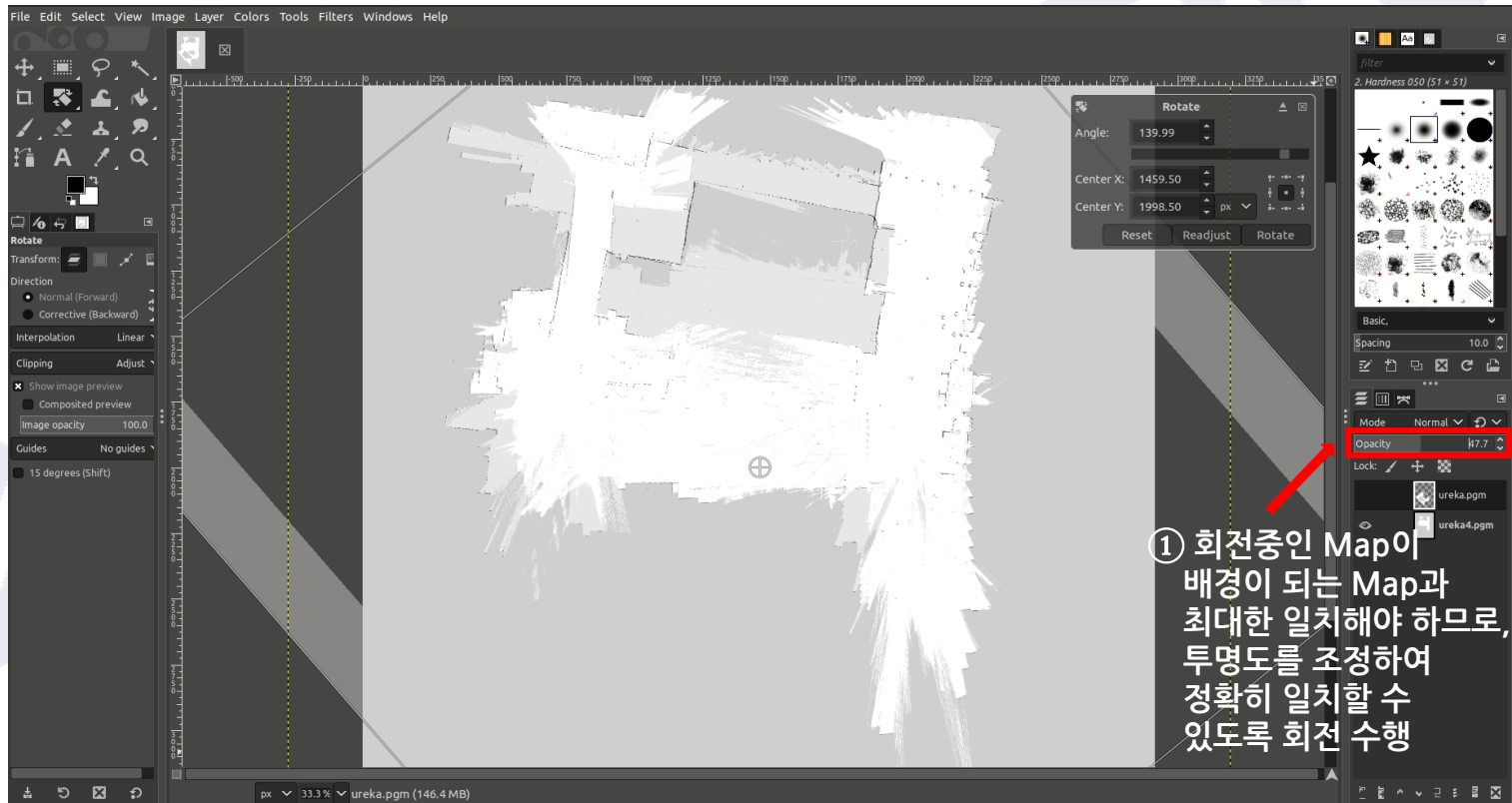


- 위 과정을 통해 각도를 맞춘 뒤, Enter 키로 Map을 고정하고 단축키 M을 눌러 마우스로 Map을 드래그하여 이동 가능.

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 병합 (2개 이상의 Map 병합)

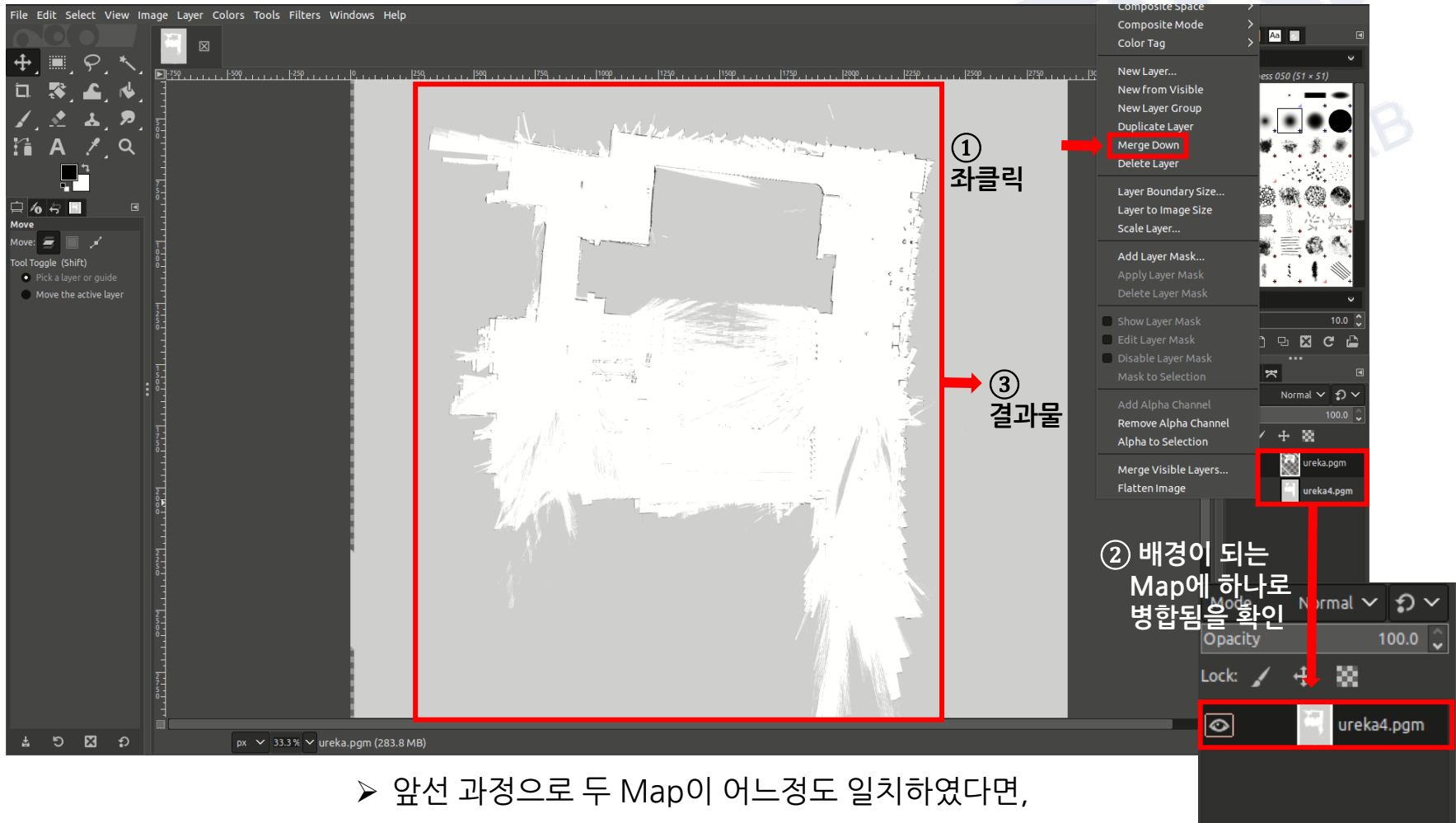


- Map을 결합하는 과정에서 보다 정확한 결합을 위해 덧붙이는 Map의 투명도를 조절함으로써 배경이 되는 Map과 일치하도록 보정 수행 가능.
- 두 Map을 일치시킨 후, 반드시 투명도(Opacity)를 다시 100으로 설정해야 함.

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 병합 (2개 이상의 Map 병합)



- 앞선 과정으로 두 Map이 어느정도 일치하였다면,
①의 Merge Down을 통한 두 layer 병합 수행.

4. Advanced ROS Techniques

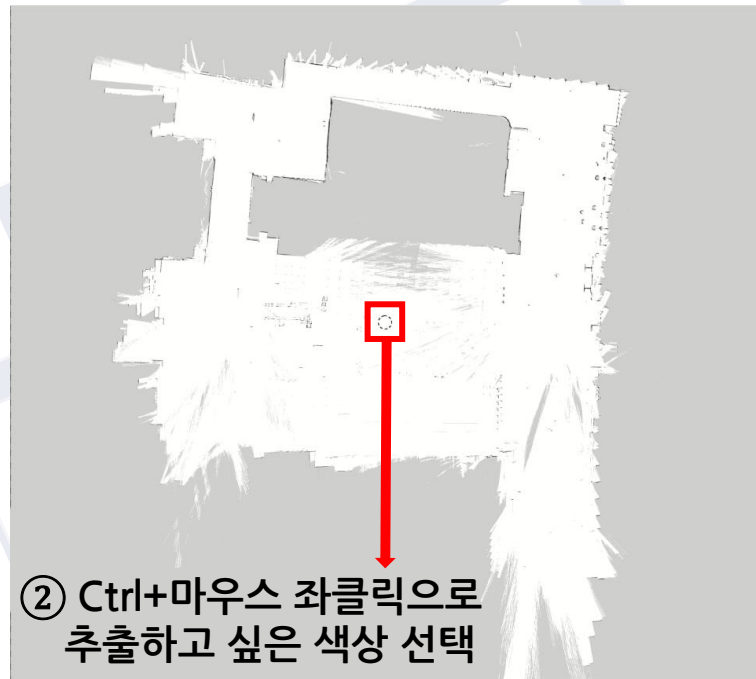
1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 보정 (1개 이상의 Map 수정)

- 만약 앞선 과정들을 수행하였음에도 추가적인 Map 보정이 필요할 경우



- ① 단축키 P를 통해
Paintbrush Tool을 선택
(Map 상의 원하는 색 추출)



- ② Ctrl+마우스 좌클릭으로
추출하고 싶은 색상 선택

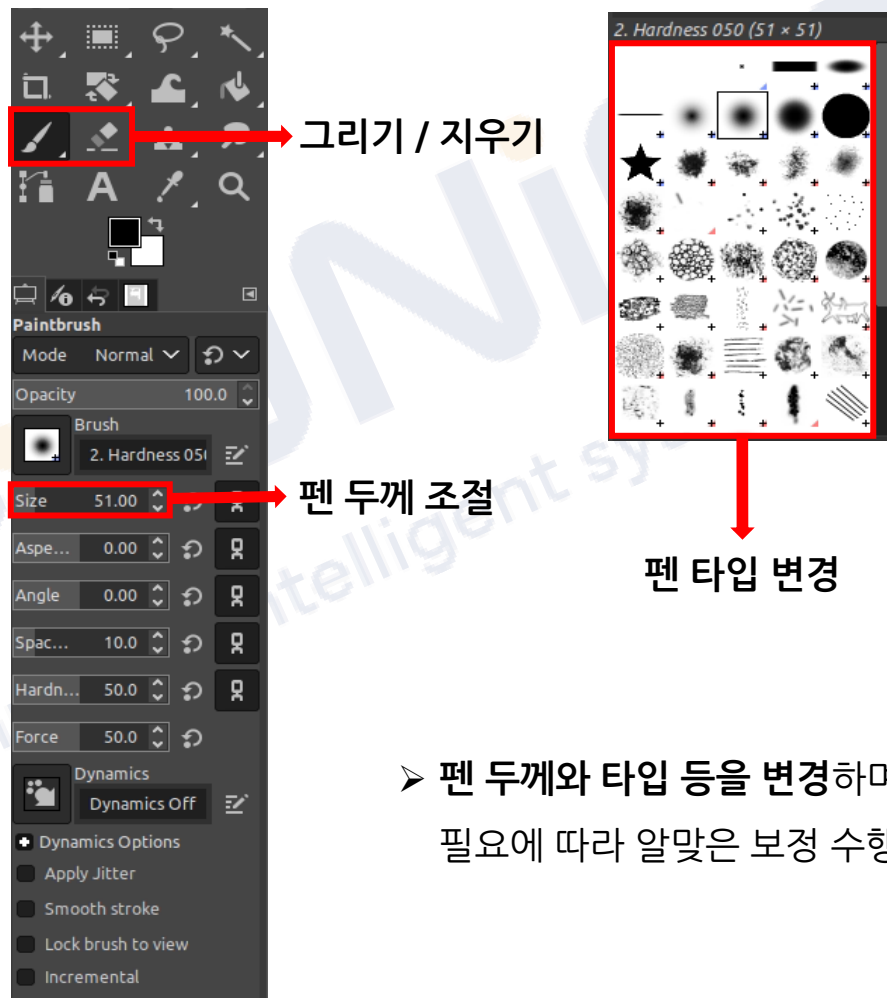
- ③ 색상 추출 후 보정하고자 하는 부분을 마우스 좌 클릭을 누른 채 드래그 하거나,
마우스 좌클릭을 반복적으로 수행하여 보정 수행.

(만약, 직선 보정 수행을 위해서는 직선 시작 지점을 **마우스 좌 클릭** 후,
도착 지점에 **Shift+마우스 좌 클릭**을 수행하면 일직선의 보정을 수행)

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 보정 (1개 이상의 Map 수정)

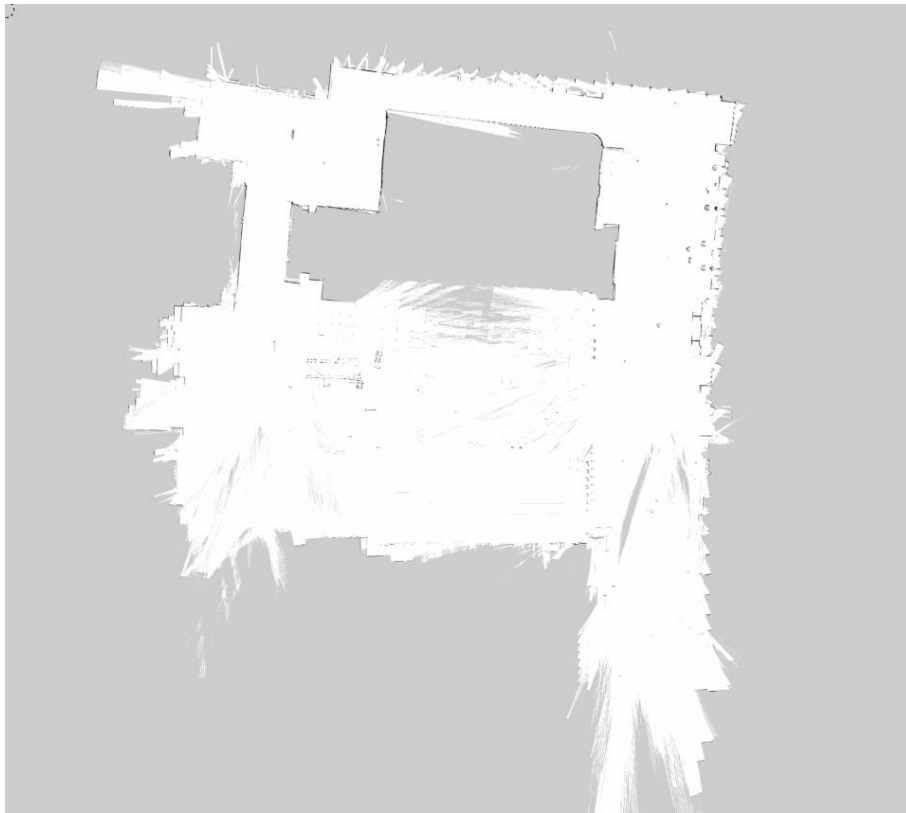


- 펜 두께와 타입 등을 변경하며,
필요에 따라 알맞은 보정 수행 가능

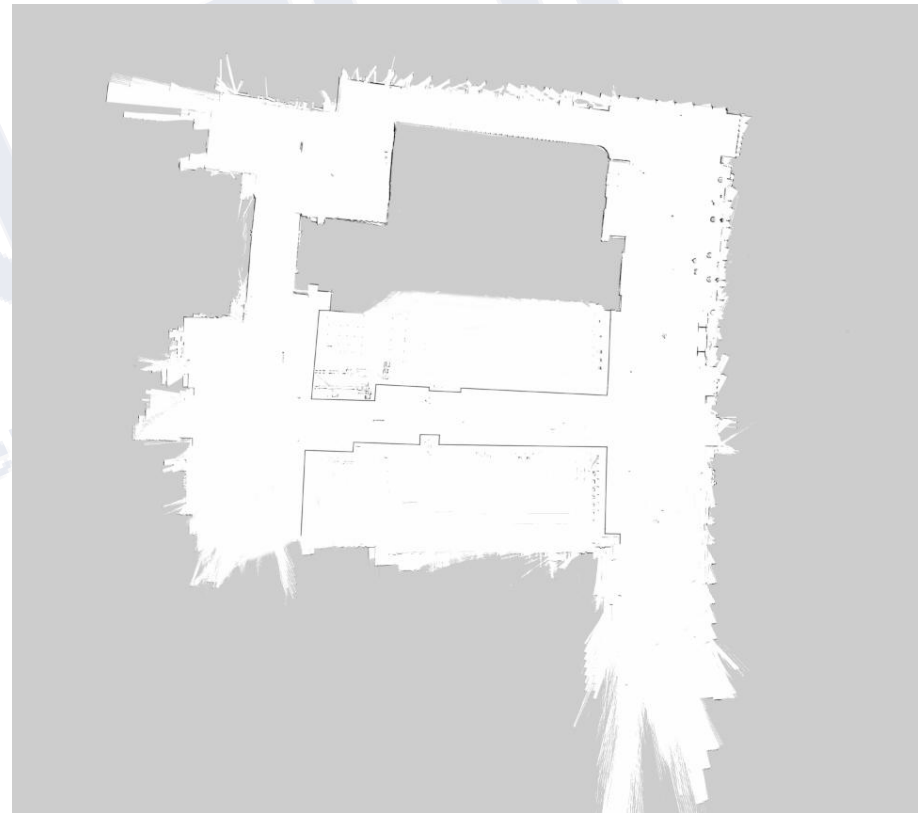
4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 보정 (1개 이상의 Map 수정)



보정 전



보정 후

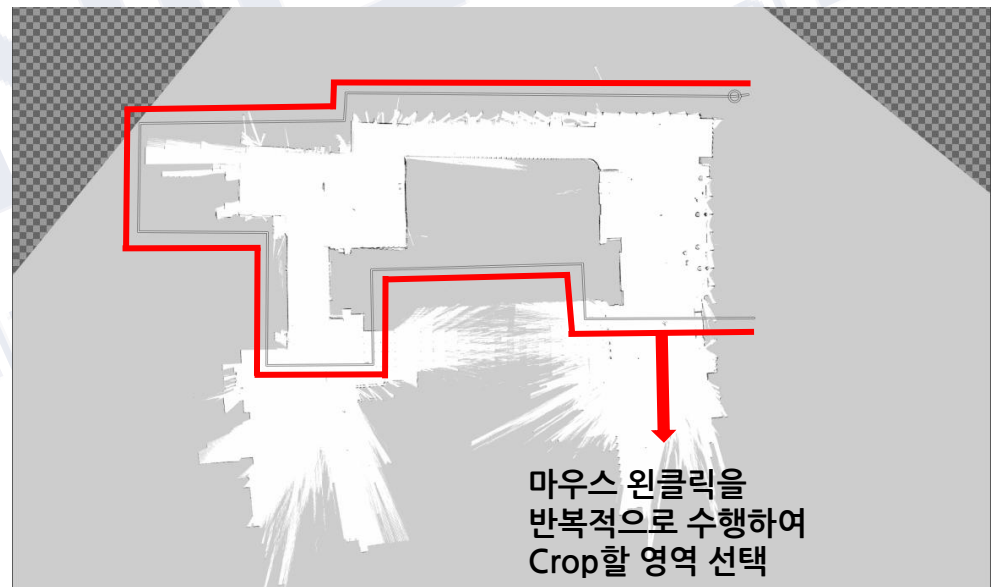
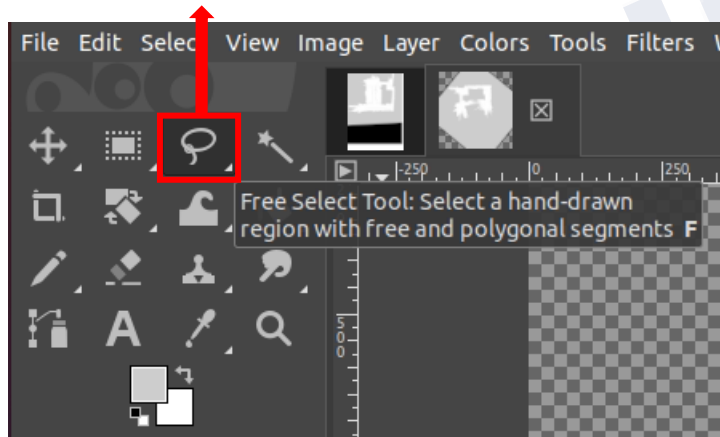
4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 보정 (1개 이상의 Map 수정)

- 보다 수월한 Map 편집을 위해, 이미지를 Crop하여 복사 및 붙여 넣는 방법

Free Select Tool 선택
(단축키 : F)

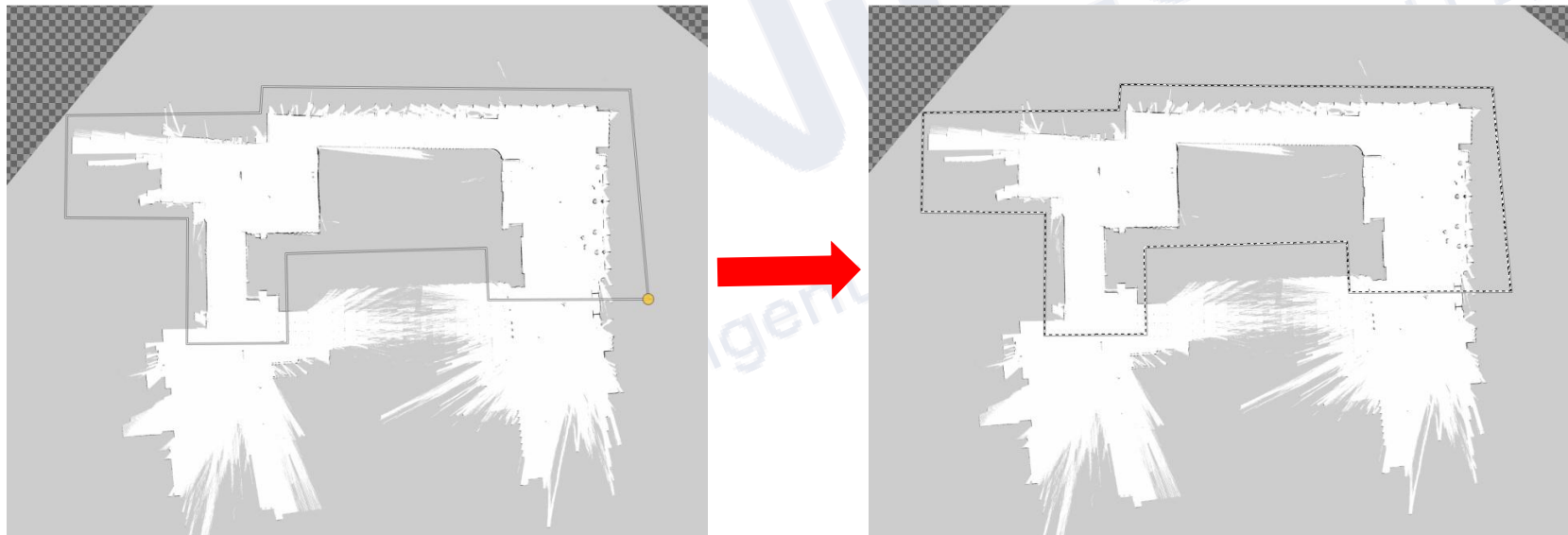


4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 보정 (1개 이상의 Map 수정)

- Crop할 영역이 완성되면, 오른쪽 그림과 같이 영역이 점선으로 나타나게 됨.

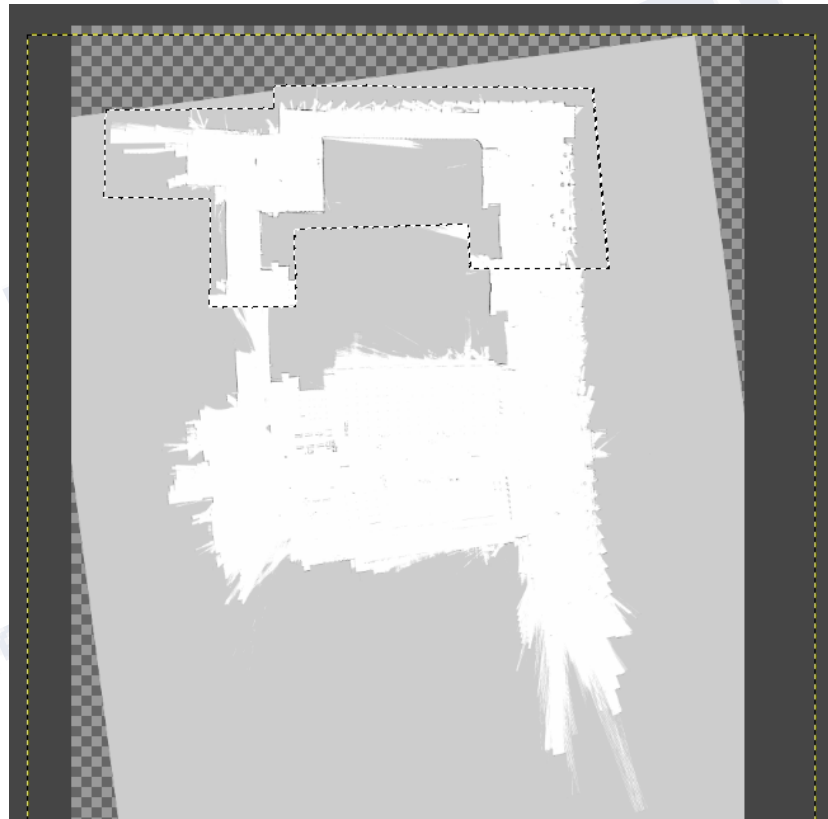


- 단축키 Ctrl + C를 통해 점선으로 둘러싸인 영역을 복사

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 보정 (1개 이상의 Map 수정)



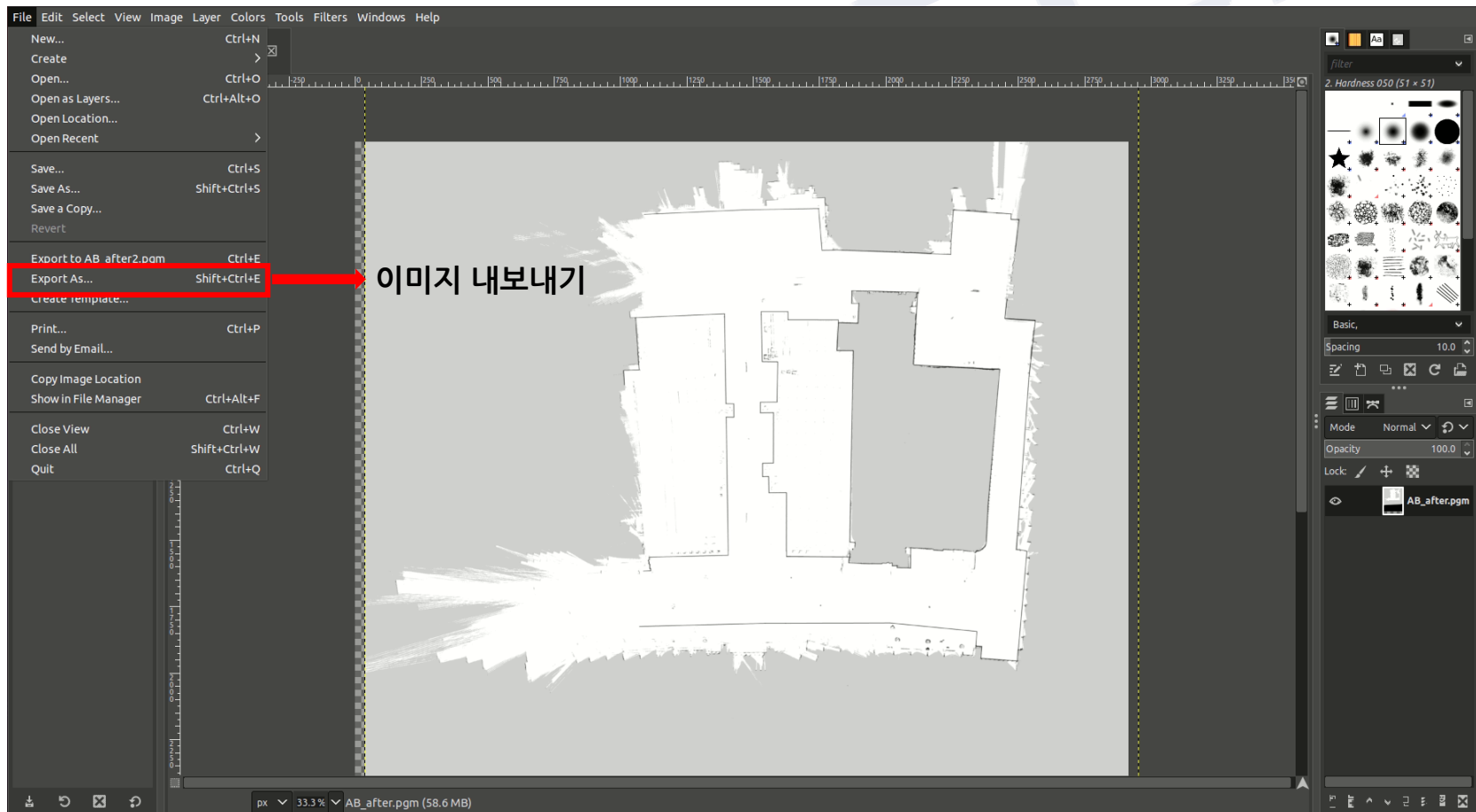
- Ctrl + V를 통해 복사한 영역을, 수정중인 다른 Map에 붙여넣기 수행.

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 저장

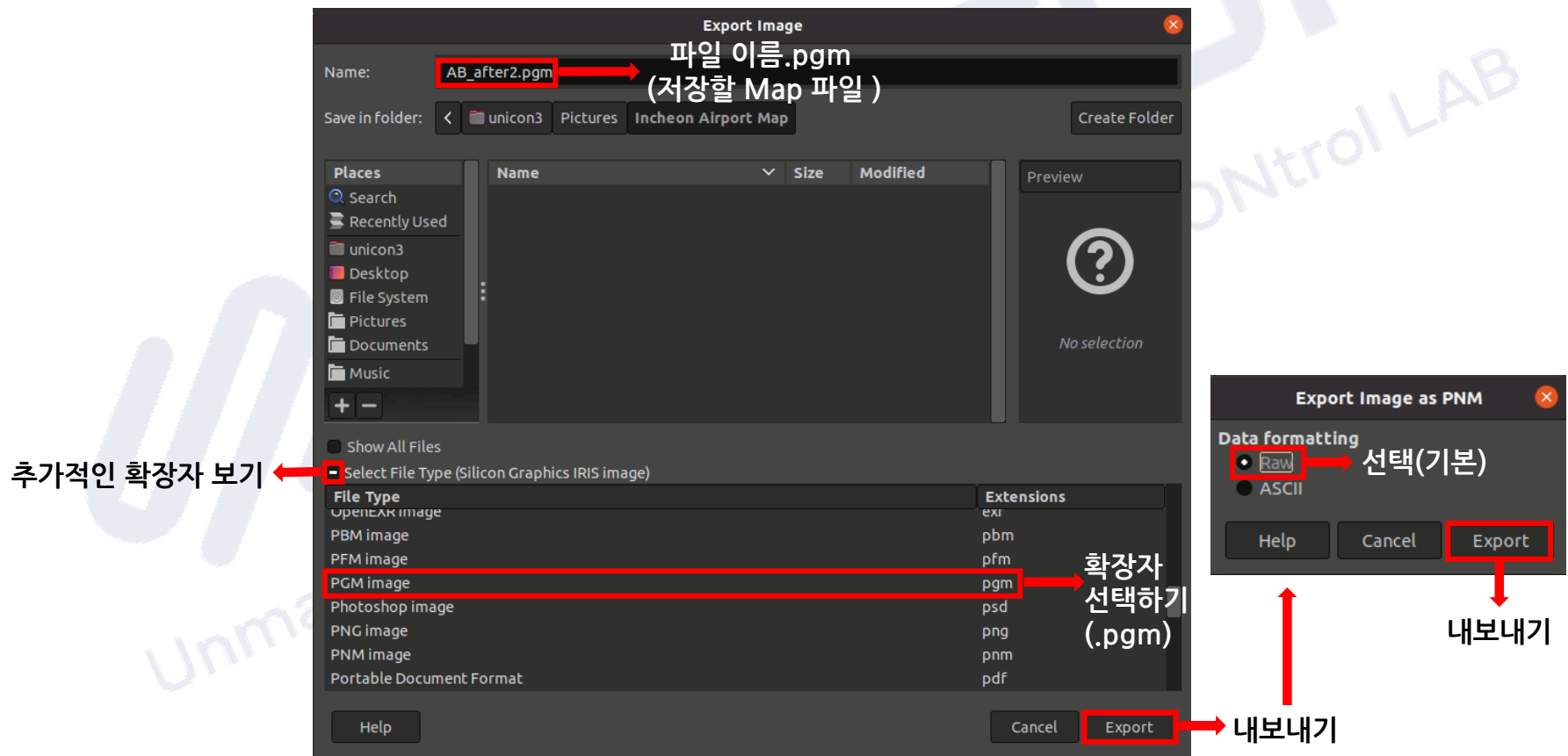
➤ Map에 대한 모든 보정 작업이 끝나고, 해당 Map 파일을 저장하기 위해서는,



4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 저장

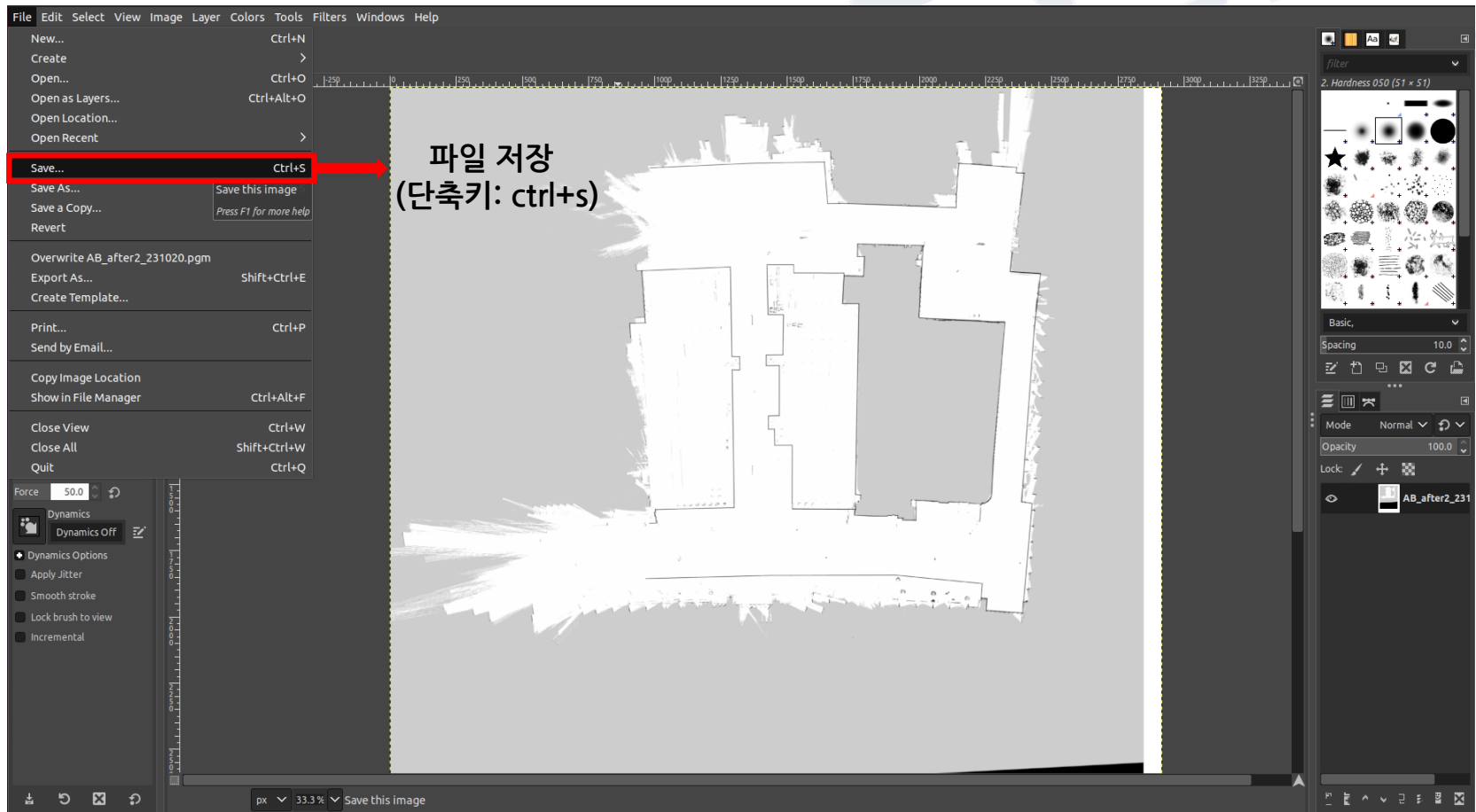


4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - Map 저장

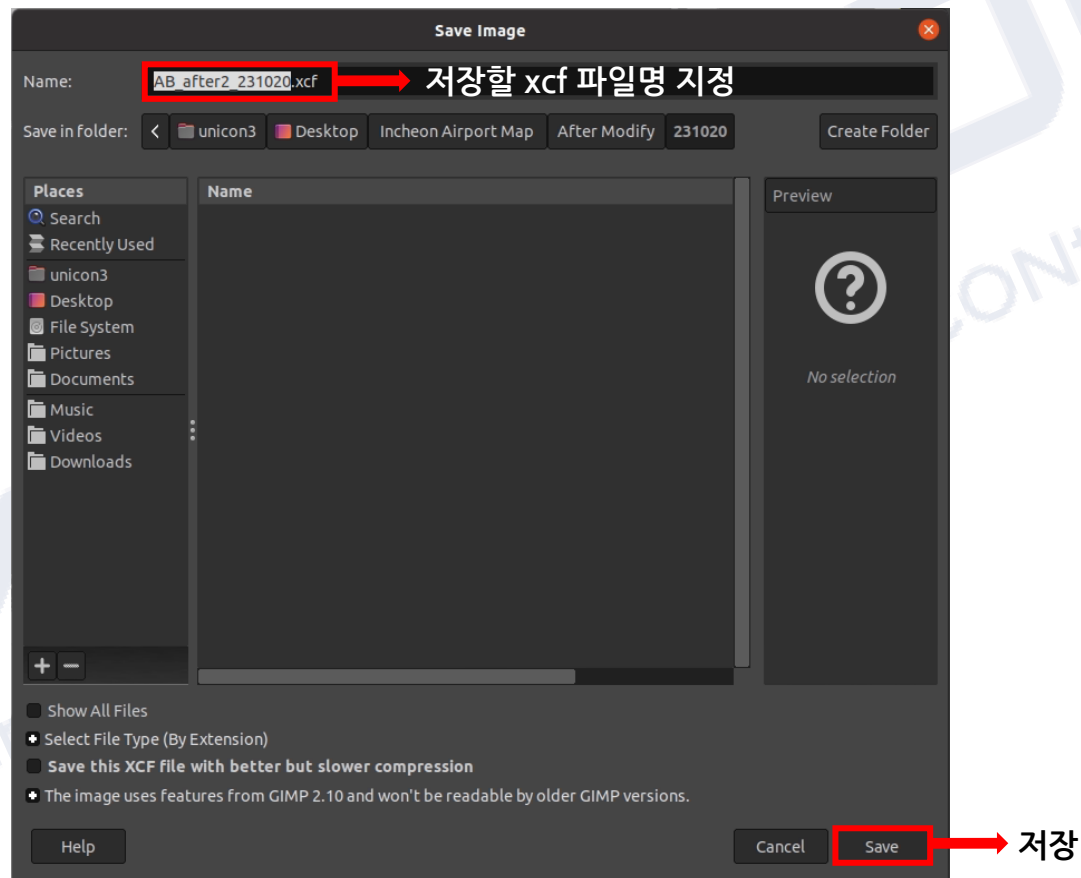
➤ 작업중인 Map 이미지 정보를 재 편집 시 사용하기 위한, xcf 파일로 저장하기 위해



4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

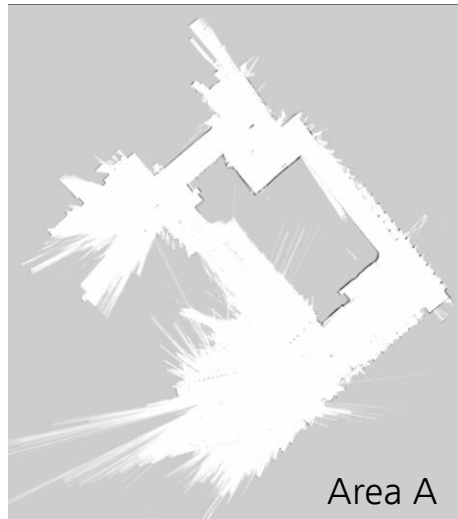
GIMP 편집기 사용 방법 - Map 저장



- 이와 같이 xcf 파일을 저장하게 되면 이미지의 레이어, 텍스트, 효과 및 다른 추가적인 요소를 보존하게 되며 향후 이미지 재 편집 시 용이함.

4. Advanced ROS Techniques

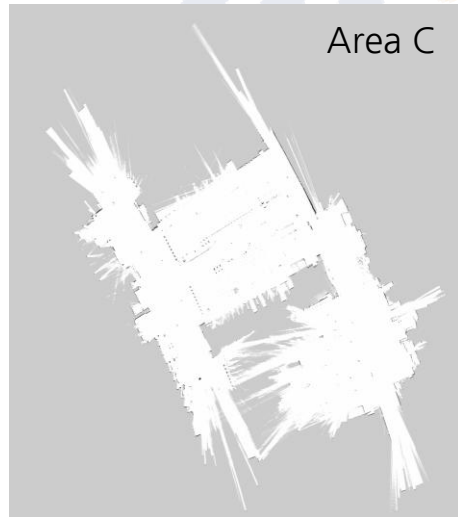
1) GIMP 기반 Occupancy Grid Map 보정



Area A



Area B



Area C



Area D

Incheon Airport 3f West A~D Area Mapping Results

GIMP 편집기 사용 방법 - Map 보정 예시



Incheon Airport 3f A~D Area Map Calibration results

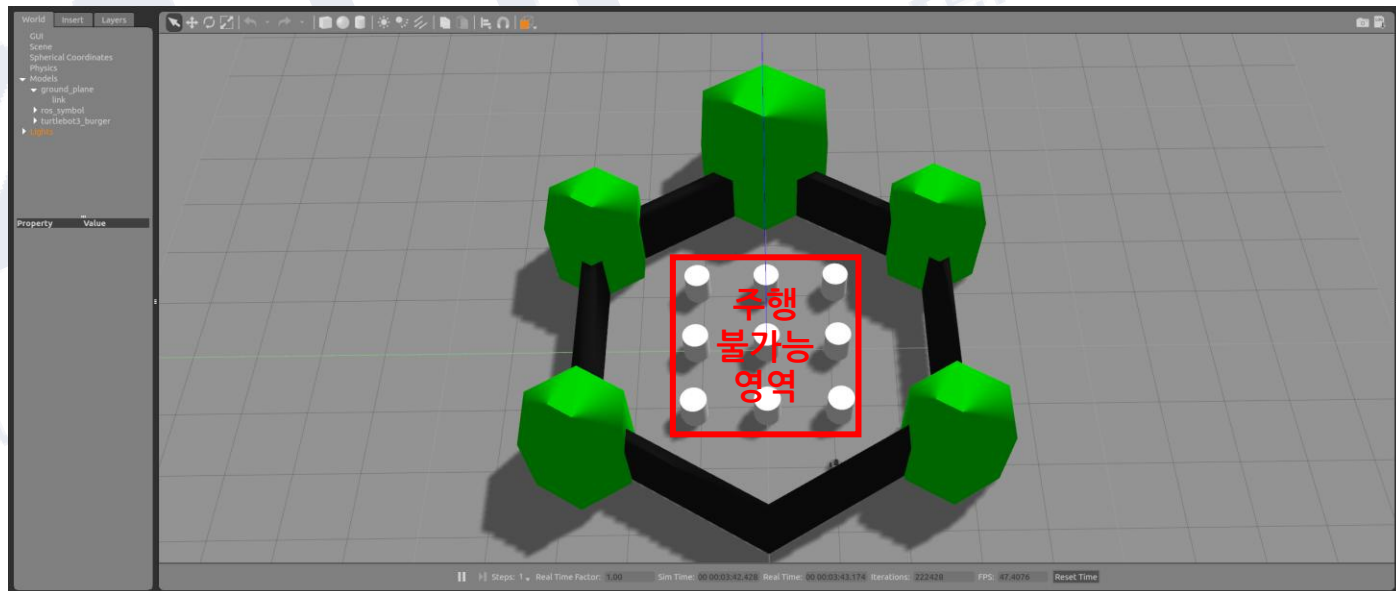
- ❖ 인천공항 3층 동편 A ~ D 구역 각각의 Mapping 결과와 A ~ D 구역의 Map을 결합 후 보정한 결과.

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - 실습

- 앞선 GIMP 편집기를 사용하여, 이전 Turtlebot3 시뮬레이션 환경에서 SLAM을 통해 얻었던 Occupancy Grid Map 보정 수행
- 실습 1. 다음 그림과 같이, Turtlebot3 burger 모델이 주어진 환경의 내부로 주행하지 못하도록 GIMP 편집기를 통해 Occupancy Grid Map을 보정

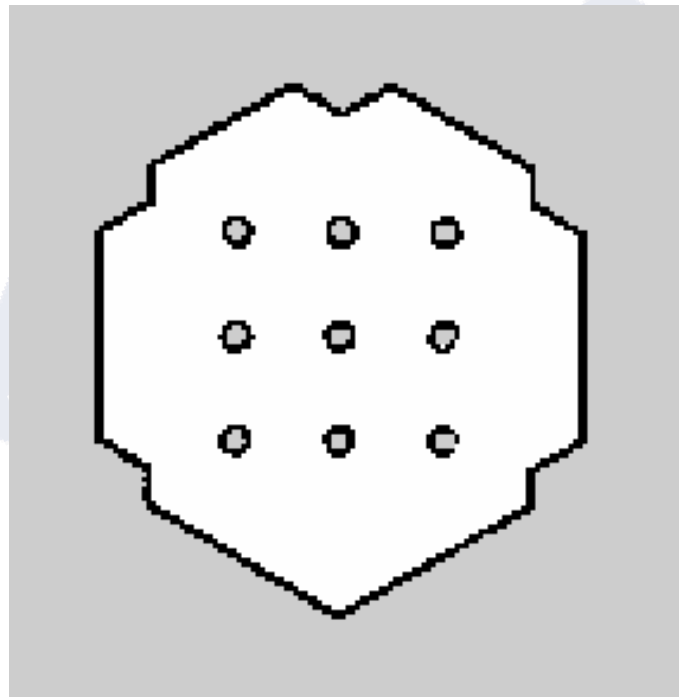


4. Advanced ROS Techniques

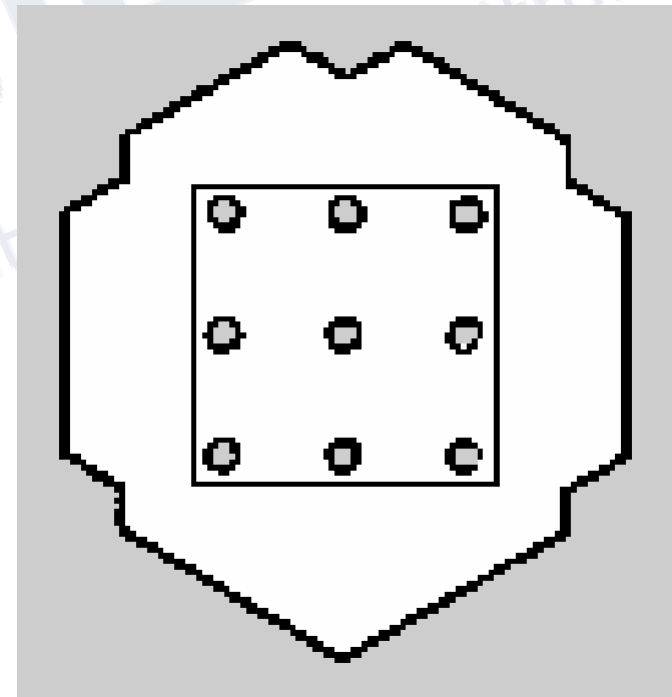
1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - 실습

- 실습 1. 다음 그림과 같이, Turtlebot3 burger 모델이 주어진 환경의 내부로 주행하지 못하도록 GIMP 편집기를 통해 Occupancy Grid Map을 보정



보정 전



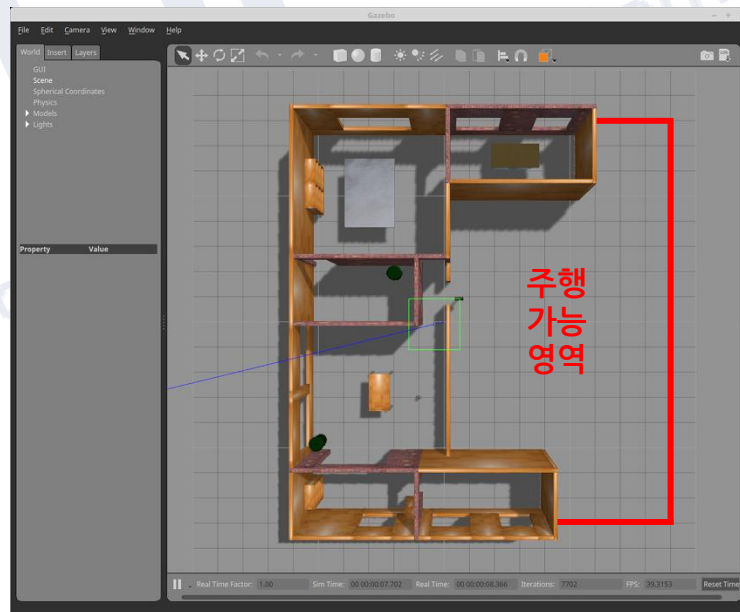
보정 후

4. Advanced ROS Techniques

1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - 실습

- 앞선 GIMP 편집기를 사용하여, 이전 Turtlebot3 시뮬레이션 환경에서 SLAM을 통해 얻었던 Occupancy Grid Map 보정 수행
- 실습 2. 다음 그림과 같이, Turtlebot3 burger 모델이 주어진 환경의 외부로 주행할 수 있도록 GIMP 편집기를 통해 Occupancy Grid Map을 보정

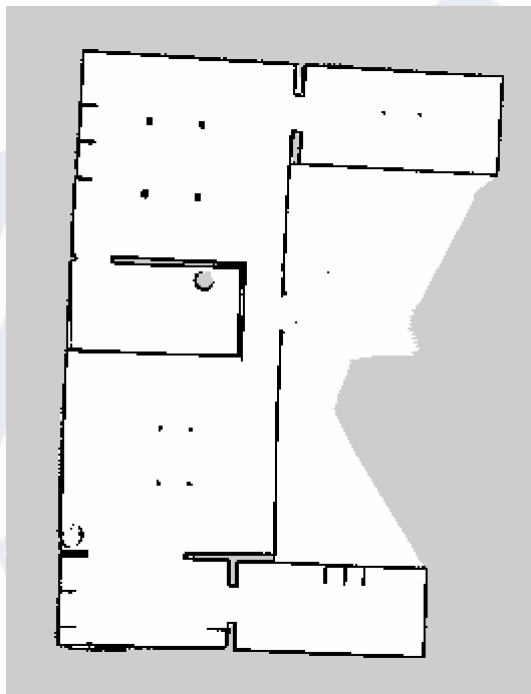


4. Advanced ROS Techniques

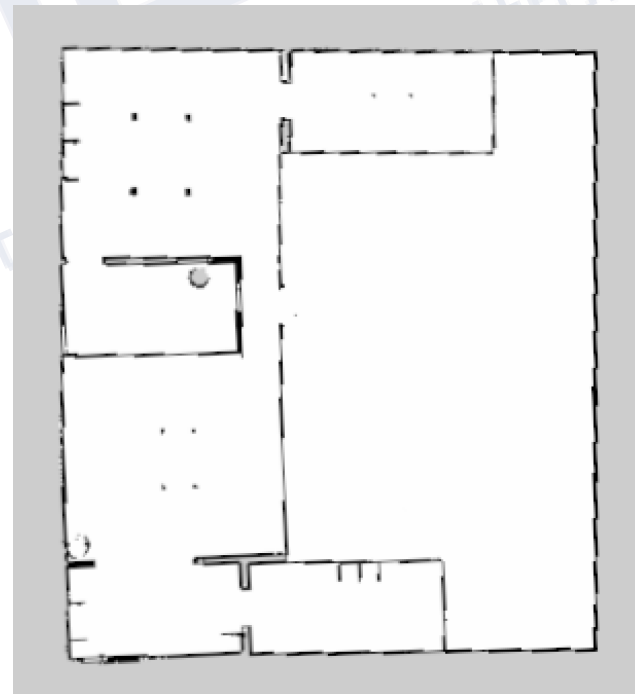
1) GIMP 기반 Occupancy Grid Map 보정

GIMP 편집기 사용 방법 - 실습

- 실습 2. 다음 그림과 같이, Turtlebot3 burger 모델이 주어진 환경의 외부로 주행할 수 있도록 GIMP 편집기를 통해 Occupancy Grid Map을 보정



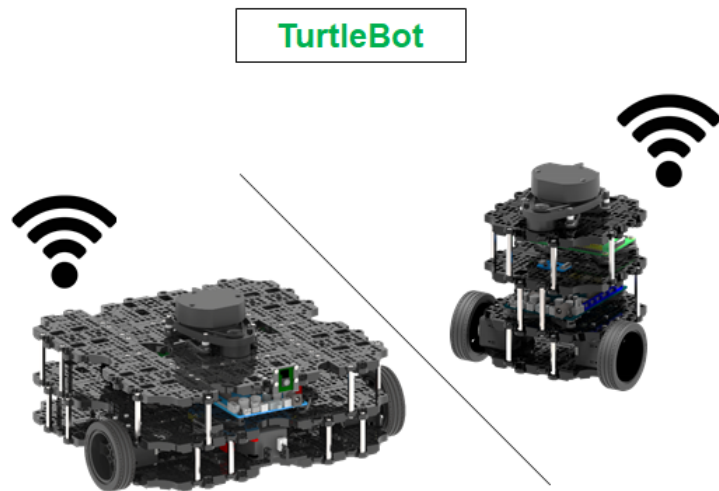
보정 전



보정 후

4. Advanced ROS Techniques

2) 로봇 원격 접속



ROS_MASTER_URI=http://<IP_of_robot>:11311
 ROS_HOSTNAME=<IP_of_robot>



ROS_MASTER_URI=http://<IP_of_robot>:11311
 ROS_HOSTNAME=<IP_of_remote_PC>

※ 로봇에서 ROS master를 동작시키는 것을 권장함

4. Advanced ROS Techniques

2) 로봇 원격 접속

- IP 주소 확인

```
$ sudo apt install net-tools
```

```
$ ifconfig
```

```
cm1n@cm1n:~$ ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 179034730 bytes 32018712839 (32.0 GB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 179034730 bytes 32018712839 (32.0 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp59s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.77.151 netmask 255.255.255.0 broadcast 192.168.77.255
    inet6 fe80::67b7:bf02:25:7fe6 prefixlen 64 scopeid 0x20<link>
    ether c0:b8:83:38:27:ff txqueuelen 1000 (Ethernet)
    RX packets 636122 bytes 258979338 (258.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1222680 bytes 1207868658 (1.2 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

4. Advanced ROS Techniques

2) 로봇 원격 접속

- bashrc에 추가 (예시)

Robot bashrc

```
export ROS_MASTER_URI=http://192.168.77.100:11311  
export ROS_HOSTNAME=192.168.77.100
```

Remote PC bashrc

```
export ROS_MASTER_URI=http://192.168.77.100:11311  
export ROS_HOSTNAME=192.168.77.151
```

※ 본인의 IP에 맞게 수정

4. Advanced ROS Techniques

2) 로봇 원격 접속

- 로봇의 토픽 확인

```
/joint_states
/map
/map_metadata
/map_updates
/move_base/DWAPlannerROS/cost_cloud
/move_base/DWAPlannerROS/global_plan
/move_base/DWAPlannerROS/local_plan
/move_base/DWAPlannerROS/parameter_descriptions
/move_base/DWAPlannerROS/parameter_updates
/move_base/DWAPlannerROS/trajectory_cloud
/move_base/NavfnROS/plan
/move_base/cancel
/move_base/current_goal
/move_base/feedback
/move_base/global_costmap/costmap
/move_base/global_costmap/costmap_updates
/move_base/global_costmap/footprint
/move_base/global_costmap/inflation_layer/parameter_descriptions
/move_base/global_costmap/inflation_layer/parameter_updates
/move_base/global_costmap/obstacle_layer/parameter_descriptions
/move_base/global_costmap/obstacle_layer/parameter_updates
/move_base/global_costmap/parameter_descriptions
/move_base/global_costmap/parameter_updates
/move_base/global_costmap/static_layer/parameter_descriptions
/move_base/global_costmap/static_layer/parameter_updates
/move_base/goal
/move_base/local_costmap/costmap
/move_base/local_costmap/costmap_updates
/move_base/local_costmap/footprint
/move_base/local_costmap/inflation_layer/parameter_descriptions
/move_base/local_costmap/inflation_layer/parameter_updates
/move_base/local_costmap/obstacle_layer/parameter_descriptions
/move_base/local_costmap/obstacle_layer/parameter_updates
/move_base/local_costmap/parameter_descriptions
/move_base/local_costmap/parameter_updates
/move_base/parameter_descriptions
/move_base/parameter_updates
/move_base/recovery_status
/move_base/result
/move_base/status
/move_base_simple/goal
/odom
/particlecloud
/raspicam_node/image/mouse_click
/rosout
/rosout_agg
/scan
```

4. Advanced ROS Techniques

2) 로봇 원격 접속

- 로봇의 터미널에 직접 접속

```
$ ssh <name_of_robot_pc>@<IP_of_robot>
```

※ ssh 방식을 통한 원격 접속을 위해서는 PC의 이름과 비밀번호가 필요함

※ ssh로 접속한 터미널에서 특정창을 여는 명령어(gedit, rviz, rqt, ...)는 추가 옵션이 필요하니 권장하지 않음

※ 따라서 ssh를 사용할 때, 편집기는 터미널에서 바로 편집하는 편집기(vim, nano, ...)를 사용하고, 토픽 데이터를 확인할 때는 IP 토픽 공유를 통한 remote PC에서 확인하는 것을 권장함

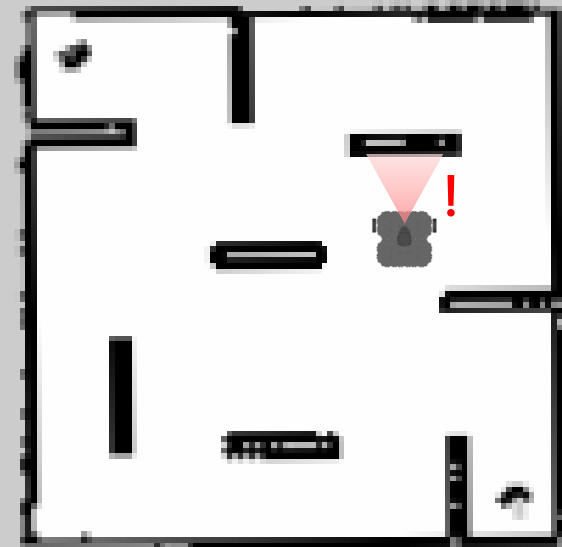
4. Advanced ROS Techniques

3) 심화 연습 문제

- Laser filter 패키지를 이용하여 라이다 기반 장애물 경고 노드 작성

Hint1 - 라이다 데이터 토픽: /scan

Hint2 - 적절한 라이다 감지 범위를 설정



4. Advanced ROS Techniques

3) 심화 연습 문제

- Laser filter 패키지를 이용하여 라이다 기반 장애물 경고 노드 작성

❖ laser_filter → https://wiki.ros.org/laser_filters

```
$ sudo apt install ros-noetic-laser-filters
```

```
$ cd ~/catkin_ws/src  
$ git clone https://github.com/cmin87394/laser_filter_example.git  
$ cd .. && catkin_make
```

filtering 된 scan 토픽: /scan_filtered

❖ Array 처리

numpy.array

numpy.where

4. Advanced ROS Techniques

3) 심화 연습 문제

- 다음 지도상에서 표시된 세 목표지점을 터틀봇이 차례대로 자율주행으로 이동하는 노드 작성
(rviz 상에서 클릭으로 goal을 찍는 것이 아닌 토픽 publish로 goal 생성)

Hint1 - 터틀봇의 위치 관련 토픽: /amcl_pose

Hint2 - Goal 토픽: /move_base_simple/goal

std_msgs/Header header

uint32 seq

time stamp

string frame_id ← 'map'

geometry_msgs/Pose pose

geometry_msgs/Point position

float64 x ← x 좌표

float64 y ← y 좌표

float64 z

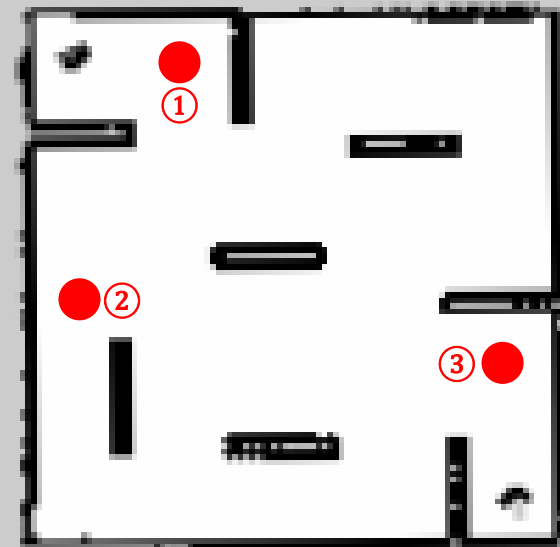
geometry_msgs/Quaternion orientation

float64 x

float64 y

float64 z ← 0.0

float64 w ← 1.0



Appendix

[1] ROS wiki, <https://wiki.ros.org>

[2] 표윤석 박사님 ROS 강의,

https://www.youtube.com/watch?v=ot_D9N-H4lQ&list=PLRG6WP3c31_VlFtFAxSke2NG_DumVZPgW

[3] 오로카, 오픈 로보틱스 커뮤니티, <https://cafe.naver.com/openrt>

QnA

E-mail : cmin87394@gmail.com

Mobile : 010-3357-8739

Thank you.

